Automatic Derivation of Feynman Rules from Symbolic Model Declarations

Remo

September 2, 2026

Abstract

Extracting Feynman rules from a Lagrangian is a mechanical but unforgiving task, and for a theory such as the Standard Model Effective Field Theory, with several hundred independent Wilson coefficients, it can no longer be realistically carried out by hand. The established solution is FeynRules, a Mathematica package that derives vertices from a declarative model file. This thesis asks whether the same functionality can be provided inside the Python ecosystem, and answers this by developing FeynPy: a toolkit in which indices, fields, gauge groups and parameters are declared as ordinary Python objects, the Lagrangian is written in a notation close to the one used on paper, and tree-level Feynman rules are returned as symbolic expressions.

FeynPy is built on Symbolica for symbolic algebra and Spenso for typed tensor indices. It resolves implicit indices, expands covariant derivatives and field strengths, handles gauge fixing and ghosts, and derives vertices through a common canonical-quantisation pipeline. The resulting expressions are canonicalised into a normal form suitable for comparison. The same representation also keeps the compiled Lagrangian programmable: fields can be transformed between bases and expanded into components while preserving derivatives, conjugation and fermion ordering, and the final vertices remain fully manipulable symbolic expressions.

The implementation is benchmarked by direct algebraic comparison with FeynRules exports. The packaged Standard Model agrees at 163/163 vertices, the unbroken background-field Standard Model at 67/67, and the Green-basis SMEFT at 184/184 reference lines, with convention differences tracked explicitly. The performance gain is substantial: in the SMEFT benchmark, producing the FeynRules reference takes about half an hour, whereas FeynPy regenerates all 3–6-point SMEFT vertices in 8.20 s and completes the full algebraic comparison in 56.91 s.

Contents

Chapter 1

Introduction

1.1 From a Lagrangian to a prediction

In particle physics, we are always searching for new theories, and the key to describing a system is its Lagrangian. A theory only becomes meaningful when it is corroborated by many experiments, and the experimentalist who wants to test that theory needs theoretical predictions: inclusive cross sections, differential distributions and other quantities that can be measured in a particle accelerator. The first step towards a bridge between the theory and those measurable quantities is the set of Feynman rules, the elementary vertices from which each amplitude is assembled.

This conversion is purely mechanical and algebraic. Section 2.5 describes the algorithm in detail, but we can briefly summarize it as follows: take a term of the Lagrangian, add a creation operator for each field, commute the operators, replace each derivative with a momentum, and eliminate external wave functions. Precisely because these operations are always the same, it is so prone to being implemented algorithmically. Clearly, there are many ramifications and complications: each field carries Lorentz indices, spinors, color, weak isospin, and flavor that must be contracted with the correct partner; each covariant derivative expands into a term for each gauge group under which the field transforms; each field strength of a non-Abelian group splits into three parts, so a product of n of them splits into 3^n; each exchange of two fermions costs a sign, the convention of which must be established once and never violated again.

For a Lagrangian with few terms and few fields, performing these operations manually can already be quite tedious. For the Standard Model, it is a well-known exercise but indeed error-prone as the terms are numerous. Then we have theories that are both more complex and far larger, such as the Standard Model Effective Field Theory (SMEFT). This is the model used as the main benchmark for this thesis, which contains 32 Lagrangian blocks and 298 independent Wilson coefficients, and generates 184 distinct

Achilleas: [DONE] experiments (many)

Achilleas: [DONE] needs theoretical predictions: inclusive cross sections, differential distributions etc.

Achilleas: [DONE] the bridge -> the first step towards a bridge

Achilleas: [DONE] recipe->algorithm

Achilleas: [DONE] momentum!

reference vertices with three to six external legs. Nowadays, no one derives it manually, verifies it manually, or, more precisely, modifies an operator and recalculates it manually.

Achilleas: [DONE] Nowadays,..

With the creation of new theories that have become increasingly complex, it became necessary to automate this process. Thus, FeynRules was born, a model that uses Mathematica to calculate Feynman's rules given a theory and its Lagrangian. Years passed, and many physicists wanted to stop working with mathematics, and thus the idea for this thesis was born: to develop a toolkit capable of calculating Feynman's rules in Python.

1.2 The state of the art

The reference implementation of this idea is FeynRules [1, 2], a Mathematica package that has been a standard entry point for building models for collider phenomenology for over a decade. Its workflow is the one this thesis deliberately imitates, so it is worth specifying explicitly. The FeynRules workflow is briefly summarized below.

First, a FeynRules model file is created, which has a declarative structure. The user fills `M$ClassesDescription` with the particle content, specifying for each field its spin, self-conjugation, indices, and quantum numbers; they fill `M$Parameters` with the parameters and mixing matrices; and declare the gauge groups in `M$GaugeGroups`, each with its own coupling, gauge boson, structure constants, and representations. The Lagrangian is then written as a normal Mathematica expression, using a small vocabulary of symbols: `DC[...]` for a covariant derivative, `FS[...]` for the field-strength tensor, `del[...]` for an ordinary derivative, and `Ga[...]` for a Dirac matrix. A single call to `FeynmanRules[...]` returns the vertices, and a further call exports the model in the UFO format, which is used by event generators such as MadGraph5_aMC@NLO.

There are two main reasons why many physicists may wish to move away from FeynRules. The first is that it is based entirely on Mathematica. This presents some inconveniences, the main one being that it does not integrate naturally with the Python-based machine-learning and analysis stack that the field now relies on for much of its other work. The second issue is computational cost. When the theory becomes complicated or the Lagrangian contains a large number of terms, the required computation time can become very high. As an example, this thesis aimed to implement the SMEFT model, which in FeynRules requires about half an hour of wall-clock time to export its reference vertices, as recorded in Table 6.5.

The gap this thesis aims to fill, therefore, isn't that the problem is unsolved or that a suitable toolkit doesn't already existnot at all. The point is that, until recently, there was no equivalent solution available in the Python ecosystem, where much of the remaining work is taking place.

1.3 Why Python, and why now

For a long time, Python was not an adequate environment for creating such a toolkit that could perform algebraic operations and, more importantly, understand the meaning of symbolic expressions such as tensor networks with typed indices, expanded over thousands of terms.

This changed, however, with the birth of Symbolica, a computer algebra system with a core written in Rust and, more importantly, a Python interface. It provides efficient manipulation of large symbolic expressions through pattern-based rewriting and a fast multivariate polynomial engine, allowing expressions containing thousands of terms to be transformed and normalized efficiently. What Symbolica does not provide by itself is the type information needed to interpret tensor indices.

Building on Symbolica, a very promising and capable library, SPENSO [3], was developed by Lucien Huber. This library adds the missing type information: it makes an index not simply a name, but part of a slot that also carries a representation, so that the library itself knows that a fundamental color index can contract with its dual and not with a Lorentz index. A complementary layer, IDENSO, provides the additional physical identities: metric contraction, the SU(N) color algebra, and the Dirac algebra.

Together, these three libraries provide exactly the services that a FEYNRULES-like tool needs and that Python previously lacked. Chapter 4 describes them in detail. This thesis is, in a nutshell, an attempt to see how far these libraries can go by further developing them.

1.4 Contributions

This thesis presents FEYNPY, a Python toolkit that derives tree-level Feynman rules from declarative model definitions. Its contributions are the following.

1. **A declarative model language in plain Python.** Indices, fields, parameters, gauge representations and gauge groups are ordinary Python objects, and a Lagrangian is written as an ordinary Python expression in a notation deliberately close to the FEYNRULES one (Chapter 3).

2. **A generic gauge compiler.** Covariant derivatives, field strengths, gauge-fixing terms and Faddeev–Popov ghosts are expanded from the model's own gauge metadata.

3. **A contraction engine with a controlled fermion-sign convention.** Vertices are produced by a pruned sum over permutations of the external legs, the direct realisation of the canonical-quantisation recipe.

4. **A canonicalisation layer that makes cross-tool comparison possible.** Dummy relabelling, tensor symmetries, a deterministic generator-ordering rewrite and a restricted use of the Jacobi identity reduce two independently derived expressions to a common normal form (Sections 5.7 and 6.2).

5. **An operator calculus on compiled Lagrangians.** Operators can be applied to an already compiled Lagrangian. Field transformations implement electroweak symmetry breaking and the background-field split, and graded operators implement infinitesimal gauge variations and BRST transformations (Sections 3.13 and 3.18).

Achilleas: [DONE] Stuff → Operations

6. **A model-agnostic comparison framework, and three validated benchmarks.** The packaged Standard Model reproduces its FeynRules export at 163/163 vertices; the unbroken background-field Standard Model at 67/67; and the Green-basis SMEFT at 184/184 reference lines, of which 161 match directly and the rest after documented charge-conjugation conventions (Chapter 6).

1.5 Scope and limitations

Finally, I would like to clearly specify the scope of this work so that the results in Chapter 6 are interpreted correctly.

FeynPy derives Feynman rules exclusively at tree level. It does not support loop corrections, counterterms, restriction files, or export to formats such as UFO, FeynArts, or CalcHEP. Integration by parts is currently limited to scalar fields.

As a result, validation is limited to the interaction vertices and does not cover the full model metadata or numerical parameter definitions.

Within these limits, however, the result is significant: for three complete, independently written models, the vertices produced by FeynPy agree algebraically with those produced by FeynRules. The precise scope and coverage of this validation are detailed in Section 6.9.

1.6 Outline

Chapter 2 aims to provide the theoretical foundation for understanding the canonical quantization, which is the functional basis of the algorithm implemented by feynpy. The chapter will address topics such as the role of symmetry, the Lie algebra mechanism that assigns fields to their indices, the structure of gauge Lagrangians, including gauge fixing, ghosts and spontaneous symmetry breaking, the logic of effective field theory and the SMEFT basis used later, and finally the canonical quantization algorithm that transforms a Lagrangian term into a vertex.

Chapter 3 serves as a user's manual. It builds a model from indices and fields to a complete QCD Lagrangian with gauge fixing, extracts its vertices. Additionally it describes the additional operations available on a compiled Lagrangian: field transformations in a physical basis, gauge variations, BRST transformations, and the included Standard Model and SMEFT implementations.

Chapter 4 introduces SYMBOLICA, SPENSO, and IDENSO and explains the small interface object that connects them to the toolkit.

Chapter 5 explains how the code works internally. It begins by explaining how to move from declarative classes to Wick's contraction engine and canonization, and then output the expression in Symbolica.

Chapter 6 provides validation between the models implemented in FeynRules and their counterparts in FeynPY. It then reports the three benchmark comparisons, examines the charge conjugation and chirality conventions that required attention, and demonstrates, through mutation tests, that the comparison is nontrivial.

Chapter 7 summarizes what has been accomplished, what proved challenging, and the next steps for the work.

Chapter 2

Theoretical Background

2.1 The beauty of Symmetry

Over the centuries, physics has developed increasingly with the aim of understanding and describing the universe and its nature. One might think that mathematical complexity has increased steadily in order to describe the universe, and in a sense this is true, but how far have we gone? Is there a limit to the complexity we can achieve? Is there a risk, however remote, that physicists might lose control of this mathematical machine, which is becoming ever more complex and detached from any conceivable reality and, ultimately, from the physical world?
To answer this question, I would like to address the topic in physics that has most captured the author's attention: *the role of symmetry.*

Simplicity Behind Complexity

'The bigger the puzzle, the harder it is to solve.'

There is certainly some truth in this statement. To put it into context, let's consider an example. If I wanted to describe a room, I would have to refer to every object inside it, and then specify its position and orientation. In short, I could spend hours describing a room in minute detail. If I then wanted to describe the entire flat, then move on to the building, and then go on to describe a city, the task would become increasingly complex. At this point, the more attentive readers will already have guessed where I'm heading.

Describing the universe seems an impossible task, unless we find a way to simply represent something far greater than anything simple. This is where physicists' interest in symmetries stems from.

Let's return to a concrete example: a beehive. I could make the task of describing it very long and tedious, scrutinising every single detail that makes it up. Or I could note a pattern that extends throughout the entire

beehive. All I need to do is describe a single hexagon and then observe that, by symmetry, the rest of the beehive follows the same structure.

But what exactly is symmetry? We can define it in various ways; let's look at two of them. From an intuitive point of view, symmetry is a pattern that exhibits a certain regularity in space and time. From a physical point of view, we can define it as a property that remains unchanged following a transformation.

Thus begins the 'hunt for symmetries' by physicists. I would like to mention some of the most common ones in physics. If we think of a wheel, it possesses rotational symmetry about an axis, the one passing through its hub. Another fundamental symmetry is temporal symmetry: if we shift a system forwards or backwards in time, the laws of physics must remain the same. Without this symmetry, physics would be unable to make predictions and therefore formulate universal laws.

Nature possesses many other symmetries. There is, for example, reflection symmetry: like in a mirror, it allows us to recognise when a configuration remains equivalent to its reflected image.

In the next section, we will delve into the mathematics that allows us to describe this fundamental concept, which underpins the evolution of physics and, with it, our understanding of the universe.

2.2 Group Theory

The previous section introduced symmetry as the invariance of a physical system under a transformation. We now need a mathematical language to describe such transformations and to understand how they affect the fields of a theory. This language is provided by group theory.

For the purposes of this thesis, the important point is that fields are not defined solely by their spin and particle content. They can also have indices, such as Lorentz, spinor, color, or weak isospin indices. These indices identify the components of the spaces into which the fields transform. In a gauge theory, for example, a quark has a color index because it transforms into the fundamental representation of $SU(3)_C$, while a gluon has an additional color index because it transforms into the adjunct representation. More generally, the representation of a symmetry group determines how a field transforms and thus how its indices can appear and be contracted.

This is also the perspective adopted by model-building tools. In FeynRules, for example, fields are declared along with the indices they carry and the representations on which the corresponding gauge group generators act.

FeynPy follows the same physical logic, albeit with its own internal representation of these objects.

The goal of the following sections is therefore to provide a foundation for understanding group theory and the meaning of indices. We will intro-

duce Lie groups and Lie algebras, their generators and structure constants, and finally their representations. These elements will allow us to precisely understand what the indices carried by quantum fields represent and how symmetry constrains the shape of the Lagrangian.

2.2.1 Lie Groups and Lie algebras

We start by defining Lie groups, Lie algebras and their relationship.

Definition 2.2.1 (Lie group)**.** A Lie group G is a group that is also a smooth manifold, such that the group multiplication $G \times G \to G$ is a smooth map.

Lie groups are curved manifolds which makes them hard to study, therefore we look at the tangent space at the identity which, equipped with a natural product is called Lie algebra.

Definition 2.2.2 (Lie algebra)**.** The Lie algebra $\mathfrak{g}$ associated to a Lie group G is the tangent space at the identity element $1 \in G$:

$$\mathfrak{g} := T_1 G. \tag{2.1}$$

Elements of $\mathfrak{g}$ should be thought of as infinitesimal group elements, in a more physical wording they are generators of infinitesimal transformations.

Having passed from the Lie group G to its Lie algebra $\mathfrak{g} = T_1 G$, we now ask whether this can be reversed: is there a map that takes us back from $\mathfrak{g}$ to G, at least in a neighbourhood of the identity? Such a map indeed exists and is called the exponential map.

Definition 2.2.3 (Exponential map)**.** The exponential map is the smooth map from a neighbourhood of $0 \in \mathfrak{g}$ to a neighbourhood of $1 \in G$ satisfying

$$\exp(0) = 1, \qquad d\exp(0) = \mathrm{id}, \qquad \exp(na) = \exp(a)^n. \tag{2.2}$$

It can be constructed explicitly as the limit

$$\exp(a) = \lim_{n\to\infty} \left(1 + \frac{a}{n}\right)^n. \tag{2.3}$$

Definition 2.2.4 (Lie bracket)**.** The Lie bracket $[\![\cdot,\cdot]\!]$ is (twice) the leading deviation of m from linearity, $m_2(a,b) = \frac{1}{2}[\![a,b]\!]$. It can equivalently be recovered from a group commutator of exponentials,

$$[\![a,b]\!] = \lim_{\epsilon\to 0} \frac{1}{\epsilon^2} \exp^{-1}\Big(\exp(\epsilon a)\exp(\epsilon b)\exp(-\epsilon a)\exp(-\epsilon b)\Big). \tag{2.4}$$

The bracket therefore directly measures the failure of G to be commutative; an abelian group has vanishing bracket.

The Lie bracket satisfies three key properties, each of which will be used repeatedly in what follows. It is bilinear, and it is antisymmetric,

$$[\![a, b]\!] = -[\![b, a]\!], \tag{2.5}$$

and it satisfies the Jacobi identity,

$$[\![[\![a, b]\!], c]\!] + [\![[\![b, c]\!], a]\!] + [\![[\![c, a]\!], b]\!] = 0. \tag{2.6}$$

The Jacobi identity is the algebraic shadow of associativity of the group multiplication; it is not an extra assumption but a consequence of G being a group.

Definition 2.2.5 (Lie algebra)**.** A vector space $\mathfrak{g}$ equipped with a bracket satisfying bilinearity, antisymmetry and the Jacobi identity is called a Lie algebra. Conversely, the exponential map associates to every such $\mathfrak{g}$ a (simply connected) Lie group G.

From here on we work directly with the algebraic data $(\mathfrak{g}, [\![\cdot, \cdot]\!])$ rather than with the group manifold itself.

2.2.2 Basis, generators, structure constants

Now we introduce a key concept that will allow us to understand how indices actually work. If we decompose everything with a chosen basis $\{T_a\}$ of $\mathfrak{g}$, we get some useful notation, in particular for the lie bracket. The elements $\{T_a\}$ are called generators of of $\mathfrak{g}$. In physics the basis is typically chosen to be imaginary

$$a = i\, a^c T_c \in \mathfrak{g}, \tag{2.7}$$

This convention makes T_a hermitian which is nice because it guarantees real eigenvalues. The downside is that we get many factors of i. For instance, a group element is parametrised as

$$h = \exp\big(i\, a^c T_c\big) \in G. \tag{2.8}$$

Definition 2.2.6 (Structure constants)**.** Expanding the Lie bracket of two generators in the chosen basis defines the structure constants $f_{ab}{}^c$ of the algebra,

$$[\![T_a, T_b]\!] = i f_{ab}{}^c\, T_c. \tag{2.9}$$

For a real Lie algebra $f_{ab}{}^c$ must be real numbers, and antisymmetry of the bracket implies

$$f_{ab}{}^c = -f_{ba}{}^c. \tag{2.10}$$

This index a, labelling a generator T_a, is what we will call an index or, once specialised to $SU(N)$ gauge theory, a colour index. Once we pass to representations, these same structure constants will reappear as the commutation relations of the representation matrices T_a^R.

2.2.3 Representation

In Yang–Mills theory fields have particular transformation properties under gauge transformations. Mathematically these transformation rules are described by representations.

Definition 2.2.7 (Representation of a Lie group)**.** A representation R of G is a map from the group to automorphisms of some vector space V,

$$R : G \to \mathrm{Aut}(V). \tag{2.11}$$

Definition 2.2.8 (Representation of a Lie algebra)**.** A representation R of $\mathfrak{g}$ is a linear map from the algebra to endomorphisms of some vector space V,

$$R : \mathfrak{g} \to \mathrm{End}(V), \tag{2.12}$$

such that the Lie bracket is reproduced as an operator commutator,

$$R\big([\![a, b]\!]\big) = [R(a), R(b)]. \tag{2.13}$$

Writing $T_a^R := R(T_a)$ for the representation matrices of the generators, the representation property of R takes the explicit form

$$[T_a^R, T_b^R] = i f_{ab}{}^c T_c^R. \tag{2.14}$$

A field ϕ transforming in the representation R carries an index $i = 1, \ldots, \dim R$ labelling a basis of the representation space V, and an infinitesimal gauge transformation acts on it as

$$\delta\phi^i = i a^c (T_c^R)^i{}_j \phi^j. \tag{2.15}$$

This is the sense in which internal indices in quantum field theory, such as colour or weak-isospin indices, are representation indices of the underlying symmetry algebra.

Now we want to study a special representation: the adjoint representation. Its feature is that the Lie algebra acts on itself.

Definition 2.2.9 (Adjoint representation)**.** A distinguished representation of any Lie algebra $\mathfrak{g}$ is the adjoint representation on the Lie algebra itself:

$$\mathrm{ad} : \mathfrak{g} \to \mathrm{End}(\mathfrak{g}), \qquad \mathrm{ad}(a)b \coloneqq [a, b]. \tag{2.16}$$

In other words, an element of the algebra acts on another element by taking the commutator. If T_a is a basis of the Lie algebra, then

$$T_a^{ad} T_b = \mathrm{ad}(T_a) T_b = [\![T_a, T_b]\!] = i f_{ab}{}^c T_c. \tag{2.17}$$

Therefore, the adjoint representation T_a^{ad} is a matrix whose elements are the structure constants

$$(T_a^{ad})_b{}^c = i f_{ab}{}^c, \tag{2.18}$$

Thus, an adjoint index is an index labelling a basis element of the Lie algebra. For example, the gauge field can be written as

$$A_\mu = A_\mu^a T_a. \tag{2.19}$$

Here μ is a Lorentz index, while a is an adjoint index. The index a labels which generator of the gauge algebra the field component is associated with.

2.2.4 Invariants

Lie algebras possess an invariant object which lets us raise and lower algebra indices, in the same way that a metric lets us raise and lower spacetime indices.

Definition 2.2.10 (Killing form)**.** A simple Lie algebra $\mathfrak{g}$ has a unique invariant symmetric bilinear form $K : \mathfrak{g}\times\mathfrak{g} \to K$, the Killing form, satisfying

$$K(a,b) = K(b,a), \qquad K\big([\![c,a]\!],b\big) + K\big(a,[\![c,b]\!]\big) = 0 \tag{2.20}$$

for all $a, b, c \in \mathfrak{g}$.

A form with this property can be constructed easily using some representation R

$$TrR(a)R(b) = B^R K(a,b) \tag{2.21}$$

The property in equation 2.20 follow from cyclicity of the trace. Due to uniqueness of K in a simple Lie algebra, all these forms must be equivalent up to a factor of proportionality B^R which depends on the particular representation R. In the basis of generators, the Killing form has components

$$K(T_a, T_b) = k_{ab}. \tag{2.22}$$

For a real, compact, simple Lie algebra k_{ab} is positive-definite and non-degenerate, and can therefore be used to raise and lower the algebra index a. In particular we may lower an index on the structure constants,

$$f_{abc} := f_{ab}{}^d\, k_{dc}, \tag{2.23}$$

which are then totally antisymmetric in all three indices,

$$f_{abc} = -f_{bac} = -f_{acb}. \tag{2.24}$$

A common simplifying choice of basis sets $k_{ab} = \delta_{ab}$, so that upper and lower algebra indices coincide numerically.

2.2.5 The Unitary Algebra $\mathfrak{su}(N)$

We specialise the index calculus developed above to the case of $\mathfrak{su}(N)$, the gauge algebra of quantum chromodynamics for $N = 3$.

Definition 2.2.11 (Defining representation)**.** $\mathfrak{su}(N)$ acts naturally on $\mathbb{C}^N$ by matrix multiplication,

$$\mathrm{def}(A)\, v = Av, \qquad v \in \mathbb{C}^N, \qquad D^{\mathrm{def}} = N. \tag{2.25}$$

This is the representation carried by matter fields such as quarks; the corresponding index, written $i = 1, \ldots, N$, is the colour index. It is a different kind of index from the adjoint index a carried by the gauge field, and the two are related by the generators T_a^{def}, which carry one of each, $(T_a^{\mathrm{def}})^i{}_j$.

Together with the identity matrix, the generators T_a^{def} form a complete basis of $N \times N$ matrices. This completeness is expressed by the relation

$$k^{ab}(T_a^{\mathrm{def}})^i{}_j(T_b^{\mathrm{def}})^k{}_l = B^{\mathrm{def}}\left(\delta^i_l\delta^k_j - \frac{1}{N}\delta^i_j\delta^k_l\right), \tag{2.26}$$

and it is this identity that allows a sum over the adjoint index a to be traded for an expression written purely in terms of Kronecker deltas on the colour indices i, j, k, l. This is a manipulation used throughout when working with $SU(N)$ gauge theories.

2.3 Gauge Theories and Their Lagrangians

Now that we understand the mathematics needed to describe symmetries and transformations, we can investigate the fundamental object that allows us to describe a system and all its characteristics: the Lagrangian of a gauge theory.

2.3.1 Fields and the indices they carry

Fields can carry different kinds of indices, depending on how they transform. For our purposes, it is useful to distinguish three main families: spacetime, gauge and flavour indices.

Spacetime transformations determine the Lorentz structure of a field. A scalar ϕ carries no spacetime index, a Dirac fermion ψ_α carries a spinor index α, and a vector field A_μ carries a Lorentz index μ. Objects such as the Dirac matrices $\gamma^\mu_{\alpha\beta}$ connect these structures, carrying one Lorentz index and two spinor indices.

Internal gauge transformations determine the representation of a field under a gauge group, as discussed in Section 2.2. A quark q^i carries a colour index in the fundamental representation of $\mathrm{SU}(3)_C$, while a gluon G^a_μ carries

an adjoint colour index. Similarly, a left-handed lepton doublet ℓ_L^j carries an $\mathrm{SU}(2)_L$ index. For an abelian group such as $\mathrm{U}(1)_Y$, there is no additional representation index: the transformation is specified by a charge.

Flavour indices distinguish different copies of fields with the same gauge quantum numbers. The three generations of quarks and leptons are the standard example. These indices appear, for instance, in coupling matrices such as the Yukawa matrices Y_u, Y_d and Y_e.

A field may carry several of these indices at the same time. The left-handed quark doublet, for example, carries a spinor index, a colour index, a weak-isospin index and a flavour index. Keeping track of all these indices quickly becomes cumbersome in explicit calculations. The `IndexType` introduced in Section 3.2 provides the machine-readable counterpart of this classification.

Chirality is closely related and will appear repeatedly throughout the thesis. The projectors

$$P_L = \frac{1-\gamma^5}{2}, \qquad P_R = \frac{1+\gamma^5}{2}, \tag{2.27}$$

select the left- and right-handed components of a Dirac field. In the Standard Model these components transform differently under $\mathrm{SU}(2)_L$. It is therefore natural to work directly with chiral fields such as the doublets Q_L and ℓ_L and the singlets u_R, d_R and e_R. When chirality is already part of the field definition, writing an additional projector may be redundant. This point will become relevant again in the validation of Chapter 6.

2.3.2 Local symmetry and the covariant derivative

Let a field ϕ transform in a representation R of a group G, so that under a global transformation

$$\phi \longrightarrow e^{i\alpha^a T_a^R}\,\phi, \tag{2.28}$$

where the parameters α^a are constant. Since the transformation does not depend on spacetime, the derivative transforms in the same way as the field,

$$\partial_\mu\phi \longrightarrow e^{i\alpha^a T_a^R}\partial_\mu\phi. \tag{2.29}$$

This makes it possible to construct Lagrangian terms that are invariant under the global symmetry.

The situation changes when the transformation is made *local*, so that $\alpha^a \to \alpha^a(x)$. The derivative now also acts on the transformation itself, and therefore $\partial_\mu\phi$ no longer transforms like ϕ. To restore the symmetry, we introduce a gauge field A_μ^a, one component per generator, and replace the ordinary derivative by the covariant derivative

$$D_\mu\phi = \partial_\mu\phi - igA_\mu^a T_a^R\phi, \tag{2.30}$$

where g is the gauge coupling. The gauge field transforms in such a way that the additional terms cancel, leaving

$$D_\mu \phi \longrightarrow e^{i\alpha^a(x) T_a^R} D_\mu \phi \tag{2.31}$$

The covariant derivative therefore transforms in exactly the same way as the field itself.

For an abelian group, such as U(1), the generator is simply the charge Q, and the covariant derivative becomes

$$D_\mu \phi = \partial_\mu \phi - igQA_\mu \phi. \tag{2.32}$$

A field may transform under several gauge groups at the same time. In that case, the covariant derivative contains one contribution from each group, with its own coupling, gauge field and generator. For example, the left-handed quark doublet transforms under $\mathrm{SU}(3)_C \times \mathrm{SU}(2)_L \times \mathrm{U}(1)_Y$, and therefore

$$D_\mu Q_L = \partial_\mu Q_L - ig_s G_\mu^a t_{(3)}^a Q_L - igW_\mu^I t_{(2)}^I Q_L - ig' Y_Q B_\mu Q_L. \tag{2.33}$$

The sign in the definition of D_μ is a convention and must be used consistently throughout the theory. The convention adopted in this thesis is the one in Equation (2.30). In the implementation, expanding the covariant derivative into the appropriate contribution from each gauge group is handled automatically, as described in Section 5.4.2.

2.3.3 The field strength and the Yang–Mills Lagrangian

The gauge field needs a kinetic term of its own, and it must be built from a gauge-covariant object. That object is the field strength, defined by the commutator of two covariant derivatives,

$$F_{\mu\nu} = \frac{i}{g} [D_\mu, D_\nu] = F_{\mu\nu}^a T_a, \tag{2.34}$$

whose components read

$$F_{\mu\nu}^a = \partial_\mu A_\nu^a - \partial_\nu A_\mu^a + g\, f^{abc}\, A_\mu^b A_\nu^c. \tag{2.35}$$

The last term is present only for a non-abelian group, and it is the origin of the gauge boson self-interactions: the Yang–Mills Lagrangian

$$\mathcal{L}_{\mathrm{YM}} = -\frac{1}{4} F^{a\,\mu\nu} F_{\mu\nu}^a \tag{2.36}$$

is quadratic in F, and therefore contains a two-gluon kinetic piece, a three-gluon vertex and a four-gluon vertex all at once. For an abelian group the structure constants vanish, Equation (2.35) reduces to $F_{\mu\nu} = \partial_\mu A_\nu - \partial_\nu A_\mu$, and the photon does not interact with itself.

The counting implicit in Equation (2.35) recurs in Section 5.4.3: a non-abelian field strength is a sum of three pieces, so a product of n of them expands into 3^n branches before anything is collected again.

2.3.4 Matter, Yukawa couplings and flavour

Once the covariant derivative has been introduced, the coupling of matter to gauge fields follows naturally. For a Dirac fermion,

$$\mathcal{L}_\psi = i\bar{\psi}\gamma^\mu D_\mu \psi, \qquad \bar{\psi} = \psi^\dagger \gamma^0, \tag{2.37}$$

while for a complex scalar the kinetic term is

$$\mathcal{L}_\Phi = (D_\mu \Phi)^\dagger D^\mu \Phi. \tag{2.38}$$

Expanding the covariant derivative separates the ordinary kinetic term from the interactions between matter and the corresponding gauge fields. In this way, the gauge structure introduced in the previous sections directly fixes the form of these interactions.

In the Standard Model, left- and right-handed fermions transform differently under the electroweak gauge group. A direct Dirac mass term such as $\bar{\psi}_L \psi_R$ is therefore not gauge invariant before electroweak symmetry breaking. Fermion masses instead originate from Yukawa interactions with the Higgs doublet,

$$\mathcal{L}_Y = -(Y_u)_{fg} \bar{Q}_L^f \tilde{\Phi} u_R^g - (Y_d)_{fg} \bar{Q}_L^f \Phi d_R^g - (Y_e)_{fg} \bar{\ell}_L^f \Phi e_R^g + \text{h.c.}, \tag{2.39}$$

where

$$\tilde{\Phi}_i = \varepsilon_{ij} \Phi_j^* \tag{2.40}$$

is the conjugate Higgs doublet and f, g denote flavour indices.

The Yukawa couplings are matrices in flavour space. After the Higgs field acquires a vacuum expectation value, these matrices become the fermion mass matrices and can be diagonalised by suitable field rotations. In the quark sector, the rotations required for the up- and down-type quarks are in general different. Their mismatch gives rise to the CKM matrix and therefore to flavour mixing in the weak interactions.

Finally, fermionic fields are Grassmann-valued. Exchanging two fermionic fields therefore introduces a minus sign,

$$\psi_1 \psi_2 = -\psi_2 \psi_1. \tag{2.41}$$

This simple property becomes important when manipulating fermionic expressions symbolically, where the ordering of the fields must be preserved. The treatment adopted in the implementation is discussed in Section 5.6.2.

2.3.5 Gauge fixing and Faddeev–Popov ghosts

Gauge invariance introduces a redundancy: field configurations related by a gauge transformation describe the same physical state. As a consequence, the quadratic operator of the gauge field contains redundant directions and

cannot be inverted directly to obtain a propagator. To quantise the theory, this redundancy must first be removed by choosing a gauge.

The Faddeev–Popov procedure implements this choice in the path integral. In a linear covariant gauge, one introduces the gauge-fixing term

$$\mathcal{L}_{\text{gf}} = -\frac{1}{2\xi}\left(\partial^\mu A^a_\mu\right)^2, \tag{2.42}$$

where ξ is the gauge parameter. The choices $\xi \to 0$ and $\xi = 1$ correspond to Landau and Feynman gauge, respectively. Intermediate expressions may depend on ξ, but physical observables must not, providing a useful consistency check.

Gauge fixing also introduces a Jacobian in the path integral. For a non-abelian gauge theory this Jacobian is represented by the Faddeev–Popov ghost fields, with Lagrangian

$$\mathcal{L}_{\text{gh}} = -\bar{c}^a \partial^\mu (D_\mu c)^a, \qquad (D_\mu c)^a = \partial_\mu c^a + g f^{abc} A^b_\mu c^c. \tag{2.43}$$

The ghost fields c^a and $\bar{c}^a$ are Lorentz scalars carrying an adjoint gauge index, but they are Grassmann-valued and therefore obey fermionic statistics. They do not correspond to physical particles instead, they compensate for the unphysical degrees of freedom introduced by the gauge-fixing procedure.

For an abelian theory in a linear covariant gauge, the structure constants vanish and the ghosts do not interact with the gauge field.

2.3.6 BRST symmetry

Fixing the gauge destroys gauge invariance, but not all of it. What survives is a global, Grassmann-odd symmetry of the gauge-fixed action, generated by the BRST operator s:

$$sA^a_\mu = (D_\mu c)^a, \qquad sc^a = -\tfrac{g}{2} f^{abc} c^b c^c, \tag{2.44}$$

$$s\bar{c}^a = B^a, \qquad sB^a = 0, \tag{2.45}$$

with B^a an auxiliary field, and $s\phi_i = ig\, c^a (T^a)_{ij}\phi_j$ on matter. Its defining property is nilpotency, $s^2 = 0$, and its practical use here is that the entire gauge-fixing and ghost sector can be written as a BRST variation,

$$\mathcal{L}_{\text{gf}} + \mathcal{L}_{\text{gh}} = s\left(\bar{c}^{\,a}\, \partial^\mu A^a_\mu + \frac{\xi}{2}\, \bar{c}^{\,a} B^a\right), \tag{2.46}$$

which makes the invariance of the full action manifest. Because s is an operator acting on the fields of a Lagrangian, it is a natural thing to implement as a rewriting rule; Section 3.18 does exactly that and verifies $s^2 = 0$ symbolically.

2.3.7 Spontaneous symmetry breaking and the physical basis

The Standard Model is written in a basis that does not describe observable particles. The electroweak gauge group $\mathrm{SU}(2)_L \times \mathrm{U}(1)_Y$ has gauge fields W_μ^I and B_μ, none of which is the photon, and all matter fields are massless because chiral gauge invariance forbids mass terms.

The resolution is that the Higgs doublet Φ has a potential

$$V(\Phi) = \mu^2 \Phi^\dagger \Phi + \lambda (\Phi^\dagger \Phi)^2 \tag{2.47}$$

whose minimum for $\mu^2 < 0$ lies away from the origin. the doublet is expanded around that vacuum as

$$\Phi = \begin{pmatrix} G^+ \\ \frac{1}{\sqrt{2}} \left(v + H + i G^0 \right) \end{pmatrix}, \tag{2.48}$$

with H the Higgs boson, $G^\pm, G^0$ the Goldstone bosons and v the vacuum expectation value. The gauge fields recombine into the physical mass eigenstates,

$$W_\mu^\pm = \frac{1}{\sqrt{2}} \left(W_\mu^1 \mp i W_\mu^2 \right), \qquad \begin{pmatrix} Z_\mu \\ A_\mu \end{pmatrix} = \begin{pmatrix} c_W & -s_W \\ s_W & c_W \end{pmatrix} \begin{pmatrix} W_\mu^3 \\ B_\mu \end{pmatrix}, \tag{2.49}$$

where s_W, c_W are the sine and cosine of the weak mixing angle and satisfy $s_W^2 + c_W^2 = 1$, an identity that reappears in Section 6.3.

For this thesis the important observation is structural. Every step above is a *substitution*: replace B_μ by a combination of Z_μ and A_μ, replace one component of Φ by a constant plus H plus iG^0, replace Q_L by a projector times a rotated field. A toolkit that can perform such substitutions on an already-compiled Lagrangian can therefore handle symmetry breaking without any dedicated symmetry-breaking code, which is the design adopted in Section 3.13.

2.3.8 The background field method

2.4 Effective Field Theory and the SMEFT

The most demanding benchmark of this thesis is not the Standard Model but an effective field theory built on top of it. This section fixes the ideas and the vocabulary that Chapter 6 later uses without further explanation.

2.4.1 The effective expansion

Suppose new physics exists at some scale Λ well above the energies currently accessible. Its effects at low energy can then be described without knowing

what it is, by supplementing the Standard Model Lagrangian with all operators that respect its symmetries and are built from its fields, ordered by mass dimension:

$$\mathcal{L}_{\text{SMEFT}} = \mathcal{L}_{\text{SM}} + \sum_i \frac{C_i^{(5)}}{\Lambda} \mathcal{O}_i^{(5)} + \sum_i \frac{C_i^{(6)}}{\Lambda^2} \mathcal{O}_i^{(6)} + \dots \tag{2.50}$$

Each operator comes with a *Wilson coefficient* C_i: a number, or, when the operator carries flavour indices, a tensor, encoding the unknown short-distance physics. Higher-dimension operators are suppressed by more powers of Λ, so the series can be truncated.

At dimension five there is only one gauge-invariant structure, the Weinberg operator, which violates lepton number and generates neutrino masses; it appears in the validation of Section 6.6.2. At dimension six the count explodes, and this is where the automation argument of Section 1.1 becomes decisive.

2.4.2 Bases, redundancy and the Green basis

Not every operator one can write is independent. Three relations reduce the list. Integration by parts discards total derivatives. Fierz identities relate different orderings of four-fermion structures. Most importantly, the leading-order equations of motion can be used to eliminate operators, because an operator proportional to the equations of motion can be removed by a field redefinition and therefore does not affect S-matrix elements.

Applying all three yields a minimal, non-redundant set. The standard such set at dimension six is the *Warsaw basis*, which contains 59 independent operators for one fermion generation, and 2499 once flavour indices are counted. It is the basis in which most phenomenological analyses are expressed. An earlier, redundant enumeration was given in.

For some purposes, however, one deliberately does *not* reduce. A *Green basis*, also called an off-shell basis, retains the operators that the equations of motion would have removed. This is redundant by construction, and that is the point: off-shell quantities such as Green functions, and the intermediate steps of matching and renormalisation calculations, are not invariant under the field redefinitions used to reach the Warsaw basis. Working in a Green basis first, and reducing to a physical basis afterwards, keeps those intermediate steps meaningful.

The model validated in Chapter 6 is of exactly this kind. It is a baryon-number-preserving Green-basis SMEFT with 298 active Wilson coefficients spread over 32 Lagrangian blocks — larger than the Warsaw basis precisely because it is redundant.

2.4.3 Evanescent operators

One last complication comes from dimensional regularisation. Calculations are performed in $d = 4 - 2\epsilon$ dimensions, where some identities of the four-dimensional Dirac algebra — for example those used to Fierz-reorder products of fermion currents — no longer hold.

There are different conventions for what is called an *evanescent operator*. Here we use the convention of the reference model: an operator is called evanescent if, although independent for general d, it can be reduced to one of the physical operators in the limit $d \to 4$. The difference between the d-dimensional structure and its four-dimensional reduction therefore vanishes in this limit.

These structures cannot in general be discarded in loop calculations. Their $\mathcal{O}(\epsilon)$ difference from the corresponding physical operators can combine with the $1/\epsilon$ poles of loop integrals and leave finite contributions. In the reference model they appear as a family of four-fermion operators whose Wilson coefficients are named with the prefix `alphaEc`. These are also exactly the operators for which the two implementations package charge conjugation differently, as discussed in Section 6.6.

Achilleas: [DONE]There are two different definitions of evanescent operators. The commonly used one is that an operator is evanescent if it vanishes as $d \to 4$. The alternative is that evanescent is an operator that can be reduced to one of the physical ones at $d \to 4$. We use the second definition.

2.5 From the Lagrangian to Feynman Rules

In this section we want to explain but algorithm has been used to calculate Feynamn Rules given a lagrangian. If the reader lacks of kwoledge in QFT he is very welcomed to read a good textbook on the topic. To fully understand the algorithm the reader should be familiar with canonical quantization, the interaction picture, or Wick's theorem, we assume this material here and use it only to fix notation.

In the previous section, we learnt about the role of field indices and Lagrangians. We can therefore move on to the central question: given a Lorentz-invariant and gauge-invariant Lagrangian, how do we obtain the associated Feynman rules? We wish to find an implementation that solves this in a systematic and mechanical way, so that it is suitable for implementation in Python or any programming language. This implementation has already been provided by FeynRules [1], and the algorithm described below reproduces, with some additional details, the derivation given in Section 9 of that article.

2.5.1 Mode Expansion and Canonical (Anti)Commutators

Let's start by considering a free field ϕ^α, where the index α collects all indices and quantum numbers carried by the field (Lorentz, Dirac, and internal). In the interaction picture, we can ϕ^α expand the field in creation

and annihilation operators,

$$\phi^{\alpha}(x) = \int d\Pi(p) \left(u^{\alpha}(p)\, a_{\phi}(p)\, e^{-ipx} + v^{\alpha}(p)\, a^{\dagger}_{\bar{\phi}}(p)\, e^{+ipx} \right), \tag{2.51}$$

where $u^{\alpha}(p)$, $v^{\alpha}(p)$ are the wavefunctions associated to ϕ^{α}, $a_{\phi}(p)$ annihilates ϕ, and $a^{\dagger}_{\bar{\phi}}(p)$ creates a conjugated field $\bar{\phi}$. Canonical quantization fixes the (anti)commutation relations between fields and creation operators; schematically,

$$\left[\phi^{\alpha}(x),\, a^{\dagger\beta}_{\bar{\phi}}(p)\right]_{\pm} = \delta_{\alpha\beta} u^{\alpha}(p)\, e^{-ipx}, \tag{2.52}$$

with a commutator for bosonic fields and anticommutator for fermionic fields.

2.5.2 The FeynRules Algorithm

In order to understand the algorithm let's start by considering a general single term of a Lagrangian

$$\mathcal{L} \supset g_{\alpha_1 \cdots \alpha_n}\, \partial \cdots \partial\, \phi^{\alpha_1} \cdots \phi^{\alpha_n}, \tag{2.53}$$

where α_i represents all the indices and quantum numbers of the field ϕ^{α_i}. $g_{\alpha_1 \cdots \alpha_n}$ is a coupling constant (possibly carrying its own indices), and $\partial \cdots \partial$ denotes some number of derivatives acting on one or more of the fields. The corresponding Feynman rule is obtained in three steps.

1. **Attach creation operators.** Multiply the term on the right by one creation operator for each field, with all momenta taken ingoing, and in the reverse order to that in which the (fermionic and ghost) fields appear in the Lagrangian term. Sandwich the result between vacuum states,

$$g_{\alpha_1 \cdots \alpha_n}\, \partial \cdots \partial \left\langle 0 \right| \phi^{\alpha_1}(x) \cdots \phi^{\alpha_n}(x)\, a^{\dagger}_{\bar{\phi}_n}(p_n) \cdots a^{\dagger}_{\bar{\phi}_1}(p_1) \left| 0 \right\rangle. \tag{2.54}$$

2. **Contract creation operators to the left.** Now every creation operator is moved to the left, using the (anti)commutation rules of the fields given by equation 2.52. Note that moving $a^{\dagger}$ to the left of a boson produces a commutator term; moving it past a fermion produces an anticommutator term and a sign from reordering the remaining operators. Concretely, each step replaces a pair $\left(\phi^{\alpha_i}(x), a^{\dagger}_{\bar{\phi}_j}(p)\right)$ by the contraction

$$\delta_{\alpha_i \beta_j}\, u^{\alpha_i}(p)\, e^{-ipx}, \tag{2.55}$$

   Once a creation operator reaches the leftmost position, it annihilates the vacuum $\langle 0|$ and is removed.

3. **Strip the overall factors.** After every creation operator has been contracted away, the surviving vacuum overlap is $\langle 0|0\rangle = 1$. The remaining derivatives act on the exponentials $e^{-ip_i x}$, pulling down the corresponding momenta. Dropping the overall exponential $\exp\big(-i(p_1+\cdots+p_n)x\big)$ and the external wavefunctions $u^{\alpha_i}(p_i)$, and multiplying the result by i, yields the Feynman rule associated to the vertex.

This procedure is nothing other than an automated application of Wick's theorem to a single interaction term.

2.5.3 Example: the QED Vertex

Now we want to see how this algorithm applies to a Lagrangian. AS in [1] we illustrate the algorithm on the photon-lepton interaction term of the QED Lagrangian,

$$\mathcal{L}_{\bar\psi\psi A} = -e\,\bar\psi\gamma^\mu\psi\,A_\mu = -e\,\bar\psi_{s,f}\,\gamma^\mu_{ss'}\,\psi_{s',f}\,A_\mu, \tag{2.56}$$

where e is the electromagnetic coupling, ψ_f the lepton field of flavour $f \in \{e,\mu,\tau\}$, and A_μ the photon field. Multiplying by the creation operators $a^\dagger_{\bar\psi}$, $a^\dagger_\psi$, $a^\dagger_A$ and sandwiching between vacuum states gives

$$-e\,\big\langle 0\big|\,\bar\psi\gamma^\mu\psi\,A_\mu\,a^\dagger_{\bar\psi}\,a^\dagger_\psi\,a^\dagger_A\,\big|0\big\rangle. \tag{2.57}$$

Moving each creation operator to the left using the canonical (anti)commutators,

$$\big\{\psi_{s,f}(x), a^\dagger_{\bar\psi_{s',f'}}(p)\big\} = \delta_{ss'}\delta_{ff'}\,u_{s,f}\,e^{-ipx}, \tag{2.58}$$

$$\big\{\bar\psi_{s,f}(x), a^\dagger_{\psi_{s',f'}}(p)\big\} = \delta_{ss'}\delta_{ff'}\,\bar u_{s,f}\,e^{-ipx}, \tag{2.59}$$

$$\big[A_\mu(x), a^\dagger_{A_\nu}(p)\big] = \delta^\nu_\mu\,\epsilon_\mu\,e^{-ipx}, \tag{2.60}$$

and annihilating the vacuum, yields

$$-e\,\delta_{ff'}\,\gamma^\mu_{ss'}\,\bar u_{sf}(p_1)\,u_{s'f'}(p_2)\,\epsilon_\mu(p_3)\,e^{-i(p_1+p_2+p_3)x}. \tag{2.61}$$

Dropping the wavefunctions and the overall exponential, and multiplying by i, gives the familiar QED vertex,

$$-ie\,\delta_{ff'}\,\gamma^\mu_{ss'}. \tag{2.62}$$

2.6 Conventions

Almost every disagreement between two independently written derivations of a Feynman rule turns out, on inspection, to be a disagreement about

a convention rather than about physics. Chapter 6 is largely an exercise in identifying which convention each of the two implementations uses. It is therefore worth collecting in one place the choices that this work makes and enforces everywhere. Table 2.1 lists them, and is referred back to from Chapters 5 and 6.

Achilleas: [DONE] which ones are used in this work.

Quantity	**Convention**
Metric signature	$g_{\mu\nu} = \mathrm{diag}(+,-,-,-)$
Covariant derivative (matter)	$D_\mu = \partial_\mu - igA^a_\mu T_a$, Equation (2.30)
Field strength	$F^a_{\mu\nu} = \partial_\mu A^a_\nu - \partial_\nu A^a_\mu + g f^{abc} A^b_\mu A^c_\nu$, Equation (2.35)
Adjoint covariant derivative	$(D_\mu X)^a = \partial_\mu X^a + g f^{abc} A^b_\mu X^c$
Generator normalisation	$[T_a, T_b] = i f_{ab}{}^c T_c$, generators hermitian
Fourier convention	all momenta incoming; $\partial_\mu \to -ip_\mu$
Overall vertex factor	every extracted rule is multiplied by $+i$
Chiral projectors	$P_{L,R} = (1 \mp \gamma^5)/2$, Equation (2.27)
Charge conjugation	$C^T = -C$
Fermion ordering	the order in which fermion fields are written in a Lagrangian term is significant and is preserved throughout the pipeline

Table 2.1: The conventions fixed once and used throughout the thesis and the implementation. The signs in the first four rows are choices, not results; what matters is that they are applied uniformly, since a single inconsistency would produce vertices that are individually plausible and collectively wrong.

With the theory, the algorithm and the conventions in place, we can turn to the toolkit itself.

Chapter 3

FeynPy: User Manual

The purpose of this chapter is to serve as a guide to creating a model in FeynPy. Theoretical explanations of the elements that make up the model are provided in the previous chapters, while an overview of how the algorithm works is given in Chapter 5. Here, we focus on the user interface. If we take the following Lagrangian as our starting point:

$$\mathcal{L}_{\text{QCD}} = i\,\bar{q}\gamma^{\mu}D_{\mu}q - \frac{1}{4}F^{a}_{\mu\nu}F^{a\,\mu\nu} - \frac{1}{2\xi}\left(\partial^{\mu}A^{a}_{\mu}\right)^{2} - \bar{c}^{a}\,\partial^{\mu}(D_{\mu}c)^{a}, \tag{3.1}$$

we note that the elements composing this Lagrangian are, first and foremost, fermionic fields $q_{i,c,f}$ carrying Dirac, colour, and flavour indices. Furthermore, we have the field-strength tensor $F^{a}_{\mu\nu}$, which must be expanded as in Equation 5.14, thus giving rise to two point, three point and four point gluon terms. There is also the covariant derivative, whose expansion introduces the interaction between the fermionic and gauge fields. Finally, the Lagrangian contains the gauge-fixing and ghost terms. These expansions are determined by the symmetry group associated with the Lagrangian. Our aim will therefore be to define the basic ingredients of the model: indices, fields, gauge representations and gauge groups, and then assemble them into a complete `Model` object. Once the model is defined, the toolkit can be used to manipulate the Lagrangian, extract Feynman rules and perform further symbolic operations.

The chapter is organised in three movements. Sections 3.1 to 3.8 build a model from its smallest pieces up to a complete `Model`. Sections 3.9 to 3.12 extract vertices from it, ending with the QCD Lagrangian above carried through to its printed rules. The remaining sections (Section 3.13 onwards) cover what can be done to a compiled Lagrangian besides reading vertices off it: rewriting it into a physical basis, exporting it to Symbolica, and acting on it with operators such as an infinitesimal gauge variation or the BRST differential. Section 3.20 closes with the three complete models that ship with the toolkit.

3.1 Core Imports

```
from symbolica import S, Expression
from symbolic.vertex_engine import I
from symbolic.spenso_structures import (
    gauge_generator, structure_constant,
    weak_gauge_generator, weak_structure_constant,
)
from feynpy import (
    Field, GhostField, GaugeGroup, GaugeRepresentation, IndexType,
    Model, Parameter, FieldTransformation,
    # declarative Lagrangian factors
    DC, FS, PartialD, Gamma, Gamma5, Metric, T, StructureConstant,
    GaugeFixing, GhostLagrangian, ProjM, ProjP, rotation,
    # pre-built index constants
    SPINOR_INDEX, LORENTZ_INDEX,
    COLOR_FUND_INDEX, COLOR_ADJ_INDEX,
    WEAK_FUND_INDEX, WEAK_ADJ_INDEX,
    # flavour
    flavor_index,
)
```

Let's start our journey with a quick overview of a minimal complete example, the scalar ϕ^4 theory:

```
Phi   = Field("Phi", spin=0, self_conjugate=True, symbol=S("phi"))
lam   = S("lam")
model = Model(lam * Phi * Phi * Phi * Phi)
L     = model.lagrangian()
V     = L.feynman_rule(Phi, Phi, Phi, Phi)   # -> 24i*lam
```

Four lines: declare a field, write a Lagrangian, compile it, ask for a vertex. Everything that follows in this chapter is an elaboration of those four lines.

3.2 Index Types

Many fields carry indices that define how they transform under a symmetry. For example, a quark $q_{i,c,f}$ carries three indices:

1. a spinor index i,
2. a colour fundamental index c,
3. a flavour index f.

In the toolkit, indices are represented by objects of type `IndexType`. Internally, each `IndexType` stores the index name, pairs a SPENSO `Representation` with a string *kind* that is used to match indices through the compilation pipeline, and the prefix used when dummy indices are generated automatically. Six pre-built constants cover the standard cases and should be used directly whenever possible.

Constant	Representation	kind string
`SPINOR_INDEX`	`bis(4)`	`"spinor"`
`LORENTZ_INDEX`	`mink(4)`	`"lorentz"`
`COLOR_FUND_INDEX`	`cof(3)`	`"color_fund"`
`COLOR_ADJ_INDEX`	`coad(8)`	`"color_adj"`
`WEAK_FUND_INDEX`	`cof(2)`	`"weak_fund"`
`WEAK_ADJ_INDEX`	`coad(3)`	`"weak_adj"`

All six can be imported directly from `feynpy`. The colour adjoint index type, for instance, is defined as:

```
COLOR_ADJ_INDEX = IndexType(
    "ColorAdj", Representation.coad(8), "color_adj", prefix="a"
)
```

Custom index types follow the same pattern, so a new gauge group with a representation Spenso already knows requires no engine changes:

```
from symbolica.community.spenso import Representation
from feynpy import IndexType

# IndexType(name, representation, kind, prefix=...)
MY_FUND = IndexType("MyFund", Representation.cof(3), "my_fund",
↪  prefix="m")
```

Flavor indices are ordinary `IndexType` objects with `is_flavor=True` set automatically by the helper:

```
# flavor_index(name, dimension, prefix=...)
Generation = flavor_index("Generation", 3, prefix="f")
```

3.3 Field Declaration

Once the relevant indices have been defined, we can move on to defining the fields of the model. The structure of the fields are insipired by the particle-class declarations used in FeynRules but expressed by a Python object. A

particle field will be characterised by the following properties: name, spin, statistics, index structure, quantum numbers, and, when needed, its mass and conjugation properties.

In FeynPy fields are represented by the `Field` class, whose properties are listed in the following table.

Parameter	Description
`name`	Symbolic name of the field.
`spin`	`0`, `Fraction(1,2)`, or `1`.
`self_conjugate`	`True` if the field is equal to its conjugate.
`symbol`	Symbolica atom for this field.
`conjugate_symbol`	Required when `self_conjugate=False`.
`indices`	Tuple of `IndexType` objects carried by the field.
`quantum_numbers`	Dict of charge labels, e.g. `{"Q": S("e")}`. Required for abelian gauge coupling.
`kind`	Set to `"ghost"` for ghost fields.
`flavor_index`	The `IndexType` to use for flavor expansion.
`class_members`	Tuple of member-name strings, one per generation.

Note that not every entry is required in every case: a real scalar needs almost nothing, while a quark needs a spinor index, a colour index and a conjugate symbol. The example below declares the four fields of the QCD Lagrangian used as the running example of this chapter, plus one charged scalar to show how an abelian charge is attached.

```
from fractions import Fraction

# Quark - spinor and fundamental colour index
Quark  = Field("q", spin=Fraction(1,2), self_conjugate=False,
               symbol=S("q"), conjugate_symbol=S("qbar"),
               indices=(SPINOR_INDEX, COLOR_FUND_INDEX))

# Gluon - non-abelian gauge boson, Lorentz and adjoint index
Gluon  = Field("G", spin=1, self_conjugate=True,
               symbol=S("G"), indices=(LORENTZ_INDEX, COLOR_ADJ_INDEX))

# Faddeev-Popov ghost of the gluon
GhostG = GhostField("ghG", ghost_of=Gluon, self_conjugate=False,
                    symbol=S("ghG"), conjugate_symbol=S("ghGbar"),
                    indices=(COLOR_ADJ_INDEX,))

# Complex scalar carrying a U(1) charge, for comparison
PhiC   = Field("PhiC", spin=0, self_conjugate=False,
```

```
                symbol=S("phiC"), conjugate_symbol=S("phiCdag"),
                quantum_numbers={"Q": S("Qphi")})
```

Two points are worth stressing. A gauge boson is an ordinary `Field`; it becomes a gauge boson only by being named as such in a `GaugeGroup`. And an abelian coupling is read from `quantum_numbers`, so a field with no entry for the relevant charge key simply does not couple to that group.

Note

The conjugated version of any non-self-conjugate field is written with `.bar`, for example `Psi.bar`, `Higgs.bar`. Use this form both in Lagrangian expressions and in `feynman_rule(...)` calls.

3.4 Gauge Groups and Representations

Before defining the full model, more physical information needs to be declared. In particular, the symmetry structure of the model must be specified through the gauge representation and the gauge group.

To define a gauge representation, three elements are required: the index, the generator, and a name of your choice. For fields carrying multiple indices of the same kind, the optional `slot` argument specifies which index slot is acted on by the generator, while `slot_policy="sum"` instructs the toolkit to sum the generator action over all matching slots rather than requiring a unique one. After creating the `GaugeRepresentation`, one can assemble the `GaugeGroup`.

GaugeRepresentation parameters

Parameter	Description
`index`	The `IndexType` for this representation.
`generator_builder`	Function that builds the generator tensor: `gauge_generator` for the built-in color SU(3) fundamental, `weak_gauge_generator` for SU(2) doublets, or a custom builder for other groups.
`name`	Label string.
`slot`	For fields with repeated same-kind indices, specifies which slot (0-based) is active.
`slot_policy`	`"sum"` to sum over all matching slots instead of requiring uniqueness.

GaugeGroup parameters

Parameter	Description
`name`	Identifier string.
`abelian`	`True` for abelian groups.
`coupling`	Symbolica symbol for the coupling constant.
`gauge_boson`	The gauge boson `Field`.
`charge`	For abelian groups: the quantum-number key to read from matter fields (e.g. `"Q"`).
`structure_constant`	For non-abelian groups: the structure-constant builder (e.g. `structure_constant`).
`representations`	Tuple of `GaugeRepresentation` objects for matter fields.
`ghost_field`	The ghost `Field` — required for `GhostLagrangian(...)`.

Having defined the indices and representations, the symmetry group can now be defined. As an example, we illustrate how to define the symmetry groups of the SM: $SU(3)_C \times SU(2)_L \times U(1)_Y$

Note that the abelian group $U(1)_Y$ is specified by its charge, while the non-abelian groups $SU(3)_C$ and $SU(2)_L$ require structure constants and a list of representations.

```
# Representation for the colour fundamental
COLOR_FUND_REP = GaugeRepresentation(index=COLOR_FUND_INDEX,
    generator_builder=gauge_generator, name="fundamental")

# U(1)_Y
U1Y = GaugeGroup(name="U1Y", abelian=True,
                coupling=S("g"), gauge_boson=B, charge="Y")

# SU(3) QCD
SU3C  = GaugeGroup(name="SU3C",  abelian=False, coupling=S("gs"),
                   gauge_boson=Gluon, ghost_field=GhostG,
                   structure_constant=structure_constant,
                   representations=(COLOR_FUND_REP,))

# SU(2) weak -- doublet representation
WEAK_DOUBLET_REP = GaugeRepresentation(index=WEAK_FUND_INDEX,
    generator_builder=weak_gauge_generator, name="doublet")

SU2L = GaugeGroup(name="SU2L", abelian=False, coupling=S("g2"),
                  gauge_boson=W, ghost_field=GhostW,
                  structure_constant=weak_structure_constant,
                  representations=(WEAK_DOUBLET_REP,))
```

3.5 Flavor Indices

Let's move on to falvours. The three generations of quarks and leptons are copies of the same field, and writing them out separately would triple the length of every Lagrangian. Flavor indices allow compact multi-generation expressions that can later be expanded into per-member Feynman rules. A field declares which of its indices is the flavour one, and optionally the names of its class members.

```
# 1.  Create a flavour index of dimension 3
Generation = flavor_index("Generation", 3, prefix="f")

# 2.  Field with a flavour index and explicit class members
lepton = Field("l", spin=Fraction(1,2), self_conjugate=False,
               indices=(Generation, SPINOR_INDEX),
               symbol=S("l"), conjugate_symbol=S("lbar"),
               flavor_index=Generation,
               class_members=("e", "mu", "ta"))
e, mu, ta = lepton.class_members
```

The pattern mirrors the `ClassMembers` entry of a FeynRules particle class.

3.6 Parameters

Couplings, masses and mixing matrices are declared as `Parameter` objects. A `Parameter` behaves like its Symbolica symbol in algebraic expressions, so a bare symbol from `S("lam")` works just as well for a plain constant; the class earns its place when the parameter carries indices or has known components.

Parameter	Description
`name`	Name of the parameter.
`symbol`	Symbolica symbol; defaults to `S(name)`.
`indices`	Tuple of `IndexType` objects it carries, e.g. two flavour indices for a Yukawa matrix.
`complex_param`	`True` if the parameter is complex; controls conjugation.
`components`	Dict mapping index tuples to fixed values; a value of `0` removes that component entirely.
`unitary_partner`	The conjugate matrix, for unitary pairs such as CKM and $\text{CKM}^\dagger$.
`value`	Optional numeric or symbolic value, stored as metadata.

```
# Plain coupling constant
lam = Parameter("lam")

# Complex Yukawa matrix on two flavour indices
Yu  = Parameter("Yu", indices=(Generation, Generation),
↪  complex_param=True)

# Diagonal matrix: every off-diagonal component is forced to zero
Ydiag = Parameter("Ydiag", indices=(Generation, Generation),
                  components={(i, j): 0
                              for i in range(1, 4)
                              for j in range(1, 4) if i != j})

# Unitary pair, used by rotation(...) in field transformations
CKM    = Parameter("CKM", indices=(Generation, Generation),
                   complex_param=True)
CKMDag = Parameter("CKMDag", indices=(Generation, Generation),
                   complex_param=True, unitary_partner=CKM)
```

An indexed parameter is used by calling it: `Yu(f, g)` inside a Lagrangian term contracts its two flavour indices against the fields that carry the labels `f` and `g`. The `components` mechanism is what makes `Ydiag` produce no off-diagonal vertices at all.

3.7 Declaring a Model

Our journey goes to the focal point, we can put everything together to start then calculation and manipulation. After declaring indices, fields, parameters, representations and gauge groups, the full model can be constructed. It is represented as a Python class containing the following information: the name, the gauge groups, the parameters and the Lagrangian.

For simpler models, it is sufficient to write the Lagrangian. For example, a quartic scalar interaction can be defined as

$$\mathcal{L}_{\phi^4} = \lambda\phi^4. \tag{3.2}$$

If the Lagrangian contains covariant derivatives or field-strength tensor, it will be necessary to specify the gauge group so that the toolkit will know the generator, couplings, gauge bosons and structure constant. For example, the QED fermion Lagrangian is

$$\mathcal{L}_{\text{QED}} = i\bar{\psi}\gamma^\mu D_\mu\psi, \tag{3.3}$$

while the complete gauge-fixed QCD Lagrangian, that we wanted to compute at the beginning of the chapter, is

$$\mathcal{L}_{\mathrm{QCD}} = i\bar{q}\gamma^{\mu}D_{\mu}q - \frac{1}{4}F^{a}_{\mu\nu}F^{a\mu\nu} + \mathcal{L}_{\mathrm{GF}} + \mathcal{L}_{\mathrm{ghost}}. \tag{3.4}$$

These models are implemented as follows:

```
# Local model - no gauge_groups needed
m = Model(S("lam") * Phi * Phi * Phi * Phi)

# Gauge-aware model
mu = S("mu")
m  = Model(
    I * Psi.bar * Gamma(mu) * DC(Psi, mu),
    gauge_groups=(U1QED,),
)

# Multi-term model (full gauge-fixed QCD)
mu, nu, a, xi = S("mu"), S("nu"), S("a"), S("xi")
qcd = Model(
    I * Quark.bar * Gamma(mu) * DC(Quark, mu)
    - (Expression.num(1)/Expression.num(4))
      * FS(SU3C, mu, nu, a) * FS(SU3C, mu, nu, a)
    + GaugeFixing(SU3C, xi=xi)
    + GhostLagrangian(SU3C),
    gauge_groups=(SU3C,),
)

# Compile
L = qcd.lagrangian()
```

The four summands of the last model correspond one-to-one to the four terms of the QCD Lagrangian written at the start of this chapter. Section 3.11 carries this example through to its vertices.

Model validation

```
# Check gauge-sector consistency before extracting rules
report = m.validate()
print(report.ok)      # True if no issues
print(report.issues)  # list of Issue(code, message)
```

3.8 Operator Catalog

Before going to how to manipulate this lagrangians lets first see common operator that we can find as valid Lagrangian building block inside `Model(...)`.

Local monomials

```
Field * Field * ...                          # plain product
Psi.bar * Psi * Phi                          # Yukawa
Psi.bar * Gamma(mu) * Psi * Photon           # vector current
Psi.bar * Gamma(mu) * Gamma5() * Psi * V    # axial current
```

Derivative operators — `PartialD`

```
PartialD(Phi, mu)                            # partial_mu Phi
PartialD(PartialD(Phi, mu), nu)              # nested derivative
Psi.bar * Gamma(mu) * PartialD(Psi, nu) * Phi
```

Gauge covariant derivative — `DC`

Convention: $D_\mu = \partial_\mu - igA_\mu$, as fixed in Table 2.1.

```
mu = S("mu")
# Fermion kinetic term
I * Psi.bar * Gamma(mu) * DC(Psi, mu)

# Complex scalar kinetic term
DC(PhiC.bar, mu) * DC(PhiC, mu)

# Restrict the expansion to one group of a multi-charged field
DC(QL, mu, SU3C)
```

Field strength — `FS`

```
# Abelian (no adjoint index)
FS(U1QED, mu, nu)

# Non-abelian (adjoint index required)
FS(SU3C, mu, nu, S("a"))

# Yang-Mills kinetic term
-(Expression.num(1)/Expression.num(4)) \
    * FS(SU3C, mu, nu, S("a")) \
    * FS(SU3C, mu, nu, S("a"))
```

Strict rules for `FS`

A non-abelian `FS` requires an explicit adjoint index; an abelian one must not carry one. All adjoint indices must be contracted, open colour indices raise a `ValueError` at compile time.

Higher powers and mixed-group:

```
from symbolic.spenso_structures import lorentz_levi_civita

# CP-odd operator built from the dual field strength
theta * lorentz_levi_civita(mu, nu, rho, sig) \
      * FS(U1QED, mu, nu) * FS(U1QED, rho, sig)

# Cubic field-strength operator
c3 * StructureConstant(SU3C, a, b, c) \
   * FS(SU3C, mu, nu, a) * FS(SU3C, nu, rho, b) * FS(SU3C, rho, mu, c)
```

Gauge fixing and ghosts helper

These are helper used to compile the gauge fixing and ghost lagrangian given a gauge group.

```
GaugeFixing(SU3C, xi=S("xi"))   # linear covariant gauge
GhostLagrangian(SU3C)           # Faddeev-Popov
```

The user is free to define it manually as a terml.

3.9 Extracting Feynman Rules

We finally have our model describe the Lagrangain and the symmetry. The purpose of this project was to achieve the Feynman Rule given a Lagrangian and now we have everything we need to extract the rules and do much more.

Call `feynman_rule` on the `Model` or the compiled `Lagrangian` object returned by `model.lagrangian()` to obtain the Feynman Rule expressed as a Symbolica expression.

```
L = model.lagrangian()

# Targeted: pass explicit fields to get one vertex
vertex = L.feynman_rule(Psi.bar, Psi, Photon)

# Custom external momenta (default labels: q1, q2, q3, ...)
vertex = L.feynman_rule(Psi.bar, Psi, Photon,
                        momenta=[S("p_in"), S("p_out"), S("k")])
```

Key options

Option	Default	Description
`simplify`	`True`	Run symbolic simplification on the result.
`simplify_gamma`	`False`	If `simplify=True`, also apply Dirac-algebra simplification to gamma chains.
`include_delta`	`False`	Include the overall momentum-conservation δ factor.
`strip_externals`	`True`	Strip external wavefunction factors.
`flavor_expand`	`False`	`True`, one flavor index, or an iterable of flavor indices to expand flavor-class members.
`key_format`	`"names"`	Result-dict keys: `"names"` (string tuples) or `"fields"` (`Field` objects).

Reading the output

Symbol	Meaning
`Delta(...)`	Overall momentum-conservation factor.
`gamma(...)`	γ matrix.
`t(...)`	Gauge generator tensor.
`f(...)`	Structure constant.
`g(...)`	Metric tensor.
`pcomp(p,mu)`	Component of momentum p along index μ.

3.10 Vertex Discovery and Filtering

Call `feynman_rule()` with no arguments to enumerate every available vertex as a dictionary. Use the lighter-weight inspection methods to browse without computing all expressions first.

```
# Enumerate all vertices -- returns dict keyed by name tuples
all_rules = L.feynman_rule(simplify=False)
for sig, expr in all_rules.items():
    print(sig, "->", expr)

# Object-keyed dictionary
all_rules = L.feynman_rule(simplify=False, key_format="fields")
```

Lightweight inspection with `vertex_signatures`

Achilleas: What is 'arity'? it appears in the code below.

```
# All signatures
for sig in L.vertex_signatures():
    print(sig.names, "arity=", sig.arity, "terms=", sig.term_count)

# Filter by arity
three_point  = L.vertex_signatures(arity=3)

# Exact field-content match (order-independent)
exact        = L.vertex_signatures(signature=(Gluon, Quark.bar, Quark))

# All signatures containing a given field multiset
ghost_sigs   = L.vertex_signatures(contains_fields=(GhostG.bar, GhostG))
```

```
# vertex_report -- filtered signatures with count summary
report = L.vertex_report(arity=3)
print(report.matched_signatures, "of", report.total_signatures)
for sig in report.signatures:
    print(sig.names)
```

Tip

For large models, use `vertex_signatures` or `vertex_report` to inspect available vertices before calling `feynman_rule`, which actually computes the symbolic expressions.

3.11 A Complete Example: Gauge-Fixed QCD

We opened this chapter with the gauge-fixed QCD Lagrangian and promised to derive its rules. All the ingredients are now available, so here is the complete script, from imports to a compiled Lagrangian.
To keep the example compact, we import the public FeynPy surface with `from feynpy import *`. The non-FeynPy objects (`S`, `Expression`, `I` and the Spenso tensor builders) are still imported explicitly.

```
from fractions import Fraction

from symbolica import S, Expression
from symbolic.vertex_engine import I
from symbolic.spenso_structures import gauge_generator,
↪  structure_constant
from feynpy import *
```

```
mu, nu = S("mu"), S("nu")
a, xi, gs = S("a"), S("xi"), S("gs")
quarter = Expression.num(1) / Expression.num(4)

# Fields
Quark = Field(
    "q",
    spin=Fraction(1, 2),
    self_conjugate=False,
    symbol=S("q"),
    conjugate_symbol=S("qbar"),
    indices=(SPINOR_INDEX, COLOR_FUND_INDEX),
)
Gluon = Field(
    "G",
    spin=1,
    self_conjugate=True,
    symbol=S("G"),
    indices=(LORENTZ_INDEX, COLOR_ADJ_INDEX),
)
GhostG = GhostField(
    "ghG",
    ghost_of=Gluon,
    self_conjugate=False,
    symbol=S("ghG"),
    conjugate_symbol=S("ghGbar"),
    indices=(COLOR_ADJ_INDEX,),
)

# Gauge group
FUND = GaugeRepresentation(
    index=COLOR_FUND_INDEX,
    generator_builder=gauge_generator,
    name="fundamental",
)
SU3C = GaugeGroup(
    name="SU3C",
    abelian=False,
    coupling=gs,
    gauge_boson=Gluon,
    ghost_field=GhostG,
    structure_constant=structure_constant,
    representations=(FUND,),
)

# Gauge-fixed QCD Lagrangian
qcd = Model(
```

```
    (
        I * Quark.bar * Gamma(mu) * DC(Quark, mu)
        - quarter * FS(SU3C, mu, nu, a) * FS(SU3C, mu, nu, a)
        + GaugeFixing(SU3C, xi=xi)
        + GhostLagrangian(SU3C)
    ),
    gauge_groups=(SU3C,),
)
L = qcd.lagrangian()
```

Before printing large expressions, inspect what the compiler generated:

```
print("compiled terms:", len(L.terms))

for sig in L.vertex_signatures():
    names = str(sig.names)
    print(f"{names:<25} arity={sig.arity}  terms={sig.term_count}")
```

The output is:

```
compiled terms: 14

(G, G)                    arity=2  terms=5
(ghG.bar, ghG)            arity=2  terms=1
(q.bar, q)                arity=2  terms=1
(G, G, G)                arity=3  terms=4
(ghG.bar, G, ghG)        arity=3  terms=1
(q.bar, q, G)            arity=3  terms=1
(G, G, G, G)           arity=4  terms=1
```

This listing already tells a small story. A single `FS` factor squared has produced two-, three- and four-gluon signatures at once; the `GaugeFixing` declaration has added four further terms to the two-gluon signature; and the `GhostLagrangian` has contributed both a ghost kinetic term and a ghost–gluon vertex.
Now ask for the individual rules. Since simplification is the default, it does not need to be passed explicitly:

```
V_qqg = L.feynman_rule(Quark.bar, Quark, Gluon)
V_cg  = L.feynman_rule(GhostG.bar, GhostG, Gluon)
V_ggg = L.feynman_rule(Gluon, Gluon, Gluon)
V_gg  = L.feynman_rule(Gluon, Gluon)
```

Selected output

Quark–gluon.

```
I*gs*gamma(bis(4,i1),bis(4,i2),mink(4,mu3))
   *t(coad(8,a3),cof(3,c1),cof(3,c2))
```

This is $ig_s\gamma^{\mu_3}t^{a_3}$, with the colour indices in the fundamental slots of the two quark legs.

Ghost–gluon.

```
gs*f(coad(8,a1),coad(8,a3),coad(8,a2))*pcomp(q1,mu3)
```

This is $g_s f^{a_1 a_3 a_2} q_1^{\mu_3}$, carrying the momentum of the antighost leg as expected from $\mathcal{L}_{\text{gh}}$.

Three-gluon.

```
-gs*g(mink(4,mu3),mink(4,mu1))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q1,mu2)
+gs*g(mink(4,mu3),mink(4,mu1))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q3,mu2)
+gs*g(mink(4,mu3),mink(4,mu2))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q2,mu1)
-gs*g(mink(4,mu3),mink(4,mu2))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q3,mu1)
+gs*g(mink(4,mu1),mink(4,mu2))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q1,mu3)
-gs*g(mink(4,mu1),mink(4,mu2))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q2,mu3)
```

This is the textbook expression

$$g_s f^{a_1 a_2 a_3}\Big[g^{\mu_1\mu_2}(q_1-q_2)^{\mu_3}+g^{\mu_2\mu_3}(q_2-q_3)^{\mu_1}+g^{\mu_3\mu_1}(q_3-q_1)^{\mu_2}\Big], \tag{3.5}$$

with all momenta incoming, written out term by term.

Gluon two-point.

```
 I*g(coad(8,a1),coad(8,a2))*pcomp(q1,mu1)*pcomp(q2,mu2)/xi
+I*g(mink(4,mu1),mink(4,mu2))*g(coad(8,a1),coad(8,a2))
    *pcomp(q1,mu1_int)*pcomp(q2,mu1_int)
-I*g(coad(8,a1),coad(8,a2))*pcomp(q1,mu2)*pcomp(q2,mu1)
```

This rule shows the gauge parameter explicitly. Setting $\xi = 1$ collapses the inverse propagator to the Feynman-gauge form. The label `mu1_int` is an automatically generated dummy Lorentz index, the contraction $q_1{\cdot}q_2$, and its name carries no meaning, which is exactly the point of the canonicalisation discussed in Section 5.7.

3.12 Flavor Expansion

Pass `flavor_expand=True` to resolve all flavor indices and produce one Feynman rule per generation combination. Alternatively, pass a single `IndexType` or a tuple to expand only selected indices.

```
# Compact (generation-generic) rules
rules_compact  = L.feynman_rule()

# Fully expanded -- one rule per member combination
rules_expanded = L.feynman_rule(flavor_expand=True)

# Expand only one specific index
rules_gen      = L.feynman_rule(flavor_expand=Generation_1)

# Direct member vertex (requires flavor_expand)
vertex_e_mu    = L.feynman_rule(e.bar, mu, Phi, flavor_expand=True)

# Signature discovery also supports flavor_expand
sigs = L.vertex_signatures(flavor_expand=True)
```

Off-diagonal zeroes

If a parameter is declared with `components={(i,j): 0}` for $i \neq j$, off-diagonal member combinations are automatically absent from the expanded rules, no manual filtering is needed.

3.13 Field Transformations: From the Gauge Basis to the Physical Basis

Let's see how to apply field transformations. Let's refresh the field transformation for the standard model. The Lagrangian is written in terms of B_μ, W^I_μ and the doublet Φ, but the vertices we want involve the photon, the Z, the $W^\pm$, the Higgs and the Goldstone bosons.

The bridge is a set of substitutions, and FeynPy implements it with `FieldTransformation`.

The basic pattern

```
from feynpy import FieldTransformation

# B_mu  ->  -sw * Z_mu + cw * A_mu
neutral_mixing = FieldTransformation(B, -sw * Z + cw * A)
```

```
L_physical = model.transform_fields(neutral_mixing, repeat=False)
```

Applied to a scalar current $(D_\mu H)^\dagger (D^\mu H)$, this moves the hypercharge current onto the physical fields. Before the transformation the only nonzero rule is

$$\Gamma(H^\dagger, H, B) = -\tfrac{i}{2} g_1 (q_1 - q_2)^{\mu_3},$$

and afterwards it has been replaced by

$$\Gamma(H^\dagger, H, A) = -\tfrac{i}{2} c_W g_1 (q_1 - q_2)^{\mu_3}, \qquad \Gamma(H^\dagger, H, Z) = +\tfrac{i}{2} s_W g_1 (q_1 - q_2)^{\mu_3},$$

while $\Gamma(H^\dagger, H, B)$ no longer exists, because B is no longer a field of the model.

Compile first, transform second

Transformations act on an *already compiled* Lagrangian. Covariant derivatives and field strengths are expanded in the original gauge basis, where the gauge group data is meaningful, and only then are the fields substituted. Writing `model.transform_fields(...)` performs both steps in that order. This is the same ordering that FeynRules uses, and it is the reason `DC` is never itself transformed.

Component rules

Component rules are the important extra ingredient for the Standard Model. The dictionary `components={slot: value}` says that the rule only applies to one fixed component of an indexed field. For example, the weak gauge field has slots `(Lorentz, WeakAdj)`, so `components={1: 3}` selects W_μ^3.

```
from models.SM import standard_model_weak_tensor_components

component_lagrangian = model.lagrangian().expand_index_components(
    WEAK_FUND_INDEX,
    WEAK_ADJ_INDEX,
    tensor_components=standard_model_weak_tensor_components(),
)

transformations = (
    # neutral gauge-boson mixing
    FieldTransformation(B, -sw * Z + cw * A),
    FieldTransformation(Wi, INV_SQRT2 * W.bar + INV_SQRT2 * W,
                        components={1: 1}),
    FieldTransformation(Wi, INV_SQRT2 / I * W.bar - INV_SQRT2 / I * W,
                        components={1: 2}),
```

```
    FieldTransformation(Wi, cw * Z + sw * A,
                        components={1: 3}),

    # Higgs doublet components
    FieldTransformation(Phi, -I * GP, components={0: 1}),
    FieldTransformation(Phi, vev * INV_SQRT2 + INV_SQRT2 * H
                             + I * INV_SQRT2 * G0,
                        components={0: 2}),
)

L_physical = component_lagrangian.transform_fields(
    *transformations,
    repeat=False,
)
```

The keyword `repeat=False` makes the rules act as one simultaneous FeynRules-style definitions block: a field created by one rule is not transformed again by another rule in the same pass. This is what the electroweak rotation needs.

Projectors and flavour rotations

The implementation chiral projectors and flavour rotations follows the same idea. These matrix factors are attached to the appropriate spinor and flavour indices automatically:

```
from feynpy import ProjM, ProjP, rotation

ckm = rotation(CKM, CKMDag)

fermion_transformations = (
    FieldTransformation(LL, ProjM * vl,       components={1: 1}),
    FieldTransformation(LL, ProjM * l,        components={1: 2}),
    FieldTransformation(lR, ProjP * l),
    FieldTransformation(QL, ProjM * uq,       components={1: 1}),
    FieldTransformation(QL, ckm * ProjM * dq, components={1: 2}),
    FieldTransformation(uR, ProjP * uq),
    FieldTransformation(dR, ProjP * dq),
)
```

The complete Standard Model implementation, including the corresponding ghost mixings and the exact ordering used in validation, can be seen in `models/SM/SM.py` by the interested user.

3.14 Symbolica Export

Now that we learned how to extract Feynman rules, with and without a flavor expansion, we are interested in procceding to the manipulation of the actual lagrangian. FeynPy is build on this Model API to handle all the information needed to expand the covariant derivative, field strength and so on that depends on the symmetry group. To manipulate a constructed Lagrangian it would be smart to have the expression of Symbolica so that we can call all the built-in function of this amazing library.

Call `to_symbolica()` on a compiled `Lagrangian` to obtain an `Expression` that can be differentiated, collected, or passed to any Symbolica routine.

```
expr = L.to_symbolica()
print(expr)

# Standard Symbolica operations work directly
expr.expand()
expr.derivative(S("Phi"))
expr.coefficient(S("lam"))
```

Coordinate-dependent export

```
# Makes spacetime dependence explicit
expr_coord = L.to_symbolica(derivative_style="coordinate")
expr_coord.derivative(S("x_mu"))
```

Flavor-expanded export

```
compact  = L_fl.to_symbolica()              # generation-generic
expanded = L_fl.to_symbolica(flavor_expand=True)  # per member (e, mu,
↪  ta, ...)
```

3.15 Applying Operators

The manipulation we can do on our `Model` goes further. We are interested in applying linear and non-linear operators to every term of our Lagrangian. Operators such as a replace operator, a derivative, or an infinitesimal Gauge Transformation. We will see that we can even apply a non-linear operator like the infinitesimal BRST transformation.

The operator layer acts on a compiled `Lagrangian` term by term, following a graded Leibniz rule. It performs field redefinitions, substitutions, gauge variations, and BRST transformations.

Required imports

```
from lagrangian.operator_action import (
    FieldOperator, TermOperator,
    OperatorAtomResult, OperatorSummand,
    OperatorExpansionError,
    replacement_operator, single_field_result,
    partial,
)
```

Simple field replacement

```
op    = replacement_operator("Phi_to_Chi", {Phi: Chi()})
L_new = L.apply_operator(op)

# Works on Model too -- apply before compiling
L_new = model.apply_operator(op)
```

FieldOperator — slot-wise action

Acts once per matched field slot, expanding via the graded Leibniz rule. As an example it's exposed the operator $O[\phi] = a * \chi + b * \Omega$

```
# Phi -> a*Chi + b*Omega  (one summand per slot)
op = FieldOperator(
    "phi_to_combo",
    on_field=lambda occ: (
        None if occ.field is not Phi else
        OperatorAtomResult(summands=(
            OperatorSummand(coefficient=S("a"), replacement=(Chi(),)),
            OperatorSummand(coefficient=S("b"), replacement=(Omega(),)),
        ))
    ),
)

# Odd operator (parity=1 triggers graded Leibniz signs)
s = FieldOperator("s", parity=1, on_field=lambda occ: ...)
```

TermOperator — whole-term action

We can apply an operator to an entire lagrangian term.

```
# Acts on each InteractionTerm as a whole (no Leibniz expansion)
from dataclasses import replace
```

```
scale2 = TermOperator(
    "scale2",
    apply_to_term=lambda term: (replace(term, coupling=2 *
    ↪  term.coupling),),
)
```

Spacetime derivative operator — `partial(mu)`

As a linear operator one of the most interesting is the derivative operator.

```
L_partial = L.apply_operator(partial(mu))          # partial_mu on every
↪  slot
L_partial = L.apply_operator(partial(mu, on=Phi))  # restrict to Phi
```

Composition and algebra

```
# Apply a sequence left-to-right
L_out = L.apply_operators(op1, op2, op3)

# Graded commutator [A, B] = A otime B - (-1)^{|A||B|} B otime A
L.operator_bracket(opA, opB).to_symbolica()
```

Derivative distribution

When a replacement maps a field to a product, any derivative acting on that field is automatically distributed over the product according to the Leibniz rule. No manual handling is required.

3.16 Integration by Parts

Two Lagrangians which differ by a total derivative give rise to the same physics. Deciding whether that is the case is exactly the question that arises when reducing a redundant operator basis to a minimal one (Section 2.4.2), and `ibp_normal_form()` answers a restricted version of it: it rewrites a compiled Lagrangian into a canonical representative modulo total derivatives, so that two Lagrangians with the Normal forms that are equivalent differ only by boundary terms.

```
# Check that  phi^2 partial partial phi  ~  -2 phi (d_mu phi)^2  modulo
↪  total derivatives
e1 = c * phi * phi * PartialD(PartialD(phi, mu), mu)
e2 = -2 * c * phi * PartialD(phi, mu) * PartialD(phi, mu)
```

```
assert Model(e1).lagrangian().ibp_normal_form().terms \
    == Model(e2).lagrangian().ibp_normal_form().terms

# Equivalently, the normal form of the difference is empty
delta = Model(e1 - e2).lagrangian().ibp_normal_form()
assert delta.terms == ()
```

Scope of the current implementation

This first version of the integration-by-parts layer handles unlabelled scalar species only: it symmetrises identical scalar slots and moves derivatives into a left-normal form. Terms containing Dirac bilinears or gauge indices are outside its scope, so it cannot yet be used to reduce a Green basis to the Warsaw basis. Section 7.2 returns to this.

3.17 Gauge Variation

We are now interested in gauge transformations. We can apply an infinitesimal gauge transformation to check the gauge invariance of a given Lagrangian. `gauge_variation(group=..., parameter=...)` returns a `FieldOperator` implementing the infinitesimal gauge variation for a givern group. Different possible transformation are exposed in the following table.

Transformation rules

Field	**Variation δ**
Matter rep R: Φ_i	$+ig\,\alpha^a T^a_{ij}\,\Phi_j$
Matter rep R: $\bar{\Phi}_i$	$-ig\,\alpha^a \bar{\Phi}_j T^a_{ji}$
U(1) charge q: Φ	$+igq\,\alpha\,\Phi$
Abelian gauge boson A_μ	$+\partial_\mu \alpha$
Non-abelian gauge boson A^a_μ	$+\partial_\mu \alpha^a - g f^{abc} \alpha^b A^c_\mu$

```
from lagrangian.operator_action import gauge_variation
from symbolic.tensor_canonicalization import canonize_full

delta = gauge_variation(group=U1QED, parameter="alpha")
dL    = L.apply_operator(delta)

# Verify invariance
canon = canonize_full(dL.to_symbolica().expand(), lorentz_indices=(mu,))
assert canon.to_canonical_string() == "0"   # => gauge invariant
```

3.18 BRST Transformations

The natural next step after the infinitesimal gauge transformation is to consider the BRST transformation. The constructor

```
brst_transformation(group, ghost, antighost=None, auxiliary=...)
```

returns an *odd* `FieldOperator` implementing the BRST transformation for one selected gauge group. The ghost must be declared with `kind="ghost"` and associated with the chosen gauge boson through `ghost_of=<gauge_boson>`. The Brst transforamtion are

$$\begin{aligned} sA^a_\mu &= D_\mu c^a = \partial_\mu c^a + g f^{abc} A^b_\mu c^c, \\ sc^a &= -\frac{g}{2} f^{abc} c^b c^c, \\ s\bar{c}^a &= B^a, \\ sB^a &= 0. \end{aligned}$$

For matter fields, we use

$$s\Phi_i = +ig\, c^a (T^a)_{ij} \Phi_j, \qquad s\bar{\Phi}_i = -ig\, c^a \bar{\Phi}_j (T^a)_{ji},$$

and, in the abelian case,

$$s\Phi = +igq\, c\, \Phi, \qquad s\bar{\Phi} = -igq\, c\, \bar{\Phi}.$$

If a field carries charges under several groups, we apply one BRST operator per group and sum the resulting contributions.

Because the operator is Grassmann-odd, it acts on products through the graded Leibniz rule. Keep in mind that the exported Symbolica expression is still commutative, therefore there will be a possible sign error in how the symbolica expression is printed out.

```
from lagrangian.operator_action import brst_transformation

# Ghost and auxiliary field declarations
c    = Field("c", spin=0, kind="ghost", ghost_of=Gluon,
             self_conjugate=False, conjugate_symbol=S("cbar"),
             indices=(COLOR_ADJ_INDEX,),
             quantum_numbers={"GhostNumber": 1})
Baux = Field("B", spin=0, self_conjugate=True,
↪  indices=(COLOR_ADJ_INDEX,))

s = brst_transformation(group=SU3C, ghost=c, auxiliary=Baux)

# Act on individual fields
```

```
mu, a = S("mu"), S("a")
print(Gluon(mu, a).apply_operator(s).to_symbolica())  # s A^a_mu
print(c(a).apply_operator(s).to_symbolica())          # s c^a
```

Nilpotency check $s^2 = 0$

The BRST transformation has a peculiar property, namly the nilpotency. We can check this property bu applying twice the operator. For example on the Gluon Field.

```
s2_G  = Gluon(mu, a).apply_operator(s).apply_operator(s).to_symbolica()
canon = canonize_full(s2_G, run_gamma=False, run_color=False,
↪  infer_indices=True)
assert canon.to_canonical_string() == "0"
```

BRST-exact gauge fixing

We can write a BRST Lagrangian as a BRST variation of some funciton K_{BRST}

$$\mathcal{L}_{\mathrm{BRST}} = s(K_{\mathrm{BRST}}), \quad K_{\mathrm{BRST}} = \bar{c}^a\, \partial^\mu A^a_\mu + \frac{\xi}{2}\, \bar{c}^a B^a.$$

Its BRST variation gives the gauge-fixing and ghost terms:

$$sK_{\mathrm{BRST}} = B^a\, \partial^\mu A^a_\mu + \frac{\xi}{2}\, B^a B^a - \bar{c}^a\, \partial^\mu D_\mu c^a,$$

with

$$D_\mu c^a = \partial_\mu c^a + g f^{abc} A^b_\mu c^c.$$

The full gauge-fixed BRST-invariant Lagrangian is then

$$L_{\mathrm{gf}} = L_{\mathrm{YM}} + sK_{\mathrm{BRST}}.$$

```
xi = S("xi")
K = Model(
    gauge_groups=(SU3C,),
    lagrangian_decl=(
        c.bar(a) * PartialD(Gluon(mu, a), mu)
        + xi / Expression.num(2) * c.bar(a) * Baux(a)
    ),
).lagrangian()

L_gf = L_YM + K.apply_operator(s)   # full gauge-fixed Lagrangian
```

Ghost field validation

`brst_transformation` validates that the ghost is declared with `kind="ghost"` and that `ghost_of` matches the group's gauge boson. A mismatch raises `ValueError` at construction time.

3.19 Expression Manipulation

Once exported via `to_symbolica()`, standard Symbolica methods (`expand`, `collect`, `coefficient`) work directly.

```
expr = L.to_symbolica()
expr.expand()
expr.coefficient(S("xi"))      # works for literal symbols
expr.coefficient(S("gS"))      # coupling-constant dependence
```

Wildcard-aware coefficient extraction

```
from lagrangian.symbolica_export import pattern_coefficient

mu1_, a1_, mu2_, a2_ = S("mu1_", "a1_", "mu2_", "a2_")  # trailing _ =
↪  wildcard
pattern       = G(mu1_, a1_) * G(mu2_, a2_)

print(pattern_coefficient(expr, pattern))  # handles wildcards
↪  automatically
```

Wildcard convention

In Symbolica, variables whose names end with `_` are interpreted as wildcards in pattern-matching operations such as `match(...)`. The resulting matches can be substituted into a pattern using `replace_wildcards(...)`. This convention is useful for extracting index-independent patterns from exported tensor expressions.

3.20 Packaged Models

Up to this point, only general-purpose tools have been described. The toolkit also comes with three complete models, which are built entirely from these tools. These models are used as examples in the validation section (see Chapter 6). Each model has its own directory under `models/`, along with its FeynRules reference data, comparison adapter, and tests.

The Standard Model

```
from models.SM import build_standard_model

sm = build_standard_model()
L  = sm.lagrangian

wwa = L.feynman_rule(sm.fields.W.bar, sm.fields.W, sm.fields.A)
hww = L.feynman_rule(sm.fields.H,     sm.fields.W.bar, sm.fields.W)
```

The builder follows the pipeline of this chapter: declare the gauge-basis fields (Q_L, L_L, Φ, B, W^I, G and the singlets), assemble the gauge, fermion, Higgs, Yukawa and ghost sectors, compile the covariant derivatives and field strengths, expand the weak indices into components, and finally apply the field transformations of Section 3.13 in a single simultaneous pass. It returns a `StandardModel` record exposing the source model, the transformation list, the compiled physical-basis Lagrangian and the field and parameter namespaces.

The unbroken background-field Standard Model

```
from models.UnbrokenSM_BFM import build_unbroken_sm_bfm

bfm = build_unbroken_sm_bfm()
```

This model stays in the unbroken gauge basis and instead performs the background/quantum split of Section 2.3.8, again as a `FieldTransformation`. Because the gauge is fixed only for the quantum fields, and covariantly with respect to the background ones, its gauge-fixing term is written out explicitly rather than through the generic `GaugeFixing` helper.

Green-basis SMEFT

```
from models.SMEFT import build_smeft_green_bpreserving

smeft = build_smeft_green_bpreserving()

Ltot  = smeft.lagrangians["Ltot"]    # EFT contribution only (the model
↪  default)
Lfull = smeft.lagrangians["Lfull"]   # Standard Model plus EFT

L = smeft.model.lagrangian()         # compiles Ltot
```

This is the largest model, and the one that motivated most of the engine's generality. It declares 298 Wilson coefficients as indexed `Parameter`s and

32 Lagrangian blocks in the baryon-number-preserving Green basis of Section 2.4.2, including dual field strengths, $\sigma^{\mu\nu}$ structures, weak ε tensors and the explicit charge-conjugation matrices of the evanescent sector. The Standard Model core is taken from the unbroken background-field model above, minus its background-field ghost and gauge-fixing sectors. Two conventions are worth noting because Chapter 6 depends on them. `Ltot` contains the EFT contribution alone, matching the FeynRules convention for the reference export, with the combined theory available separately as `Lfull`. And the model is written in terms of chiral fields (`lL`, `qL`, `eR`, `uR`, `dR`) so chirality is carried by the field names rather than by explicit projectors.

3.21 Quick Reference

Here `L` denotes a compiled `CompiledLagrangian`, typically from `model.lagrangian()`.

Construction

Call	Purpose
`IndexType(name, rep, kind)`	Define a custom index type.
`flavor_index(name, dim, ...)`	Define a flavor index type.
`Field(name, spin, self_conjugate, ...)`	Declare a particle field.
`GhostField(name, ghost_of=..., ...)`	Declare a ghost field.
`GaugeRepresentation(index, generator_builder, ...)`	Define a matter representation.
`GaugeGroup(name, abelian, coupling, ...)`	Define a gauge group.
`Parameter(name, symbol=None, indices=(), ...)`	Declare a parameter or indexed coupling tensor.

Model and compilation

Call	Purpose
`Model(expr, ...)`	Shorthand model declaration from source terms.
`model.lagrangian()`	Compile; returns `CompiledLagrangian`.
`model.validate()`	Source-level validation report.

Declarative factors (inside `lagrangian_decl`)

Call	Purpose
`Gamma(mu)`	Declarative Dirac γ^μ placeholder.
`PartialD(field, mu)`	Ordinary ∂_μ field.
`DC(field, mu)`	Covariant derivative with convention $D_\mu = \partial_\mu - igA_\mu$ (alias `CovD`).
`FS(group, mu, nu[,a])`	$F_{\mu\nu}$; adjoint index required for non-abelian groups (alias `FieldStrength`).
`GaugeFixing(group, xi=...)`	Linear covariant gauge-fixing declaration.
`GhostLagrangian(group)`	Ordinary non-abelian Faddeev–Popov ghost declaration.

Extraction

Call	Purpose
`L.feynman_rule(F1, F2, ...)`	One vertex for those external fields.
`L.feynman_rule()`	All discovered vertices as a dict.
`L.vertex_signatures(...)`	Grouped signatures without vertex computation.
`L.vertex_report(...)`	Structured filtered report with grouped counts.

Compiled Lagrangian layer

Call	Purpose
`L.to_symbolica(...)`	Export / display as a Symbolica `Expression`.
`L.transform_fields(*rules)`	Substitute fields, e.g. gauge basis → physical basis.
`L.expand_index_components(...)`	Expand finite gauge indices into explicit components.
`L.apply_operator(op)`	Apply a `FieldOperator` or `TermOperator`.
`L.apply_operators(...)`	Compose operators left-to-right.
`L.operator_bracket(A, B)`	Graded commutator $[A, B]$.
`L.ibp_normal_form()`	Scalar-only IBP normal form.

Operator constructors

Call	Purpose
`replacement_operator(name, {F: r})`	Simple field-replacement operator.
`FieldOperator(name, parity, ...)`	Slot-wise operator with graded Leibniz rule.
`TermOperator(name, ...)`	Whole-term operator.
`partial(mu)`	Runtime spacetime-derivative operator.
`gauge_variation(group, parameter=...)`	Infinitesimal gauge variation δ_α.
`brst_transformation(group, ghost, ...)`	Odd BRST differential.
`FieldTransformation(source, expr, ...)`	Declarative field redefinition.
`rotation(V, Vdag)`	Flavour-rotation matrix factor for a replacement.

Packaged models

Call	Purpose
`build_standard_model()`	Standard Model in the physical basis.
`build_unbroken_sm_bfm()`	Unbroken Standard Model with the background-field split.
`build_smeft_green_bpreserving()`	Green-basis SMEFT, EFT-only by default.

Expression manipulation

Call	Purpose
`expr.expand() / .coefficient(s)`	Standard Symbolica algebra.
`pattern_coefficient(expr, pat)`	Wildcard-aware coefficient extraction.
`canonize_full(expr, ...)`	Full tensor simplification and canonicalisation.

Chapter 4

Computer-Algebra Foundations: Symbolica and Spenso

Why is it now possible to implement Feynman rules in Python? Previously, Python did not have a symbolic algebra engine, so it could not automatically compute Feynman rules. More importantly, it could not efficiently handle the large tensor expressions needed for quantum theory. Now, Symbolica makes this possible by providing high-performance symbolic manipulation while keeping the flexibility of Python.

Before explaining how the toolkit turns a declared model into Feynman rules in Chapter 5, it helps to introduce the main libraries used in the project. The toolkit represents every symbolic object, like couplings, momenta, gamma matrices, or entire vertices, with Symbolica. As mentioned in the previous chapter, indices are also important, and the Spenso library gives them meaning. Symbolica handles the expressions, Spenso explains the indices, and a small extra layer called idenso adds the physical identities for these typed indices. This chapter introduces each library, explains its role in the project, and discusses why it was chosen.

4.1 Symbolica: a modern computer-algebra core

Symbolica [4] is a high-performance computer algebra system (CAS) developed by Ben Ruijl. Its core is written in Rust, and it can be used through interfaces for Python, Rust, and C++. It is used in high-energy-physics research, including at CERN. It arose from the practical needs of collider physics, where symbolic computations can generate expressions containing billions of terms and requiring terabytes of memory. Established tools, such as the Form family and general-purpose systems like Mathematica, can be difficult to maintain or are not designed for computations at this scale [4, 5].

Three properties of SYMBOLICA are particularly relevant to this thesis and have influenced the design of the toolkit. They are introduced in the following sections.

Expressions, symbols, and pattern matching

Expressions are the central objects in SYMBOLICA: they are parsed, constructed, transformed, matched, differentiated, evaluated, and printed throughout the computation. A `Expression` is an ordinary in-process object that can be stored directly in Python data structures, compared, hashed, and passed through the toolkit without serialisation. Expressions are built from exact or approximate numbers, symbols, functions, sums, products, and powers, and are automatically normalised when created. Symbols can be created with `S("name")`, while complete expressions can be parsed with `E("...")`:

```
from symbolica import S, E, Expression
x, y, f = S("x", "y", "f")
e = f((1 + x)**2)          # a function application, f of (1+x)^2
```

One of the most used capability for this project is *pattern matching with wildcards*. Any symbol whose name ends in an underscore is treated as a wildcard in `match`/`replace` operations, so algebraic rewriting reads almost like a regular expression for mathematics:

```
x_, n_ = S("x_", "n_")
e = f((1 + x)**2).replace(f(x_**n_), f(x_)**n_)
print(e)   # ->  f(1 + x)**2
```

The toolkit leans on this pervasively. As we will see in Chapter 5, the central step of the contraction engine (attaching an external leg's index label to an open index of a coupling) is literally one call to `coupling.replace(...)`, and the user-facing helper `pattern_coefficient` (Section 3.19) is a thin wrapper around the same wildcard machinery. Standard CAS operations are available on the same objects: `.expand()`, `.collect()`, `.derivative()` (with the chain and product rules applied automatically), `.factor()`, `.coefficient()`, series expansion, and even `solve_linear_system`.

Polynomial algebra: the source of the speed

What makes SYMBOLICA so useful is the highly efficient algorithms it offers for multivariate polynomial operations. These include the greatest common divisor, factorisation, and interpolation over the rationals [4]. This polynomial engine is one of the main sources of its performance. For example,

in Symbolica's own multivariate-GCD benchmark, a calculation that takes about 89 s in MATHEMATICA and more than an hour in SYMPY is completed in about 4 s [4, 5].

This is important for the toolkit developed in this thesis, because the contraction engine described in Chapter 5 generates many terms by summing over permutations of the external legs. It then uses Symbolica's `.expand()` and `.collect()` methods to simplify the resulting sums and rewrite them in a compact polynomial form. Since this normalisation is performed repeatedly on potentially large expressions, the efficiency of Symbolica's polynomial engine has a direct effect on the overall performance.

Tensor and graph canonicalisation

Symbolica is not limited to scalar algebra. For this project, one of its most useful features is the possibility to bring tensor expressions into a canonical form. Given a product of tensors with contracted indices, `Expression.canonize_tensors` renames dummy indices in a deterministic way and orders the tensor factors according to their symmetry properties. Internally, this is based on a graph-canonicalisation algorithm for tensor networks [5]. In this way, expressions that differ only by dummy-index names, tensor symmetries, or the ordering of commuting factors are reduced to the same form.

In FeynPy this cannot be applied directly, because tensors are kept as typed Spenso objects during the calculation, while Symbolica expects its symmetry information on ordinary Symbolica heads. For this reason we introduced the wrapper `canonize_spenso_tensors`. It temporarily replaces the Spenso tensor heads by Symbolica ones, attaches the needed symmetry information, performs the canonicalisation, and then restores the original objects.

The wrapper also keeps Lorentz, spinor, colour and weak dummy indices separated. On top of this, `canonize_full` applies the additional identities needed in the physics pipeline. These include metric contractions, the ordering of nested flat derivatives, the antisymmetry of f^{abc}, and local Jacobi reductions involving products of structure constants.

Why a fast CAS matters here

The toolkit expands covariant derivatives and field strengths into many smaller terms, and afterwards evaluates the possible Wick contractions. The number of terms can therefore grow quite fast. Having a CAS that can manipulate and canonicalise large symbolic expressions efficiently is not only convenient here, but really necessary.

4.2 Spenso: typed tensor indices and tensor networks

We now turn to the role of Spenso. From Symbolica's point of view, an expression such as `G(mu, a)` is simply a function applied to two arguments. In fact it doesn't know that the index `mu` is a Lorentz index or that `a` is an adjoint colour index. Her is where Spenso [3] comes into play providng additional structure to the whole. It represents each tensor index as a typed slot, consisting of an abstract label together with a representation, and it can then match and contract compatible indices automatically.

Slots and representations

The central concept in Spenso is the `Slot`, which combines a `Representation` with an abstract index label. A `Representation` identifies the type and dimension of the index and determines its duality and contraction rules. The toolkit uses the six representations listed in Table 4.1, each created once and reused for all indices of the corresponding type.

Constructor	Self-dual?	Used for
`Representation.mink(4)`	yes	Lorentz vector indices ($\mu, \nu, \dots$)
`Representation.bis(4)`	yes	Dirac bispinor indices on fermion fields
`Representation.cof(3)`	no	SU(3) colour fundamental (quark) index
`Representation.coad(8)`	yes	SU(3) colour adjoint (gluon / generator / f^{abc}) index
`Representation.cof(2)`	no	$\mathrm{SU}(2)_L$ weak-isospin fundamental (doublet) index
`Representation.coad(3)`	yes	$\mathrm{SU}(2)_L$ weak-isospin adjoint (triplet) index

Table 4.1: The Spenso representations used by the toolkit. Each is wrapped by one of the toolkit's own `IndexType` constants of Section 3.2 (`LORENTZ_INDEX`, `SPINOR_INDEX`, `COLOR_FUND_INDEX`, `COLOR_ADJ_INDEX`, `WEAK_FUND_INDEX`, `WEAK_ADJ_INDEX`).

The *self-dual* column reflects an important distinction between the representations introduced in Chapter 2. Let's consider, for example, a colour fundamental index, carried by a quark q^i. We also have a colour antifundamental index, carried by an antiquark $\bar{q}_i$, and this belong to dual representations. In the same way therefore `cof(3)` is dualisable, and Spenso only contracts slots with the same abstract label when one belongs to the fundamental representation and the other to its dual. By contrast, Lorentz, bispinor, and adjoint representations are treated as self-dual in the library.

Matching slots of these types may therefore be contracted directly. Once the toolkit has assigned the correct representation to each slot, Spenso can decide which repeated indices are compatible for contraction.

Building typed tensors

Now we want to understand how to build and use tensors. The first thing we want is every index label wrapped into a Spenso `Slot`. So that an index label such as `mu` or `i` is not just name. A slot records both the label and the representation to which it belongs. Only after this typing step is the tensor converted back into a normal Symbolica `Expression`, so that it can be multiplied, expanded, matched, and simplified like any other symbolic factor.

In practice, constructing a typed tensor has three steps:

1. choose the Spenso `Representation` of each index;
2. turn each abstract label into a typed `Slot`;
3. call a tensor head on those slots and export the result with `.to_expression()`.

For example, the Dirac gamma matrix is already part of Spenso's HEP tensor library. The toolkit wraps it as follows:

```
def gamma_matrix(left_spinor, right_spinor, lorentz):
    return TensorName.gamma()(
        bispinor_index(left_spinor),
        bispinor_index(right_spinor),
        lorentz_index(lorentz),
    ).to_expression()
```

Here `bispinor_index(left_spinor)` means `Representation.bis(4)(left_spinor)`, while `lorentz_index(lorentz)` means `Representation.mink(4)(lorentz)`. Thus the expression does not merely contain the labels `left_spinor`, `right_spinor`, and `lorentz`; it contains the information that the first two are Dirac spinor slots and the last one is a Lorentz slot. After the call to `.to_expression()`, this becomes a Symbolica expression, but the Spenso slot information remains encoded inside the tensor arguments.

The same pattern is used for colour generators and structure constants:

```
def gauge_generator(adj_index, fund_left, fund_right):
    return TensorName.t()(
        color_adj_index(adj_index),
        color_fund_index(fund_left),
        color_fund_index(fund_right),
    ).to_expression()
```

```
def structure_constant(a, b, c):
    return TensorName.f()(
        color_adj_index(a),
        color_adj_index(b),
        color_adj_index(c),
    ).to_expression()
```

Metrics are built in the same spirit, but directly from the representation:

```
def lorentz_metric(mu, nu):
    return LORENTZ.g(mu, nu).to_expression()
```

There is one more important example to focus on, namley not every tensor needed by the toolkit is provided by Spenso's standard HEP library. A useful example is the weak doublet invariant ε_{ij}. It is introduced by creating a new `TensorName`, and then calling it on weak-fundamental slots:

```
WEAK_EPS2 = TensorName("weak_eps2")

def weak_eps2(i, j):
    return WEAK_EPS2(
        weak_fund_index(i),
        weak_fund_index(j),
    ).to_expression()
```

This makes ε_{ij} a typed Spenso tensor just like the built-in gamma matrices or structure constants: its arguments know that they live in $\mathrm{SU}(2)_L$ fundamental representation slots. The constructor does not by itself teach Symbolica that the tensor is antisymmetric. That information is added later by the canonicalisation layer, where ε_{ij} is temporarily replaced by a head marked `is_antisymmetric=True`. This is what allows expressions such as ε_{ij} and $-\varepsilon_{ji}$ to be recognised as the same tensor structure.

This distinction is important. Typed Spenso tensors such as γ^μ, t^a, f^{abc}, metrics, and ε_{ij} carry representation information inside their arguments.

On the other hand in our code we have ordinary field `G(mu,a)` that carry indices. These are still plain Symbolica function applications. Their index meaning is stored separately in the toolkit through the field's `IndexType` metadata. The compiler and canonicalisation layers combine both sources of information: Spenso handles the typed tensor factors, while the toolkit metadata tells the pipeline how the indices on fields should be interpreted.

Tensor networks and automatic index matching

One of the most important features of Spenso is `TensorNetwork`. A product of typed tensors is a network whose edges are the shared index labels; Spenso

can *parse such a network directly from a Symbolica expression*, match indices automatically, and contract the network either numerically, using stored components, or symbolically. Data can be stored densely (`DenseTensor`) or sparsely (`SparseTensor`), and a tensor with no data attached behaves as a purely symbolic object. The toolkit mostly uses tensors symbolically, but it also extends the HEP tensor library for the custom invariants that need explicit tensor-network evaluation.

Custom invariants

Spenso provides a built-in HEP tensor library, accessed through `TensorLibrary.hep_lib()`, which contains the standard tensors such as γ matrices, σ matrices, group generators, structure constants, and metrics. The project adds its own tensor heads for invariants that are not part of this default library, such as the weak doublet ε_{ij}, the colour Levi-Civita tensor, the symmetric d^{abc}, and the charge-conjugation matrix. Only the invariants that are needed for explicit tensor-network evaluation are given numeric components in the extended library; their symmetry properties are handled separately by the canonicalisation layer.

4.3 idenso: physics-aware simplification

The third piece is **idenso**, a companion to Spenso that supplies physics-aware simplification passes over typed tensor networks. Three are used by the toolkit:

- `simplify_metrics` contracts chains of Kronecker deltas and metric tensors, e.g. $g^{\mu\nu}g_{\nu\rho} \to \delta^{\mu}{}_{\rho}$, and absorbs metrics contracted against other tensors;
- `simplify_color` applies SU(N) colour identities — Fierz-type collapses of networks of generators t^a and structure constants f^{abc};
- `simplify_gamma` performs the Clifford (Dirac) algebra on chains of γ matrices.

The toolkit composes the three into one pass, `simplify_invariants`. This run in a deliberately fixed order (metrics, then colour, then gamma, then metrics again) because each pass can expose fresh metric pairs left behind by the previous one. The advantage of idenso is that it does not perform any operations on subexpressions it does not recognise; therefore, the sequence comprising those steps can be safely called on any vertex factor. This pass, together with the tensor-canonicalisation step built on Symbolica's `canonize_tensors` (Section 5.7), is what turns the raw permutation sum produced by the contraction engine into the compact vertex that `feynman_rule(...)` finally prints.

4.4 The bridge: `IndexType`

The toolkit ties the three libraries together with one small object, `IndexType`, discussed from the user's point of view in Section 3.2. Each `IndexType` pairs a single Spenso `Representation` with a plain string *kind* (`"lorentz"`, `"spinor"`, `"color_fund"`, ...) and a default label prefix. The `Representation` is what Spenso needs to decide contractibility; the `kind` string is a cheap key that the toolkit's own compiler uses to group and match indices without repeatedly inspecting the representation object.

In summary, the division of responsibility is sharp and deliberate:

Layer	**Responsibility**
SYMBOLICA	Represent and manipulate symbolic expressions; pattern matching; polynomial normalisation; tensor/graph canonicalisation.
SPENSO	Give indices a representation and dimension; decide which indices contract; build and evaluate tensor networks.
idenso	Apply physics identities (metrics, SU(N) colour, Dirac algebra) to typed tensor networks.
`IndexType`	Bridge the above to the toolkit by pairing one representation with a stable "kind" key.

With these foundations in place, the next chapter can describe the pipeline that consumes them: how a declared `Model` is progressively rewritten into elementary interaction terms and then contracted, through Symbolica and Spenso, into a Feynman rule.

Chapter 5

Implementation: From `Model` to Feynman Rule

Chapter 3 served as a user guide, whilst Chapter 4 outlined the libraries on which FEYNPY is based. Now that we know how to use the toolkit and understand its language which, although it is Python, may not be familiar to everyone, we can move on to understanding how it works internally. How do we get from the declarative Python objects that the user writes to the Symbolica expression that emerges as a Feynman rule?

We physicists, in understanding the origin and physical meaning of a theory, make use of many conventions and simplifications (which, moreover, differ from physicist to physicist), and the code must recognise these conventions. When considering the Standard Model, for example, nobody would want to write every index, every generator and every tensor component explicitly. This would be a pointless and tedious task, and if not done with care and focus, it could even lead to unintentional errors. The toolkit is therefore intended to take a compact input and expand it according to the conventions and declarations encoded in the model.

Internally, the code will have to carry out a great deal of bookkeeping. In other words, it must be capable of expanding terms such as the covariant derivative according to the correct rules dictated by the symmetry groups that each Model possesses. It must be able to keep track of the order of the fields and their different indices. To do this, a sequence of small rewriting steps, each of which makes the expression slightly more explicit, is applied. One can visualise the structure as an hourglass: the user-facing interface allows several compact ways of declaring a Lagrangian, but these different inputs are progressively rewritten until they meet at the central object, the InteractionTerm. From this point on, the code can perform Wick contractions, canonicalisation and simplification of the resulting expression.

We want to proceed with an example in mind. Let's consider the quark

kinetic term of QCD,

$$i\,\bar{q}\,\gamma^{\mu}D_{\mu}q \;\supset\; g_s\,\bar{q}_i\,\gamma^{\mu}\,t^{a}_{ij}\,q_j\,G^{a}_{\mu}, \tag{5.1}$$

is, by the end of the pipeline, turned into the quark–gluon vertex $+ig_s\,\gamma^{\mu}\,t^{a}_{ij}$ in the conventions used here. The full gauge-fixed QCD Lagrangian,

$$\mathcal{L}_{\mathrm{QCD}} = i\,\bar{q}\,\gamma^{\mu}D_{\mu}q \;-\; \tfrac{1}{4}F^{a}_{\mu\nu}F^{a\,\mu\nu} \;-\; \tfrac{1}{2\xi}\big(\partial^{\mu}A^{a}_{\mu}\big)^{2} \;-\; \bar{c}^{\,a}\,\partial^{\mu}(D_{\mu}c)^{a}, \tag{5.2}$$

is a good stress test because it exercises every branch of the compiler at once: a covariant matter kinetic term, a field strength, a gauge-fixing term and a ghost term.

5.1 Overview of the pipeline

Let's start to look at the internals with a general overview, which is illustrated in Figure 5.1.[1] This shows the stages that lead from a declared `Model` to a Feynman-rule `Expression`. We can summarise it in four layers, see (Table 5.1).

The remaining sections walk the layers from the outside in: declaration (Section 5.2), analysis and index resolution (Section 5.3), gauge compilation (Section 5.4), the `InteractionTerm` intermediate representation that ties everything together (Section 5.5), the Wick-contraction engine (Section 5.6), and canonicalisation (Section 5.7). Section 5.8 then collects the hardest problems encountered while building the pipeline, the places where the design had to be most deliberate.

5.2 The outer layer: declaring a model

5.2.1 Inert metadata

In the chapter 3 we have seen all the building blocks of FEYNPY that will enable us to define the Model. These building blocks are the declarative *metadata* stored in `feynpy.metadata`; these are frozen dataclasses that hold physics data and carry no algorithmic logic. Let's summarise them here:

- `IndexType` pairs a Spenso representation with a "kind" string and a label prefix (more in Section 4.4);
- `Field` stores a name, spin, self-conjugacy, its tuple of `indices`, its Symbolica symbol and conjugate symbol, quantum numbers, and optional flavour-class data, inferring its `kind` (scalar/fermion/vector) and `statistics` from the spin;

[1] Each TikZ figure in this chapter has a plain-text counterpart kept as a Mermaid source diagram under `viz/mermaid/`.

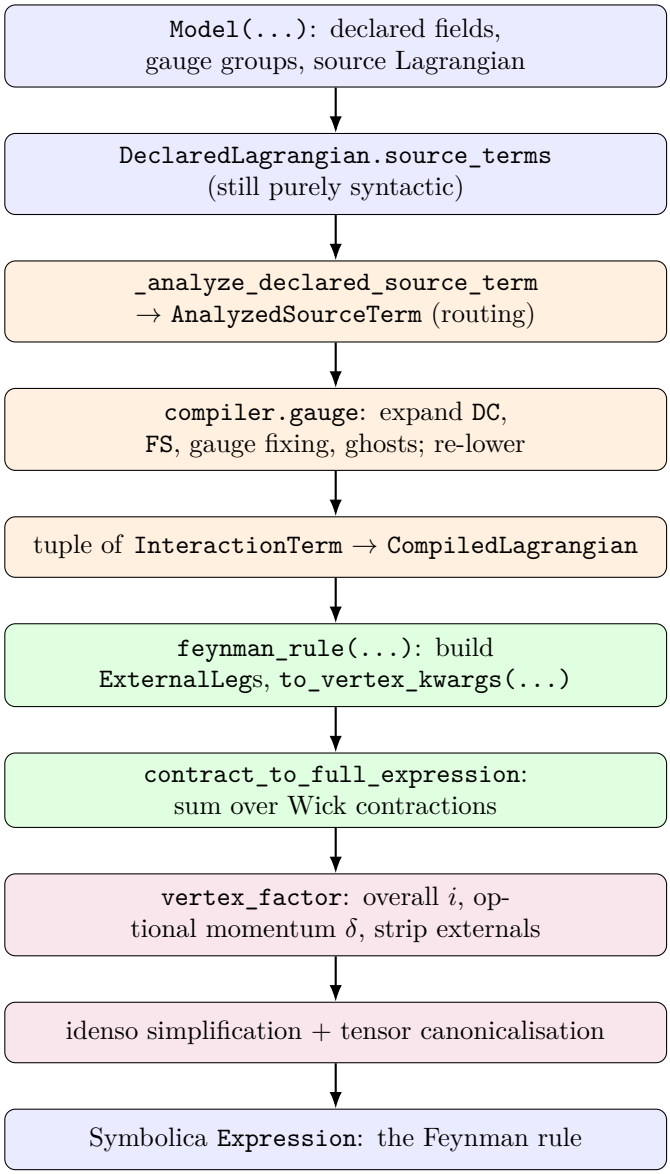

Figure 5.1: The pipeline from a declared `Model` to a Feynman-rule `Expression`. Blue = declaration layer (`feynpy`); orange = lowering/compiler layer (`feynpy.lowering`, `compiler.gauge` and helpers); green = contraction engine (`symbolic.vertex_engine`); purple = post-processing (`symbolic.vertex_postprocessing`, `symbolic.tensor_canonicalization`).

Layer	Package(s)	Responsibility
Declaration	`feynpy` (`metadata`, `declared`, `core`)	Represents the physics as the user wrote it: declared fields, gauge groups, and source Lagrangian terms, still in a mostly syntactic form.
Lowering & compilation	`feynpy.lowering`, `lagrangian`, `compiler`	Classifies and validates each source term, expands gauge structures such as covariant derivatives, field strengths, gauge-fixing terms and ghosts into local monomials, and collects the result into `InteractionTerm`s.
Contraction engine	`symbolic.vertex_engine`	Builds external legs and vertex arguments, then sums over Wick contractions to assemble the full expression.
Post-processing	`symbolic.vertex_postprocessing`, `symbolic.tensor_canonicalization`	Applies the overall factor of i, optionally extracts the momentum-conserving δ, strips external-leg factors, and simplifies and canonicalises the resulting tensor expression.

Table 5.1: The four layers of the pipeline, coloured as in Figure 5.1: declaration (blue), lowering/compilation (orange), contraction (green), and post-processing (purple).

- `Parameter` behaves algebraically like its symbol and may carry indices and explicit components;
- `GaugeRepresentation` and
- `GaugeGroup` record the symmetry data, whether a group is abelian, its coupling, gauge boson, ghost field, structure-constant builder and matter representations

The deliberate point is that metadata is inert: a faithful, machine-readable copy of what a physicist would write on paper, with all behaviour deferred downstream.

5.2.2 The declarative DSL as wrapper objects

Now that we have defined the building blocks of the theory, we can put them together into the Lagrangian itself. This is done with a small domain-specific language (DSL) defined in `feynpy.declared`:[2] `DC` (covariant derivative), `FS` (field strength), `Gamma`, `PartialD`, `Metric`, `T`, `StructureConstant`, `GaugeFixing`, `GhostLagrangian`. I want to emphasise that these constructors do not immediately build a Symbolica tensor expression. They build small immutable wrapper objects that will later be analysed and lowered into compiled interaction terms.

To understand how the code reads the Lagrangian let's consider as an example the equation 5.2. When the user writes the equation 5.2 the first term looks like this

```
I * Quark.bar * Gamma(mu) * DC(Quark, mu)
```

Obviously the Lagrangian in equation 5.2 is composed of many terms, and so the first thing the code needs to do is to distinguish each monomial and separate its scalar coefficient from its field and tensor factors. More precisely the code assembles a `_DeclaredMonomial` which distinguishes between the coefficient and an ordered tuple composed of all the other factors. In our case its content is simply a scalar coefficient (`I`) and an ordered tuple that stores every factor. In this case here a `_FieldFactor(Quark, conjugated=True)`, `GammaFactor(mu)`, `CovariantDerivativeFactor(Quark, mu)`.

A useful subtlety is that a few helpers are *dual*. `Gamma(mu)` and `T(a)` are declarative placeholders: they say "insert the gamma matrix" or "insert the generator" and let the lowering pass infer the endpoints from the neighbouring fields. By contrast, `Gamma(i, j, mu)` and `T(a, i, j)` bypass that shorthand and return the fully explicit Spenso tensors `gamma(i, j, mu)` and `t(a, i, j)`

[2] A domain-specific language is a small language designed for a particular task. Here it is a restricted set of Python objects and functions that lets the user write Lagrangian terms in a notation close to standard physics notation.

immediately. This gives the user both a compact notation without specifying every index explicitly when the structure is obvious, and a fully explicit notation when it is not.

5.2.3 `Model` and the two Lagrangian containers

Once individual source terms have been written, the full source Lagrangian is stored in the container `DeclaredLagrangian`. This is a very simple object: essentially it is just an ordered tuple `source_terms`. In the most common case these source terms are `_DeclaredMonomials`, but they can also be special declarations such as `GaugeFixing(...)` or `GhostLagrangian(...)`. So `DeclaredLagrangian` is nothing but the source-level Lagrangian, still written in the declarative language.

Everything is put together in the `Model` (in `feynpy.core`); this is the larger object that owns this declared Lagrangian together with the rest of the model data: `fields`, `parameters` and `gauge_groups`. Now the input is converted into a `DeclaredLagrangian`.

Actual compilation happens only when `model.lagrangian()` is called. At that point the declared source terms are turned into a `CompiledLagrangian`. This is where terms containing `DC(...)` or `FS(...)` are expanded using the metadata stored in the model, while terms that are already local are lowered directly. The result is no longer source syntax: it is a tuple of compiled `InteractionTerm`s. This compilation is lazy and cached, so it is only done the first time it is needed.

To summarise: `DeclaredLagrangian` is the source-level Lagrangian, `CompiledLagrangian` is the compiled one, and `Model` is the full object that stores the declared Lagrangian together with the extra information needed to validate and compile it.

5.3 Analysis and lowering: reading the Lagrangian and resolving indices

Between `DeclaredLagrangian` and `CompiledLagrangian` there is an intermediate step. The declared Lagrangian is the user-written Lagrangian, while the compiled Lagrangian is a tuple of local `InteractionTerm`s. Therefore we first need to make every implicit structure explicit, validate its index labels, and decide whether this term needs a model-dependent expansion. This process is what we call lowering.

For example, consider again the declared quark kinetic term

```
I * Quark.bar * Gamma(mu) * DC(Quark, mu)
```

Lowering checks that `mu` is used consistently as a Lorentz index and that the implied spinor and colour structure is well-formed, and classifies the term

as a `covariant_core`. This still is not a final local interaction term, because `DC` must first be expanded using the model's gauge data. The compiler then performs that expansion, producing ordinary local monomials such as the kinetic piece and the quark–gauge-boson vertex, and those local monomials are what finally become compiled `InteractionTerm`s.

So the lowering layer (`feynpy.lowering`) is the bridge between declarative syntax and the compiler. For each source term it answers three questions: what kind of term is this, are its indices well-formed, and (if the term is already local) what is its canonical interaction form?

5.3.1 Classifying each source term

In order to know what needs to be expanded, and how, by the compiler we want to analyse the terms. The single entry point is `_analyze_declared_source_term(...)`, which returns an `AnalyzedSourceTerm`. This object has the role of routing correctly every monomial through the compilation. If the term is already local, it already contains the final `InteractionTerm`; otherwise it stores the structured object that the compiler must expand. The compiler then consumes this analysed object, expanding it according to the gauge structure. The categories are summarised in Table 5.2.

Category	**Meaning**	**Example**
`interaction`	a local `InteractionTerm`; no compilation	`lam * phi**4`
`covariant_core`	canonical kinetic $\bar{\psi}\, i\gamma^\mu D_\mu \psi$ or $(D_\mu \Phi)^\dagger (D^\mu \Phi)$	quark kinetic term
`generic_covariant_monomial`	any other monomial that still contains `DC` or another covariant operator needing expansion	current $\times$ current
`gauge_fixing`	a gauge-fixing declaration	`GaugeFixing(SU3, xi)`
`ghost`	a ghost declaration	`GhostLagrangian(SU3)`
`field_strength_monomial`	a monomial built from `FS(...)` factors	$-\frac{1}{4} F^a F^a$

Table 5.2: The classification produced by `_analyze_declared_source_term`

So the analyser asks whether the term is already local; if not it proceeds with the classification. It checks if it is one of the two covariant kinetic shapes, afterwards if it is a generic covariant operator, which is a mix of covariant derivative, field strength and fields. There is also a dedicated classification for the gauge-fixing and ghost declarations (used in the declarative

helper), and finally pure field-strength monomials.

5.3.2 Resolving the indices

As explained earlier, we do not want the user to have to write out every index in the Lagrangians, as many are implicit and obvious to a physicist. In this 'lowering' stage, terms with implicit index structure are made explicit. The lowering process determines which symbolic labels belong to which field slots and which contractions the user intended to apply. A large amount of the design effort has been devoted to this stage, as it is precisely here that silent errors are most dangerous.

Monomial-wide label validation. A first validation before lowering is done. The analyser builds one registry for every label *name* appearing anywhere in the monomial (on field slots, on `PartialD`, `Gamma`, `T`, f^{abc} and `FS`, and even on raw Spenso tensors in the coefficient) and requires that each name correspond to exactly one `IndexType`. Meaning that if the analyser encounters a label `f` as a flavour index it cannot appear on another field in the monomial as an adjoint index for example. So if the same name is used for two incompatible kinds the term is rejected with an explicit error.

Slot binding and inference. Once each symbolic label has been assigned an index type (for example Lorentz, spinor, colour or flavour) the local monomial is resolved in a predefined order. Firstly, all labels that the user has entered directly into the fields are retained. Next, factors such as `Gamma` and `T`, which sit between two neighbouring fields, are resolved by identifying which field slots they connect to, to understand how to contract. Lowering replaces them by the corresponding explicit Spenso tensor, represented as a Symbolica expression, and multiplies that tensor into the coupling. Next, any other explicit labels such as those from declared factors or tensors already present in the coefficient, are matched to the remaining free field slots of the same type. This matching must be one-to-one; if multiple assignments are possible, the lowering process generates an ambiguity error. After all explicit information has been used, the process proceeds to default assignment: it assigns new labels to obvious pairs, such as an adjacent $\bar{\psi}/\psi$, then to the sole remaining compatible pair, and finally introduces new dummy labels for any slots still left open. The result is that a compact source monomial is transformed into a fully explicit indexed interaction term, or rejected with a clear error if the notation is genuinely ambiguous.

5.3.3 Remembering fermion pairings

During lowering, the code records one further piece of information: how the fermion factors are paired into closed Dirac bilinears. This information

is stored in the `closed_dirac_bilinears` metadata, a list of ($\bar{\psi}$-slot, ψ-slot) pairs. The code determines these pairings automatically from the local spinor structure of the term. When the pairing is unambiguous, the metadata preserves which $\bar{\psi}$ is paired with which ψ, so that the contraction engine can later apply the correct fermion-sign convention. Its importance is discussed in Section 5.8.

5.4 Compilation: turning gauge structure into ordinary interactions

Now the phase of full expansion of every operator of the Lagrangian is done. The compiler stage is the first stage that needs all the information stored in the model concerning the gauge symmetries. Through the lowering stage we analysed the terms and now with the compilation we turn the remaining unexpanded object into a local interaction term. The main dispatch lives in `src/compiler/gauge.py`: `_compile_declared_lagrangian_terms(model)` walks the analysed source terms and sends each one to a specialised handler (Figure 5.2). Throughout this stage the conventions are

$$D_\mu = \partial_\mu - ig\, A^a_\mu T_a, \qquad F_{\mu\nu} = \frac{i}{g}[D_\mu, D_\nu] = F^a_{\mu\nu} T_a, \tag{5.3}$$

with the adjoint basis fixed by

$$[T_b, T_c] = i\, c^a{}_{bc}\, T_a, \qquad F^a_{\mu\nu} = \partial_\mu A^a_\nu - \partial_\nu A^a_\mu + g\, c^a{}_{bc}\, A^b_\mu A^c_\nu. \tag{5.4}$$

For the usual SU(N) basis one may simply read $c^a{}_{bc}$ as the familiar f^{abc}. Extra invariant tensors of the group, such as ϵ-tensors or symmetric d-symbols, can certainly appear in local operators and coefficients, but they do not modify the definitions of D_μ or $F_{\mu\nu}$.

5.4.1 Expansion and interaction-term building

The compiler essentially performs two operations. First, it expands each structured gauge operator into simple local monomials. Only after the complete expansion are these monomials fed back into the normal lowering process, generating a `InteractionTerm` (Section 5.3).

This is a deliberate design choice: it prioritises generality and a single, uniform compilation pipeline over efficiency. For example, dedicated routines could be written for structures such as $F^a_{\mu\nu}F^{a,\mu\nu}$, $f^{abc}F^{a\nu}_{\mu}F^{b\rho}_{\nu}F^{c\mu}_{\rho}$, or contractions involving four field-strength tensors . Similar dedicated routines could also be introduced for each covariant structure separately. This would make the expansion faster and, more importantly, closer to the canonical forms already familiar from textbooks. Here, however, there is no separate hard-coded compiler for these particular contractions, or for every

Achilleas: [DONE]Here do you mean $F^2 = F_{\mu\nu}F^{\mu\nu}$ and similarly for F^3,F^4?

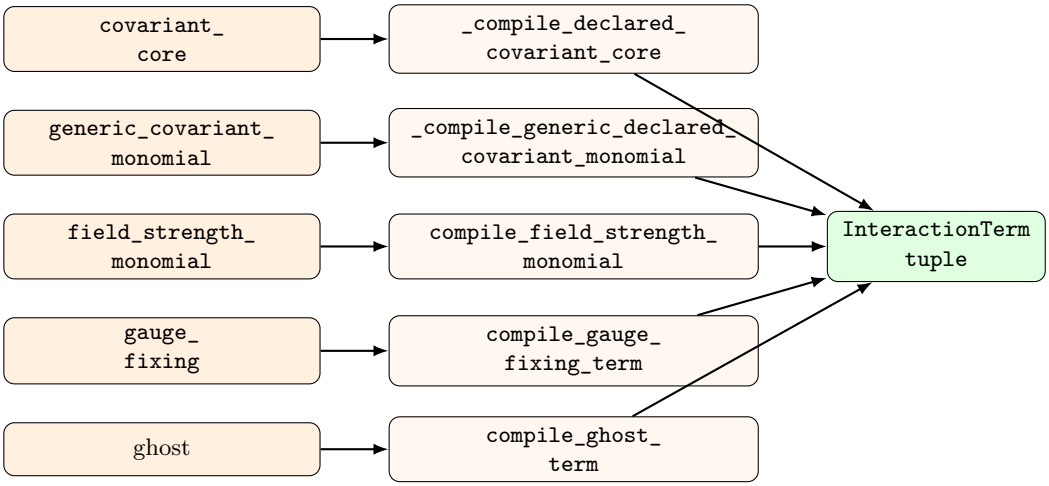

Figure 5.2: Compiler dispatch. Each analysed category is handled by one routine, and every routine returns ordinary `InteractionTerm` objects. Structured operators are first expanded into local monomials and then lowered through the same path as hand-written interactions.

possible power of a covariant derivative. Instead, all combinations are generated automatically through standard distribution and then passed to the same lowering phase.

The cost is that the intermediate output is much more verbose than textbook notation. For example, the non-abelian contraction $F^a_{\mu\nu}F^{a,\mu\nu}$ expands into nine local monomials before canonicalisation combines equivalent terms again (Section 5.7). More generally, since each non-abelian field strength contains three terms, a product of n field-strength tensors initially generates 3^n branches. Much of the effort has therefore gone into the canonicalisation process.

5.4.2 Covariant derivatives

Let us examine the expansion of the covariant derivative in more detail. For a field ϕ transforming under several gauge groups, we define the general covariant derivative as

$$D_\mu\phi = \partial_\mu\phi - i\sum_G g_G A^{a_G}_{G,\mu}\,\rho_G\big(T^{a_G}_G\big)\,\phi, \tag{5.5}$$

where the sum runs over all gauge groups G under which ϕ transforms. Here, $T^{a_G}_G$ is a generator of G, while

$$\rho_G : \mathfrak{g}_G \longrightarrow \mathrm{End}(V_G) \tag{5.6}$$

is the representation map acting on the corresponding representation slot of ϕ.

The two relevant cases are:

- **Abelian group.** There is a single generator, represented by the charge Q_G. Therefore,

$$D_\mu \phi \supset -ig_G Q_G A_\mu^{(G)} \phi. \tag{5.7}$$

- **Non-abelian group.** The gauge field carries an adjoint index a, and the corresponding generator acts on the appropriate representation index of the field:

$$(D_\mu \phi)_i \supset -ig_G A_{G,\mu}^a \left(t_{G,R}^a\right)_{ij} \phi_j. \tag{5.8}$$

Let's take as an example a field that carries representation indices associated with several gauge groups, each generator acts only on its corresponding index. For example, for a field $\phi_{i\alpha}$ transforming in the tensor-product representation $R_1 \otimes R_2$,

$$D_\mu \phi_{i\alpha} = \partial_\mu \phi_{i\alpha} - ig_1 A_\mu^a \left(t_{R_1}^a\right)_{ij} \phi_{j\alpha} - ig_2 B_\mu^b \left(t_{R_2}^b\right)_{\alpha\beta} \phi_{i\beta}. \tag{5.9}$$

The routine `expand_cov_der` implements Eq. (5.5) literally. It always emits one partial-derivative piece. It then resolves which gauge groups act on the field and emits one gauge piece for each of them; if the user did not name a group, it uses all compatible groups. For a conjugated occurrence $(D_\mu \phi)^\dagger$, the sign of the gauge piece flips.

More slots in the same representation. The user might want to play with multiple slots in many representations that could be the same, in which case the generator may be able to act on more than one place. The code handles these cases. The default policy for this situation is conservative and rejects it as ambiguous. A representation can instead opt into `slot_policy='sum'`, in which case the compiler sums over the possible active slots.

Several gauge groups at once. A standard example is the left-handed quark doublet Q_L, which transforms under $\mathrm{SU}(3)_c \times \mathrm{SU}(2)_L \times \mathrm{U}(1)_Y$. In that case

$$D_\mu Q_L = \partial_\mu Q_L \; - \; ig_s \, G_\mu^a \, t_{(3)}^a \, Q_L \; - \; ig \, W_\mu^I \, t_{(2)}^I \, Q_L \; - \; ig' \, Y_Q \, B_\mu \, Q_L, \tag{5.10}$$

where we have $t_{(3)}^a$ as the colour generators, $t_{(2)}^I$ as the weak generators and Y_Q as the hypercharge. So `DC(Q_L, mu)` expands to four pieces: the ordinary derivative plus one gauge current for each of the three groups. Naming one group, for example `DC(Q_L, mu, gauge_group=SU3C)`, keeps only that group's current together with the partial derivative.

5.4.3 Field strengths

Let us examine the expansion of field strengths in more detail. As for the covariant derivative, the compiler expands `FS(...)` using the gauge-group data of the model so that it becomes a sum of local terms.

Using the convention

$$D_\mu = \partial_\mu - igA_\mu, \tag{5.11}$$

with

$$A_\mu = A^a_\mu T_a, \qquad [T_b, T_c] = ic^a{}_{bc} T_a, \tag{5.12}$$

the Lie-algebra-valued field strength is

$$F_{\mu\nu} = \partial_\mu A_\nu - \partial_\nu A_\mu - ig[A_\mu, A_\nu] = F^a_{\mu\nu} T_a, \tag{5.13}$$

with components

$$F^a_{\mu\nu} = \partial_\mu A^a_\nu - \partial_\nu A^a_\mu + g\, c^a{}_{bc} A^b_\mu A^c_\nu. \tag{5.14}$$

In the DSL, this is written as `FS(G, mu, nu, a)` for a non-abelian gauge group G, while for an abelian gauge group one writes `FS(G, mu, nu)`.

A single non-abelian `FS` factor is first rewritten as the sum

$$F^a_{\mu\nu} = P_1 + P_2 + P_3, \tag{5.15}$$

where

$$P_1 = \partial_\mu A^a_\nu, \qquad P_2 = -\partial_\nu A^a_\mu, \qquad P_3 = g\, c^a{}_{bc} A^b_\mu A^c_\nu. \tag{5.16}$$

Note that for an abelian field strength, only P_1 and P_2 are present.

A monomial containing several field-strength factors is expanded by taking the full Cartesian product of all the different pieces.

To understand it more, let's consider the iconic example of the pure Yang–Mills Lagrangian

$$\mathcal{L}_{\mathrm{YM}} = -\frac{1}{4} F^a_{\mu\nu} F^{a\,\mu\nu}. \tag{5.17}$$

Each non-abelian field strength contributes three pieces, so their product generates

$$3 \times 3 = 9 \tag{5.18}$$

raw branches before canonicalisation and collection. These branches belong to three sectors:

Piece combination	**Count**	**Sector**
$(P_1, P_2) \times (P_1, P_2)$	$2 \times 2 = 4$	quadratic kinetic sector
$(P_1, P_2) \times P_3$ and $P_3 \times (P_1, P_2)$	$2 + 2 = 4$	cubic gauge interaction
$P_3 \times P_3$	1	quartic gauge interaction

Each resulting branch is then lowered in the same way as an ordinary local interaction.

Note that a product of n non-abelian field strengths produces 3^n raw branches, whereas a product of n abelian field strengths produces 2^n branches. More generally, a mixed product with m non-abelian and k abelian field strengths produces $3^m 2^k$ branches.

5.4.4 Gauge fixing and ghosts

There is a declarative helper for the gauge fixing term which can be accessed by `GaugeFixing(gauge_group, xi=...)`. This helper implements the linear covariant gauge term

$$\mathcal{L}_{\text{gf}} = -\frac{1}{2\xi} \left(\partial^\mu A^a_\mu\right) \left(\partial^\nu A^a_\nu\right), \tag{5.19}$$

with the adjoint index omitted in the abelian case. It constructs the corresponding explicit local monomial, lowers it through the ordinary compilation pipeline, and tags the result as `gauge_fixing` so that the reporting and validation layers can recognise it. The user can naturally write the gauge fixing manually, in which case it is recognised and receives the same tag.

Note that the generic `GaugeFixing(...)` helper describes only linear covariant gauge fixing. More general R_ξ gauges in spontaneously broken theories, for example, require additional Goldstone-dependent gauge-fixing terms, which must be written explicitly by the user.

There is also the ghost Lagrangian helper `GhostLagrangian(gauge_group)`. For a non-abelian gauge group, the Faddeev–Popov ghost Lagrangian can be written as

$$\mathcal{L}_{\text{gh}} = -\bar{c}^a \, \partial^\mu \left(D_\mu c\right)^a . \tag{5.20}$$

Up to a total derivative, integration by parts gives

$$\mathcal{L}_{\text{gh}} \simeq \left(\partial^\mu \bar{c}^a\right) \left(D_\mu c\right)^a , \tag{5.21}$$

where

$$\left(D_\mu c\right)^a = \partial_\mu c^a + g \, c^a{}_{bc} A^b_\mu c^c. \tag{5.22}$$

The ghost sector therefore becomes

$$\mathcal{L}_{\text{gh}} \simeq \left(\partial^\mu \bar{c}^a\right) \left(\partial_\mu c^a\right) + g \, c^a{}_{bc} \left(\partial^\mu \bar{c}^a\right) A^b_\mu c^c. \tag{5.23}$$

The compiler turns the ghost sector into two ordinary local terms: the ghost kinetic term and the ghost gauge interaction term. Since the ghost field transforms in the adjoint representation, the code can use the same gauge-action and index-contraction machinery that it uses for other adjoint fields. What remains specific to ghosts is not the contraction structure, but their interpretation: they still carry Grassmann statistics, a fixed field ordering, and ghost number.

5.5 The pivot: the `InteractionTerm` intermediate representation

To return to the analogy used at the start of the chapter, we are now at the waist of the hourglass. Every branch of the code converges and passes through this data type: `InteractionTerm` (in `feynpy.interactions`). Everything above this narrow point is declarative, everything below it is generic symbolic machinery.

```
@dataclass(frozen=True)
class InteractionTerm:
    coupling: object                          # Symbolica Expression
    fields: tuple[FieldOccurrence, ...]     # bare field factors, in
    ↪  order
    derivatives: tuple[DerivativeAction, ...] = ()
    closed_dirac_bilinears: tuple[tuple[int, int], ...] = ()
    label: str = ""
    sector: str = ""
```

The `coupling` is a Symbolica expression holding the numerical and tensor prefactor (couplings, generators, structure constants, metrics); `fields` is an ordered tuple of `FieldOccurrence` objects, each a field with its slot-resolved index labels and conjugation flag; `derivatives` records, as a `DerivativeAction`, which field each derivative hits and along which Lorentz index; and `closed_dirac_bilinears` carries the fermion-pairing provenance of Section 5.3. A `CompiledLagrangian` is simply a tuple of such terms. An `InteractionTerm` is a purely static, index-labelled description of one monomial of the Lagrangian. This is the direct code counterpart of the right-hand side

$$g_{\alpha_1\cdots\alpha_n}\,\partial\cdots\partial\,\phi^{\alpha_1}\cdots\phi^{\alpha_n}$$

..

As we know the ordering of `fields` is relevant and since Symbolica multiplication is commutative, we use the ordered tuple which keeps record of fermion order. The method `to_vertex_kwargs(external_legs)` performs the last model-aware step: it strips away all model-layer vocabulary and reduces one term plus a chosen set of external legs to flat, parallel lists. After which only Symbolica and Spenso objects remain.

Flavour expansion

When the field carries a flavour index (Section 3.5) by default the terms remain generic. For example $\lambda_{ij}\,\bar{\ell}_i\ell_j\,\phi$ will keep the lepton. By requesting `flavor_expand=True` FEYNPY enumerates every member combination, substitutes the corresponding `Parameter` component (e.g. $\lambda_{e\mu}$) producing one fully explicit `InteractionTerm` per combination in the same data structure.

Field transformations

There is a second rewriting that lives at this layer. A `FieldTransformation` (Section 3.13) is applied not to the declared source terms but to the compiled ones: each `InteractionTerm` is visited, every `FieldOccurrence` matching a rule is replaced by the corresponding expression, and the term is rebuilt.

Placing the stage here, rather than earlier, is a deliberate choice with two consequences. First, `DC` and `FS` have already been expanded using the gauge-group metadata of the original basis, which is the only basis in which that metadata is meaningful, in fact B_μ has a hypercharge, the photon does not. Second, because the transformation operates on an `InteractionTerm`, it inherits the structure that the term carries: derivatives recorded in `derivatives` are distributed over a replacement product by the Leibniz rule automatically, conjugated occurrences receive the conjugated replacement, and the ordering that fixes fermion signs is preserved. A rule that maps one field to a sum simply multiplies the number of terms, and canonicalisation collects them again afterwards.

In the Standard Model build, the transformation stage is run with all rules acting simultaneously, so that a field introduced by one rule is not consumed by another in the same pass. The result is an ordinary `CompiledLagrangian`, indistinguishable in type from the one that entered, and vertex extraction proceeds from it unchanged.

5.6 The inner core: Wick contraction in the vertex engine

Now we arrive at the heart of the code, where the actual algorithm is implemented. We find the implementation of Wick contraction in the vertex engine (`symbolic.vertex_engine`). The only thing this engine cares about are fields, roles, index labels, momenta and a coupling tensor, and performs a combinatorial contraction that is the code-level realisation of the canonical-quantisation algorithm of Section 2.5.

5.6.1 The permutation sum

The core routine is `contract_to_full_expression`. Given an interaction with n field slots and n external legs, it sums over all $n!$ ways of matching legs to slots. Each permutation `perm` assigns leg `perm[i]` to slot `i`; the sum of the surviving contributions is the full Wick contraction.

For each permutation the code does the following things in order. First it remaps every open index in the coupling to the corresponding external-leg label, this ensures that the coupling will actually match the permuted fields. Then it checks whether the pairing is physically allowed, this is done using `factor_leg_compatible`. When it happens that a slot and a leg disagree in

species, spin, statistics or conjugation, the permutation is discarded at once. Surviving fermionic terms pick up a Grassmann sign from the fermion part of the permutation, unless the closed-bilinear exception of Section 5.6.2 applies. Then derivatives are turned into momenta because of the Fourier rule $\partial_\mu \to -ip_\mu$ (all momenta incoming), so each `DerivativeAction` contributes a factor $(-i)\,p^\mu$ with p the momentum of the matched leg (meaning that it will carry the corresponding label). Finally each leg contributes its external wavefunction (U for bosons, $UF/\overline{UF}$ for fermions) and a species Kronecker delta, repeated indices are contracted with the Spenso metric, and the contribution is multiplied by the plane wave $\exp(-i\,\Sigma p\cdot x)$ before being added to the total.

Bosonic vertices carry no extra signs; fermionic ones do. The structure of the loop is:

```
for perm in permutations(range(n)):
    term = coupling_with_open_indices_remapped_along(perm)
    if not all(factor_leg_compatible(i, perm[i], ...) for i in range(n)):
        continue                        # prune impossible pairing
    if statistics == "fermion" and not use_closed_dirac_bilinear_signs:
        term *= fermion_sign(perm, fermion_slots)
    for mu, tgt in zip(derivative_indices, derivative_targets):
        term *= (-I) * pcomp(ps[perm[tgt]], mu)
    for i, j in enumerate(perm):
        term *= external_factor(role[i], alphas[i], betas[j], ps[j], ...)
    term *= plane_wave(sum_of_matched_momenta, x)
    total += term
```

Note that nothing here depends on a particular index kind. An index is only a `(kind, label)` pair.

5.6.2 Fermion signs and the closed-bilinear compatibility exception

FeynPy uses the standard ordered-Grassmann convention. A Lagrangian monomial is treated as an ordered product of elementary fields, and fermion fields are Grassmann odd. Exchanging two fermionic contractions therefore gives a minus sign. The pipeline records the field order from the beginning, so the vertex engine can still apply this sign at the final Wick-contraction stage.

In code this means that each contraction receives the flat Grassmann sign of the permutation restricted to the fermionic slots. This sign is the FeynPy convention. For example, consider the identical-fermion current–current operator

$$(\bar{\psi}\,\gamma^\mu\psi)(\bar{\psi}\,\gamma_\mu\psi). \tag{5.24}$$

It has a *direct* contraction, where each current stays intact, and a *crossed* contraction, where the two currents exchange one identical fermion line. In the ordered-Grassmann convention, the one used by FeynPy, the exchange is odd, so the result is

$$\text{direct} - \text{crossed}.$$

FeynRules gives a different answer for this special structure:

$$\text{direct} + \text{crossed}.$$

FeynRules effectively treats the two closed bilinears as bosonic composite factors before resolving the identical-fermion Wick contractions. Doing so loses the minus sign from exchanging the elementary fermion fields.

Achilleas: [DONE]what do you mean here by correct'? I think that this is a case where Feynrules is just wrong.

5.6.3 Output policy and universal factors

There is a function, namely `vertex_factor`, that has the role of running the contraction and then applies a few presentation choices. By default it amputates the external wavefunctions U, UF and $\overline{UF}$, so the user sees the bare vertex. Optionally, the user can choose to write `include_delta=True`, which gives the momentum-conservation factor $(2\pi)^d\,\delta(\Sigma p)$. In every case the result is finally multiplied by the overall $+i$ that every Feynman rule carries. These choices only decide how the answer is shown.

5.7 Making the answer canonical

The raw contracted expression is correct, but not unique. The same vertex can appear with different dummy-index names, different orderings of antisymmetric tensors, or still expanded into many field-strength pieces. In the next paragraph some choices of the postprocessing of the final output are presented.

The first pass, in `symbolic.vertex_postprocessing`, does the physics-facing clean-up. As the output comes out of the contraction engine it has the full range of Kronecker deltas of the permutation. The first step is therefore to contract species Kronecker deltas, collapse repeated bispinor indices with the Spenso metric, and, on request, simplify closed gamma chains with idenso (Section 4.3).

The second pass, in `symbolic.tensor_canonicalization`, puts the surviving expression into a unique tensor form using Symbolica's `canonize_tensors` (Section 4.1). That routine only understands symmetry flags on plain symbols, not on Spenso tensor heads. So the code temporarily replaces every Spenso tensor — metrics, generators, structure constants, γ matrices, and the project's custom invariants — by a plain symbol with the right symmetry, runs the canonicalisation, and swaps the Spenso tensors back. Dummy

indices of different kinds (Lorentz, colour, spinor, weak) are kept in separate groups, so a Lorentz dummy is never confused with a colour one.

On top of this, `canonize_full` adds a few physics reductions used when checking identities such as gauge variation and BRST (Sections 3.17 and 3.18): it makes flat derivatives commute, reduces products of structure constants with the Jacobi identity, and drops Yang–Mills terms that vanish by the antisymmetry of f^{abc}.

Together these passes turn the over-expanded compiler output into a compact, comparable Feynman rule.

5.8 Where the difficulty lived

Building the pipeline was less about one clever algorithm than about a few recurring hard problems. These are the places where most of the design effort went, and they shaped the architecture.

Index bookkeeping. The hardest bugs came from indices, not from algebra. Early code keyed labels only by index kind, which fails as soon as a field carries two slots of the same kind, For example two Lorentz indices, or two colour indices. The fix is now layered: labels are packed by slot, each label name is bound to one index type across the whole monomial, ambiguous assignments are rejected rather than guessed, and gauge representations expose an explicit `slot_policy`. The guiding rule is to fail early: an ambiguous index assignment is an error at declaration time, never a silent wrong contraction.

Fermion signs. Matching the FeynRules convention for products of closed Dirac bilinears, without breaking generic multi-fermion terms, required carrying the bilinear pairing from lowering all the way into the contraction engine (Section 5.6.2). Getting that boundary right took careful checks on benchmark operators and is now a fixed convention.

Making expansion generic. Replacing a special-case F^2 handler with a generic `FS` expansion, and likewise for covariant derivatives, unlocked F^3, F^4 and mixed-group operators. The cost was many more raw local monomials, and a performance problem in the contraction engine: the old compatibility check compared only broad field roles and therefore tried many impossible pairings. The species-aware pruning of Section 5.6 fixed that without changing any resulting formula.

Keeping the layers honest. Finally, much of the work went into not letting the layers leak into each other: declarative syntax must never reach the

contraction engine, the engine must never know what a covariant derivative is, and comparison against external references must live outside the engine. One job per layer, and one intermediate representation at the waist, is what made it possible to validate the packaged Standard Model against the reference export vertex by vertex.

5.9 Summary

The pipeline is a chain of small, inspectable rewrites. A `Model` holds the source Lagrangian and the inert data of the theory. Lowering classifies each term, checks its indices, and turns local terms into `InteractionTerm`s. The compiler expands gauge structure, covariant derivatives, field strengths, gauge fixing and ghosts, into ordinary local monomials and lowers them through the same path. Every branch therefore meets at the same intermediate representation. The symbolic engine then contracts it by a pruned permutation sum, applies momenta, external factors, fermion signs and the overall $+i$, and canonicalisation makes the answer unique. The hard parts, like indices, fermion signs, conventions, generic expansion and layer boundaries, are exactly where the design had to be most careful, and they are what the validation in the next chapter rests on.

Chapter 6

Validation and Final Comparison

Chapter 5 explained how FeynPy calculates Feynman rules from a model definition and a Lagrangian. A significant portion of this project subsequently focused on verifying the correctness of FeynPy and its ability to reproduce existing FeynRules models.

The two results are not expected to be *identical*: different expansions and tensor identities imply that they need not match term by term. Instead, we verify whether they are the same expression from an algebraic point of view, differing only in the renaming of indices, the canonical order of tensor factors, and the explicitly listed set of tensor and algebraic identities. Section 6.2 clarifies this concept and illustrates, with examples, how the resulting equality test works. Bringing both sides into a common canonical form to establish equality was itself one of the most complex parts of this work.

The results obtained within the limited timeframe of this thesis are as follows. The packaged Standard Model matches its FeynRules export exactly; the unbroken background Standard Model matches its own export exactly; and the Green-basis SMEFT model matches all 184 exported lines of FeynRules' `Ltot` for the EFT sector at the operator-content level. Of these 184 SMEFT vertices, 161 match directly, 15 match after one globally fixed charge-conjugation convention, and 8 match after an additional transformation that is explicitly tabulated per line.

6.1 Validation strategy

Model validation has three levels. *Model fidelity* (Section 6.4) checks, block by block and coefficient by coefficient, that the FeynPy source transcribes the operators of the FeynRules source. *Vertex equality* (Sections 6.2, 6.5 and 6.6), the core of the comparison, verifies whether the Feynman rules

derived from the two Lagrangians agree as tensor expressions. *Coverage* (Section 6.9) checks how much of the model the equality test can actually reach, since the FeynRules export only covers a finite set of vertex arities.

All three levels are necessary and interdependent. In fact, without fidelity, a positive equality result could simply confirm an incomplete transcription. Alternatively, without coverage, it might reflect a correspondence limited to a convenient subset of vertices. Together, these three levels define the validation claim: the implemented model is faithful to the source, the compared vertices coincide algebraically, and the declared coverage clarifies which parts of the model have actually been tested.

6.2 What "equal" means: the canonical form

Before the comparison can begin, the FeynRules output is loaded from the JSON export as text in Mathematica syntax, while the FeynPy side is generated by calling `feynman_rule(...)` on the corresponding fields. Both sides are then parsed into Symbolica tensor expressions. At this point the comparison is not yet done: two algebraically equal tensor expressions might still appear different because they use different dummy index names, list the commutative factors in a different order, or expand the same algebraic object in different ways.

We first clarify two pieces of terminology that are used throughout the comparison. The *signature* of a row is the external-field content, written for example as `B|dR|dRbar`. The *coefficient head* is the name of the Wilson parameter multiplying a term, for example `alphaEcll`. In the SMEFT comparison, equality is checked one coefficient head at a time inside a fixed signature: the terms proportional to `alphaEcll` are compared only against terms proportional to `alphaEcll`.

For SMEFT, the object actually compared is a *canonical tensor-monomial map*. One entry of this map has the form

$$\text{canonical tensor monomial} \quad \longmapsto \quad \text{exact scalar coefficient.} \tag{6.1}$$

The monomial key records the tensor factors, their typed indices, and the momenta. The scalar coefficient records everything that is scalar with respect to the Lorentz, spinor, and gauge tensor spaces: numbers, signs, powers of couplings, and symbolic flavour or Wilson coefficients, including their flavour order and complex conjugation. Two rows are equal only if these maps are identical entry by entry.

This canonical form is not unique in an absolute mathematical sense. It depends on the chosen tensor identities, ordering conventions, and tensor basis. The purpose is to obtain a deterministic representation within that declared basis, not to perform maximal simplification.

The canonical map is built in four stages.

1. **Register tensor metadata.** Every tensor head has all the information needed to canonicalise it. For example, the metric is symmetric, while the Lorentz epsilon tensor, the structure constants f^{abc}, the weak epsilon tensor, and the charge-conjugation matrix C are antisymmetric.

2. **Split each term into scalar and tensor parts.** In a product such as
$$2\,\alpha(f_1, f_2)\, g_3^2\, f^{abe} f^{cde}\, p_\mu,$$
the factor $2\,\alpha(f_1, f_2)\, g_3^2$ belongs to the scalar coefficient, while $f^{abe} f^{cde} p_\mu$ belongs to the tensor monomial.

3. **Choose one representative for dummy indices and factor order.** The code enumerates allowed relabellings of dummy indices within each index type, applies the tensor symmetries from step 1, sorts commuting tensor factors, and keeps the lexicographically smallest resulting key. External indices are held fixed. This step is what makes dummy names irrelevant while keeping genuine external labels meaningful.

4. **Collect equal keys.** If two expanded terms reduce to the same canonical key, their scalar coefficients are added exactly. If the sum is zero, the key disappears. The final result is therefore a dictionary of nonzero canonical tensor monomials and exact coefficients.

After these generic stages, the implementation applies the generator-order, $SU(2)$, and Jacobi rewrites before comparing the maps.

Example 1: dummy names and symmetry. Consider
$$g^{\mu\nu}\, p_\mu k_\nu \qquad \text{and} \qquad g^{\rho\sigma}\, k_\rho p_\sigma. \tag{6.2}$$
As printed expressions they differ: the contracted Lorentz labels have different names and the momenta are written in the opposite order. The canonicaliser first uses the symmetry of the metric, then relabels the contracted dummy indices and sorts the commuting momentum factors. Both expressions therefore produce the same schematic map entry,
$$\{g(L_1, L_2),\, p(p, L_1),\, p(k, L_2)\} \longmapsto 1,$$
where L_1, L_2 denote canonical Lorentz dummy labels. The exact names μ, ν, ρ, σ disappear because they are summed dummy indices; the momentum labels p and k remain because they are external labels, not dummy indices.

Example 2: generator ordering. A covariant-derivative expansion can leave two generators in different orders on the two sides, for example $(T^b)^i{}_j(T^a)^j{}_k$ on one side and $(T^a)^i{}_j(T^b)^j{}_k$ on the other. These generators are noncommuting, so they are not included in the ordinary commuting-factor sorting. Instead the comparison uses the single Lie-algebra identity

$$[T^a, T^b] = if^{abc}T^c, \tag{6.3}$$

implemented by `_normalize_generator_product_order_report`. If the adjoint labels are out of canonical order, the product is rewritten as the ordered product plus the commutator term:

$$(T^b)^i{}_j(T^a)^j{}_k \quad \longrightarrow \quad (T^a)^i{}_j(T^b)^j{}_k - if^{abc}(T^c)^i{}_k,$$

with the sign fixed by the commutator convention. This is not a general search over all possible reorderings; it is one deterministic rewrite to a fixed generator basis.

Example 3: the Jacobi identity. The Jacobi step is similarly narrow. The code does not try to simplify arbitrary products of structure constants; it looks only for a pair of structure constants of the same adjoint representation sharing exactly one dummy adjoint index, with four remaining free labels. After antisymmetry has put each f in canonical label order, the four free labels are sorted and called a, b, c, d, while the shared dummy label is called e. There are then only three pairings:

$$p_{12} = f^{abe}f^{cde}, \qquad p_{13} = f^{ace}f^{bde}, \qquad p_{14} = f^{ade}f^{bce}.$$

The only Jacobi relation used is

$$p_{12} - p_{13} + p_{14} = 0, \qquad \text{so} \qquad p_{13} = p_{12} + p_{14}. \tag{6.4}$$

The canonical basis is chosen to be $\{p_{12}, p_{14}\}$; this is a conventional tensor basis, not a physically preferred one, and another independent pair would also work if used consistently. An occurrence of the middle pairing p_{13} is replaced by the sum $p_{12} + p_{14}$, including the signs induced by any antisymmetric permutation of the labels. Occurrences already in the p_{12} or p_{14} basis are left alone. If a product has no such eligible pair, it is left unchanged and the row then has to match without this identity or it fails.

For example, two outputs can differ as

$$E_{\mathrm{FR}} = 3p_{12} + 2p_{14}, \qquad E_{\mathrm{FP}} = p_{12} + 2p_{13}.$$

Using Equation (6.4), the FeynPy form reduces to

$$E_{\mathrm{FP}} = p_{12} + 2(p_{12} + p_{14}) = 3p_{12} + 2p_{14} = E_{\mathrm{FR}}.$$

Thus Jacobi can relate genuinely different tensor networks; in such rows canonicalisation needs reduction to the chosen tensor basis, not only dummy-index relabelling and factor sorting.

The same restricted machinery is also used for the weak-generator identities of $SU(2)$.

6.3 Standard Model & unbroken background-field benchmark

The Standard Model package in `models/SM/` is the first complete validation benchmark. It implements a gauge-basis declaration, field transformations to the physical basis, and a comparison of the resulting interaction vertices against the bundled FeynRules `SM.fr` export.

The result is 163/163 exact symbolic matches for the exported nonzero flavour-expanded tree-level vertices, split by sector in Table 6.1.

Sector	Vertices
Gauge self-interactions	8
Matter currents	51
Yukawa	42
Higgs and Goldstone	38
Ghost	24
Total	163

Table 6.1: Sector split of the Standard Model FeynRules comparison. Every row in every sector is a direct exact symbolic match.

The comparison runs with ghosts included and gauge fixing disabled, so it validates the gauge, matter, Yukawa, Higgs/Goldstone and ghost sectors but *not* the gauge-fixing sector. It also makes no claim about parameter-card semantics, loop-level output, restriction files, or external formats such as UFO.

Two particular relations needed to be handled carefully. The Higgs/-Goldstone and ghost sectors are compared modulo the weak-mixing relation $c_W^2 + s_W^2 = 1$. In fact, once the physical basis is used, the two implementations can distribute this identity differently across a rule. The ghost sector additionally uses momentum conservation on its external legs, since a ghost rule is fixed only up to which leg momentum is eliminated.

A second, independent benchmark is the unbroken background-field Standard Model in `models/UnbrokenSM_BFM/`, matching its own FeynRules reference at 67/67 (42 three-point and 25 four-point vertices). This matters for SMEFT, which is formulated in the same unbroken basis: it shows that

the basis, field naming, and generator conventions are already validated independently of the EFT operators.

6.4 SMEFT: model fidelity

Ever since I started my thesis, we have been discussing a very ambitious goal: developing a toolkit capable of implementing SMEFT. That goal has now been achieved, and I will present the benchmark results. The SMEFT model with Green basis is implemented in `models/SMEFT/SMEFT.py`. This has been compared with the `Ltot` export from FeynRules, which comprises EFT only and is available in `models/SMEFT/reference/Ltot_SMEFT_FeynRules.json`. The FeynPy `Ltot` model follows the same convention, i.e. the EFT contribution only, whilst the SM plus EFT combination is available separately as `Lfull`.

Before comparing any vertices, the models transcription itself was checked to verify that FeynPy had all the capabilities required to implement the operators. All 32 EFT Lagrangian blocks in `SMEFT_Green_Bpreserving.fr` have been implemented, and `Ltot` consists exactly of those 32 blocks in the same order. All 298 active Wilson coefficients in the FeynRules source code are declared in the FeynPy model and used in at least one operator. Two further coefficients, `alphaEcdlq` and `alphaEcqeu`, appear in the FeynRules file but not in FeynPy, and both are commented out in FeynRules and are not used in any FeynRules operator, so their omission is correct rather than a gap. (This is an observation regarding the transcription, not the coverage of the comparison: of the 298 declared coefficients, the comparison of vertices in Section 6.5 covers 295, for the structural reason indicated in Section 6.9.) The core of the Standard Model is taken from `UnbrokenSM_BFM.fr`, minus its background-field ghost and gauge-fixing sectors, which have been deliberately excluded since the subject of the comparison is the `Ltot` based exclusively on EFT.

The structural differences between the two sources are systematic conventions, not per-operator adjustments, as summarised in Table 6.2. This matters for Section 6.6: because the FeynPy Lagrangian contains no per-operator hand-fitting, a residual disagreement is evidence of a *convention* difference rather than of a transcription error.

Direct syntax translations such as `FS[...]`, `DC[...]`, colour generators and ordinary field calls are not listed separately in the table, because they do not change the algebraic convention being implemented.

6.5 SMEFT: comparison result

All Green-basis and evanescent sectors used by the bundled reference are included in the compiled FeynPy `Ltot`. The final result is given in Table 6.3.

FeynRules	FeynPy	Nature
`2 Ta[...]`	`weak_t`	weak-generator normalisation
`Phitilde`, `Phitildebar`	`phitilde`, `phitildebar`	weak-ϵ expansion
`sigmamunu`	`sigma_term`, `sigma_matrix`	spinor-chain helpers
`Dual[FS]`	`_dual_fs`, `_dual_covd_fs`	dual field-strength helpers
`lag + HC[lag]`	explicit `.conj()` terms	expansion
`CC[...]`	`dirac_charge_conjugation`	expansion

Table 6.2: Systematic convention differences between the FeynRules source and the FeynPy transcription. No comparison-tuning comments, fudge factors, or per-operator sign corrections appear in the model file.

Quantity	Result
FeynRules reference rows	184
Operator-content matches	184/184
Direct exact symbolic matches	161/184
Matches modulo the global `Ec` convention	15/184
Matches modulo pinned row-specific packaging	8/184
Unresolved packaging rows	0/184
Exact symbolic unequal rows	0
Exact symbolic error rows	0
Bosonic canonical tensor-map matches	32/32
Bosonic canonical coefficient sectors	93/93
Chirality-validated projectors	2998

Table 6.3: Final SMEFT comparison summary for the EFT-only Green-basis reference. The three-way exact-symbolic split is graded as described in Section 6.6.

We define a correspondence between an operator and its content when every contribution from the Wilson coefficients generated by FeynRules can be mapped to the same canonical tensor-monomial map in FeynPy. The two raw outputs need not necessarily contain the same number of terms nor use the same field packaging.

We distinguish between three types of correspondence. A direct correspondence requires no additional conventions: the field signature is the same and the canonical tensor-monomial maps coincide. A correspondence modulo the global `Ec` convention coincides after applying the single charge-conjugation transformation defined in Section 6.6. Finally, a fixed-packaging correspondence requires an additional, row-specific mapping, established in advance, which includes the partner signature, the phase and the handling of duplicate-leg symmetry.

6.6 Charge-conjugated fermion structures

We now focus specifically on the fermionic sector, in particular on the evanescent[1] four-fermion operators with charge conjugation. These terms are accompanied by coefficients whose names begin with `alphaEc`. Charge conjugation is handled differently by FeynRules and FeynPy, and this creates an apparent discrepancy between the two outputs that must be considered carefully.

FeynRules decomposes `CC[...]` into its component fields, whereas FeynPy retains the structure as an explicit tensor C. Without a convention to reconcile this difference, direct comparison via the canonical map rejects all 21/21 four-fermion `Ec` rows, not because the operator content is inconsistent, but because of this difference in representation. The rest of this section defines the translation between the two representations. The comparison then separates this issue into one global convention and a small number of exceptional packaging cases.

6.6.1 Global `Ec` convention

The comparison therefore translates the explicit-C representation used by FeynPy into the fermion-flow representation appearing in the FeynRules output. A representative FeynRules source term is

```
alphaEcll[f1,f2,f3,f4] CC[LLbar[sp1,ii,f1]].LL[sp1,jj,f2]
                       .LLbar[sp2,jj,f3].CC[LL[sp2,ii,f4]]
```

[1]The term "evanescent" refers here to operator structures required by the dimensional-regularisation scheme, but redundant in strictly four dimensions.

After `CC[...]` is expanded, the exported FeynRules vertex contains no residual charge-conjugation matrix: its spinor chains are resolved into the crossed pairing $(1,4),(2,3)$.

The corresponding FeynPy term keeps the charge-conjugation tensors explicit:

```
p["alphaEcll"](f1, f2, f3, f4)
* ll(sp1, w1, f1, bar=True)
* dirac_charge_conjugation(sp1, sp2)
* ll(sp2, w2, f2)
* ll(sp3, w2, f3, bar=True)
* dirac_charge_conjugation(sp3, sp4)
* ll(sp4, w1, f4)
```

This is the same operator content, but the spinor network is packaged as adjacent explicit-C contractions, $(1,2),(3,4)$.

Accounting for the corresponding charge-conjugation and fermion-flow conventions gives the fixed translation

$$(1,2),(3,4) \longrightarrow -(1,4),(2,3). \tag{6.5}$$

The minus sign combines the antisymmetry of the charge-conjugation matrix, $C^T = -C$, with the fermion-field reordering needed to form the crossed pairing.

The same translation is applied uniformly to the complete four-fermion `Ec` sector; it is not chosen independently for individual vertices. The other global choices fail on almost the entire sector: crossed/$+1$ settles $0/21$ rows, direct/-1 settles $3/21$, and direct/$+1$ settles $2/21$. With the accepted convention, 15 of the 21 `Ec` rows are resolved modulo this single translation. These rows are therefore reported separately from the 161 direct matches.

6.6.2 Exceptional charge-conjugation packaging

We want to dive a little bit deeper in the vertices that needed more care. With the six we talked about in the last section there are two more we want to analyze. Therefore there are eight rows in the complete SMEFT comparison that require one additional translation. They arise from two distinct sources.

Two rows belong to the Weinberg operator and are unrelated to the four-fermion `Ec` sector. FeynRules and FeynPy package the charge-conjugated lepton structure using different fermion orderings. Their comparison therefore uses the antisymmetrised FeynPy combination

$$\mathrm{FeynPy}(\bar{l}_L, l_L) - \mathrm{FeynPy}(l_L, \bar{l}_L). \tag{6.6}$$

The remaining six rows belong to the four-fermion `Ec` sector. For these vertices, the global translation of Equation (6.5) is necessary but not sufficient because equivalent external-leg configurations are assigned to different field signatures by the two implementations. Their comparison therefore uses explicitly specified partner signatures, together with the corresponding relative signs and duplicate-leg symmetries.

These exceptional translations are fixed in advance: the comparison does not search over alternative signatures or signs at acceptance time. Dedicated tests also verify that modifying the prescribed translation destroys the match. They therefore describe a finite set of representation differences rather than adjustable matching conditions, and the corresponding rows are reported separately from the direct canonical-map matches.

6.7 Chirality

There is also another difference between FeynRules and FeynPy concerning how projectors are handled. FeynRules explicitly prints the chiral projectors `ProjM` and `ProjP`, whereas FeynPy uses separate chiral fermion fields: `lL` and `qL` for left-handed doublets, and `eR`, `uR` and `dR` for right-handed singlets. The printed projector is therefore redundant in the final canonical tensor representation, because the same chirality information is already fixed by the external field.

It is nevertheless checked before being removed. For each fermion chain, the projector printed by FeynRules must agree with the chirality implied by its external fields. The relevant relations are

$$\overline{P_L\psi} = \bar{\psi}P_R, \qquad \overline{P_R\psi} = \bar{\psi}P_L, \qquad \gamma^\mu P_L = P_R\gamma^\mu, \qquad \gamma^\mu P_R = P_L\gamma^\mu, \tag{6.7}$$

which determine the allowed projector once the chirality and structure of the chain are fixed. The same check is applied to chains involving charge conjugation.

All 2998 explicit `ProjM`/`ProjP` factors appearing in the FeynRules JSON reference satisfy this test: 2806 occur in ordinary fermion chains and 192 in charge-conjugation chains. A projector inconsistent with the corresponding fields, or a chain that must vanish for chirality reasons, is instead reported as a validation failure.

The projectors are therefore removed only after their information has been verified; no chirality information is lost during canonicalisation.

6.8 Mutation sensitivity

To check that this comparison was not too lenient, I applied mutations to the source to see if the comparison would actually fail. The validation suite

therefore contains mutation tests in which vertices known to be correct are deliberately altered, and the comparison must reject them. The concrete tests are implemented in `models/SMEFT/tests/test_smeft_comparison_sensitivity.py` and `models/SM/tests/test_feynrules_comparison_sensitivity.py`.

For SMEFT, the mutations we applied contain errors such as a rescaled Wilson coefficient, a general sign reversal, incorrect chirality, transposed flavour indices and a missing i factor. Each mutation is detected, whilst the corresponding unmodified lines are accepted.

The Standard Model benchmark contains similar tests concerning chirality, CKM conjugation, the ordering of colour generators, derivative momenta, i-factors and overall signs. These modifications are also rejected.

The mutation tests therefore demonstrate that canonicalisation removes only the specified algebraic and representation differences; it remains sensitive to changes that alter the physical or algebraic content of a vertex.

6.9 Coverage and limits

The SMEFT model declares 303 parameters: 5 Standard Model parameters and 298 Wilson coefficients. Of the 298 Wilson coefficients, 295 appear in at least one validated FeynRules reference vertex.

The five Standard Model parameters (λ, μ_H, and the three Yukawa matrices) are absent because the reference `Ltot` contains only the EFT contribution.

Three EFT coefficients also lie outside the present vertex comparison: `alphaKB`, `alphaR2B`, and `alphaOmuH2`. This is a structural limitation of the reference rather than an omitted operator. The FeynRules export used here contains vertices with three to six external legs, while these three coefficients generate only two-point vertices. They therefore cannot appear in the available comparison.

These coefficients are reported as unvalidated rather than counted as covered. Closing the gap would only require extending the FeynRules reference to include two-point vertices; no change to the canonical comparison itself would be necessary.

This is the complete Wilson-coefficient coverage gap. Every other EFT coefficient contributes to at least one validated reference vertex.

The comparison is further limited to tree-level vertex tensor structures. It does not validate parameter-card semantics, loop-level quantities, or external model formats such as UFO. As noted in Section 6.3, the Standard Model benchmark also excludes the gauge-fixing sector.

6.10 Generality of the comparison machinery

We were interested in having a comparison machinery that was not model specific. In fact the comparison infrastructure is not specific to SMEFT, there is a core that is shared by the three benchmarks listed in Table 6.4.

Model	Result	Model-specific layer
`models/SM`	163/163	sector adapter
`models/UnbrokenSM_BFM`	67/67	unbroken-basis parser
`models/SMEFT`	184/184	EFT coefficient and packaging logic

Table 6.4: Validation benchmarks using the shared comparison machinery.

But applying the machinery to another model requires an adapter for the corresponding FeynRules export, a map between field names, and any model-specific convention substitutions.

For example SMEFT needs a more specialised parser because its reference contains indexed Wilson coefficients, slashed momenta in spinor chains, weak ϵ tensors, and generic index deltas. The subsequent canonicalisation, tensor-map comparison, and chirality validation are, however, shared with the other benchmarks.

6.11 Reproducibility

From the repository root, the complete regression suite is run with

```
.venv/bin/python -m pytest -q
```

and reports 519 passing tests for the final state considered in this thesis.

The SMEFT validation gate is

```
.venv/bin/python -m models.SMEFT.comparison --check --allow-cc-packaging
```

which accepts the documented charge-conjugation and exceptional packaging translations while failing on any unresolved row, symbolic inequality, comparison error, or unexplained FeynPy-only signature.

A stricter diagnostic mode is

```
.venv/bin/python -m models.SMEFT.comparison --check
```

which requires every row to match without packaging translations. It therefore fails for the present reference by construction and is used only to track changes in the set of convention-dependent rows.

The human-readable comparison report can be regenerated with

```
.venv/bin/python -m models.SMEFT.comparison
```

The principal validation outputs are stored in `models/SMEFT/COMPARISON.md` and in the per-signature report `models/SMEFT/comparison/artifacts/vertex_comparison_report.json`, from which the results presented in this chapter are obtained.

6.12 Computational performance

We now discuss the computation times of the two toolkits. The exact figures depend on the computer used; they should therefore be regarded as indicative timings rather than universal benchmarks. Nevertheless, they give a clear idea of the practical difference between FeynRules and FeynPy.

Step	Wall-clock time
FeynRules model loading	~ 15 min
FeynRules reference generation	~ 15 min
FeynPy rule generation, arity 3–6	8.20 s
FeynPy comparison gate	56.91 s

Table 6.5: Indicative runtimes for the SMEFT validation workflow.

The FeynPy rule-generation timing is obtained with `models/SMEFT/generate_feynpy_rules.py`; the comparison timing corresponds to the validation gate of Section 6.11. The dominant cost in producing the reference is therefore the FeynRules workflow: about half an hour is needed before the JSON reference is available. Once the reference exists, FeynPy regenerates the local SMEFT rules and performs the complete algebraic comparison on the scale of seconds to one minute.

6.13 Summary

We can now summarise what we have achieved. The three validation benchmarks confirm the agreement between FeynPy and the corresponding FeynRules references.

For the packaged Standard Model, all 163 exported vertices are in agreement. For the unbroken background-field Standard Model, all 67 exported vertices are in agreement. For SMEFT, all 184 EFT reference lines agree in terms of operator content: 161 are direct mappings of the canonical map, 15 require the single global charge conjugation convention, and 8 require an additional documented packaging translation.

The SMEFT comparison covers 295 of the 298 active Wilson coefficients. The remaining three generate only two-point vertices and therefore do not fall within the scope of the available FeynRules reference.

Finally, deliberate mutation tests confirm that the comparison remains sensitive to various changes. Within these limits, the validation provides a direct algebraic benchmark of the Feynman rules produced by FeynPy. We can therefore state that, within this validation scope, FeynPy correctly reproduces the SM, the unbroken background-field SM, and SMEFT Feynman rules. The timing results also show that, once the model is implemented, FeynPy can regenerate the SMEFT rules in seconds and complete the full validation comparison in about one minute.

Chapter 7

Conclusions and Outlook

7.1 Conclusions

This thesis aimed to implement a toolkit within the Python ecosystem that could manipulate Lagrangians and compute their Feynman rules. The toolkit was intended as an alternative to FeynRules, allowing one to move away from Mathematica while taking advantage of new symbolic-computation capabilities. Within the scope of tree-level vertex derivation and validation, these goals were achieved.

FeynPy allows a model to be defined using Python objects that describe all the necessary physical quantities, such as indices, fields, gauge groups, and parameters, along with a Lagrangian written in a notation similar to that used by FeynRules. From this information, it derives tree-level Feynman rules as Symbolica tensor expressions. The implementation is generic, not constrained to a specific model: covariant derivatives, field strengths, gauge fixing, and ghost sectors are constructed from the model declarations themselves.

A compiled Lagrangian can also be manipulated after its construction. Field transformations are used, for example, to move from gauge to physical bases, while gauge variations and BRST transformations can be evaluated directly.

The main result of the thesis is the implementation of three existing models and the validation of their generated vertices against FeynRules. Comparing two independently derived tensor expressions requires more than simply comparing their literal form. The canonicalisation procedure of Section 6.2 provides a common representation in which these expressions can be compared, so that equality is tested as a tensor identity.

With this procedure, the packaged Standard Model agrees with its FeynRules reference at 163/163 vertices, the unbroken background-field Standard Model at 67/67, and the Green-basis SMEFT at 184/184 reference lines. Of the latter, 161 agree directly, 15 require the global `Ec` charge-

conjugation convention, and 8 require documented row-specific packaging translations. Mutation tests also show that the comparison detects deliberate changes in signs, coefficients, and index structures (Section 6.8).

In this sense, the validation framework is itself part of the result. It gives a reproducible way to decide whether independently generated tensor-valued Feynman rules represent the same interaction.

7.2 Limitations and outlook

The current implementation is limited to tree-level Feynman rules. It does not include loop calculations, counterterms, or renormalization. Furthermore, it produces symbolic vertices rather than external model files, so FeynPy is not yet a direct replacement for FeynRules in a complete phenomenological workflow.

The most immediate practical extension would therefore be a UFO interface. Much of the required information is already available in structured form, and the next step is to translate particles, parameters, and tensor structures into the UFO representation. This would allow FeynPy models to be used directly with tools such as MadGraph5_aMC@NLO and the wider UFO ecosystem.

A second extension concerns the integration-by-parts layer. In its present form it handles unlabelled scalar species only (Section 3.16): it symmetrises identical scalar slots and moves derivatives into a left-normal form. This is enough to decide equivalence modulo total derivatives for scalar operators, but not for terms carrying Dirac bilinears or gauge indices. Extending it to those structures would make a qualitatively new operation available, namely the reduction of a redundant operator basis to a minimal one. The SMEFT model validated here is written in the Green basis precisely because it retains the operators that the equations of motion would remove (Section 2.4.2). An integration-by-parts layer covering fermions and gauge indices, used together with the equations of motion, would allow the same toolkit to carry that model to the Warsaw basis, rather than only deriving vertices in whichever basis it was given.

Finally, more models could be implemented and validated. Models with extended gauge sectors, supersymmetry, or higher-dimensional SMEFT operators would test different parts of the implementation and provide a broader assessment of its generality.

7.3 Closing remark

Deriving the Feynman rules of a Lagrangian is an important step in testing a new theory and studying its implications and limits. We were particularly interested in SMEFT, where the number and variety of possible interactions

is much larger than in the Standard Model. Performing and checking this procedure by hand is therefore no longer realistic. The goal of this work was to achieve a simpler and faster workflow for calculating these fundamental rules.

A large part of the work was in finding a good way to exploit the capabilities of Symbolica while keeping a DSL that is intuitive and sufficiently close to FeynRules. The main challenge was to make the transition from declared Python objects to Symbolica tensor expressions as safe and transparent as possible, while keeping the DSL general and flexible rather than model-specific.

FeynPy combines a Python model description, generic gauge and tensor manipulation, and a canonical validation procedure in a single framework. Its agreement with the FeynRules reference vertices across three complete models provides evidence that this approach works. The result is not yet a replacement for the full FeynRules ecosystem, but it shows that reliable Feynman-rule derivation and validation can be carried out natively within Python.

Bibliography

[1] Neil D. Christensen and Claude Duhr. FeynRules - Feynman rules made easy. *Comput. Phys. Commun.*, 180:1614–1641, 2009.

[2] Adam Alloul, Neil D. Christensen, Céline Degrande, Claude Duhr, and Benjamin Fuks. FeynRules 2.0 - A complete toolbox for tree-level phenomenology. *Comput. Phys. Commun.*, 185:2250–2300, 2014.

[3] Lucien Huber and Ben Ruijl. Spenso: a sparse and dense tensor library with automatic index matching and tensor networks. `https://github.com/alphal00p/spenso`. Accessed 2026.

[4] Ben Ruijl. Symbolica: Modern Computer Algebra. `https://symbolica.io`. Accessed 2026.

[5] Ben Ruijl. Symbolica: modern computer algebra. Seminar, CERN, 2024. `https://indico.cern.ch/event/1449388/`.

Appendix A

A Catalogue of Worked Vertices

This appendix collects a set of vertices derived with FeynPy, ordered roughly by increasing complexity, as declaration/output pairs. Every expression below is the verbatim printed output of `feynman_rule(...)` with `simplify=True`; nothing has been reformatted beyond inserting line breaks. The purpose is twofold: to give a reader an idea of what the output actually looks like before committing to Chapter 3, and to make the notation of Section 6.2 concrete.

Reading the output

Four printed heads account for almost everything.

Printed head	Meaning
`mink(4, mu1)`	Lorentz index of leg 1
`bis(4, i1)`	spinor (bispinor) index of leg 1
`cof(3, c1), coad(8, a1)`	colour fundamental and adjoint indices
`g(...)`	metric, or a Kronecker delta on a gauge index
`t(...), f(...)`	SU(N) generator and structure constant
`gamma(...), gamma5(...)`	γ^μ and γ^5
`pcomp(q1, mu2)`	component μ_2 of the incoming momentum q_1

Legs are numbered in the order they are passed to `feynman_rule`, all momenta are incoming, and every rule carries the overall factor $+i$ of Table 2.1. Index names ending in `_int` or `_canon` are generated dummies and carry no meaning.

A.1 Scalar ϕ^4

```
Phi = Field("Phi", spin=0, self_conjugate=True, symbol=S("phi"))
L   = Model(lam * Phi * Phi * Phi * Phi).lagrangian()
L.feynman_rule(Phi, Phi, Phi, Phi)
```

```
24I*lam
```

The factor $24 = 4!$ is the number of ways of assigning four identical fields to four legs, produced by the permutation sum of Section 5.6 rather than inserted by hand.

A.2 QED

```
Psi    = Field("Psi", spin=Fraction(1,2), self_conjugate=False,
                symbol=S("psi"), conjugate_symbol=S("psibar"),
                indices=(SPINOR_INDEX,), quantum_numbers={"Q": S("Qpsi")})
Photon = Field("A", spin=1, self_conjugate=True,
                symbol=S("A"), indices=(LORENTZ_INDEX,))
U1QED  = GaugeGroup(name="U1QED", abelian=True, coupling=S("e"),
                    gauge_boson=Photon, charge="Q")

L = Model(I * Psi.bar * Gamma(mu) * DC(Psi, mu),
          gauge_groups=(U1QED,)).lagrangian()
L.feynman_rule(Psi.bar, Psi, Photon)
```

```
I*Qpsi*e*gamma(bis(4, i1),bis(4, i2),mink(4, mu3))
```

That is $iQ_\psi e\,\gamma^{\mu_3}$, the vertex derived by hand in Section 2.5. Note that the interaction was never written: it came out of the expansion of `DC`.

A.3 Scalar QED

```
# Photon and U1QED are those of the QED example above
PhiC = Field("PhiC", spin=0, self_conjugate=False,
             symbol=S("phiC"), conjugate_symbol=S("phiCdag"),
             quantum_numbers={"Q": S("Qphi")})

L = Model(DC(PhiC.bar, mu) * DC(PhiC, mu),
          gauge_groups=(U1QED,)).lagrangian()
```

```
L.feynman_rule(PhiC.bar, PhiC, Photon)            # current
L.feynman_rule(PhiC.bar, PhiC, Photon, Photon)    # seagull
```

```
-I*e*Qphi*pcomp(q1,mu3) + I*e*Qphi*pcomp(q2,mu3)

2I*e^2*Qphi^2*g(mink(4, mu3),mink(4, mu4))
```

The first is $-ieQ_\phi(q_1 - q_2)^{\mu_3}$; the second is the seagull $2ie^2Q_\phi^2 g^{\mu_3\mu_4}$, which arises from the term quadratic in the gauge field produced when both covariant derivatives are expanded. One declared term, two vertices of different arity.

A.4 QCD

The declaration is the one given in full in Section 3.11. Its four terms produce the following.

Quark–gluon

```
I*gs*gamma(bis(4, i1),bis(4, i2),mink(4, mu3))
   *t(coad(8, a3),cof(3, c1),cof(3, c2))
```

$ig_s\gamma^{\mu_3}t^{a_3}_{c_1c_2}$.

Three-gluon

```
-gs*g(mink(4,mu3),mink(4,mu1))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q1,mu2)
+gs*g(mink(4,mu3),mink(4,mu1))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q3,mu2)
+gs*g(mink(4,mu3),mink(4,mu2))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q2,mu1)
-gs*g(mink(4,mu3),mink(4,mu2))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q3,mu1)
+gs*g(mink(4,mu1),mink(4,mu2))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q1,mu3)
-gs*g(mink(4,mu1),mink(4,mu2))*f(coad(8,a1),coad(8,a2),coad(8,a3))
    *pcomp(q2,mu3)
```

Collecting the six terms in pairs gives the textbook form

$$g_s f^{a_1a_2a_3}\Big[g^{\mu_1\mu_2}(q_1 - q_2)^{\mu_3} + g^{\mu_2\mu_3}(q_2 - q_3)^{\mu_1} + g^{\mu_3\mu_1}(q_3 - q_1)^{\mu_2}\Big].$$

Four-gluon

```
-I*gs^2*g(mink(4,mu3),mink(4,mu4))*g(mink(4,mu1),mink(4,mu2))
       *f(coad(8,a),coad(8,a1),coad(8,a3))*f(coad(8,a),coad(8,a2),coad(8
       ↪  ,a4))
-I*gs^2*g(mink(4,mu3),mink(4,mu4))*g(mink(4,mu1),mink(4,mu2))
       *f(coad(8,a),coad(8,a2),coad(8,a3))*f(coad(8,a),coad(8,a1),coad(8
       ↪  ,a4))
-I*gs^2*g(mink(4,mu3),mink(4,mu1))*g(mink(4,mu4),mink(4,mu2))
       *f(coad(8,a),coad(8,a3),coad(8,a4))*f(coad(8,a),coad(8,a1),coad(8
       ↪  ,a2))
+I*gs^2*g(mink(4,mu3),mink(4,mu1))*g(mink(4,mu4),mink(4,mu2))
       *f(coad(8,a),coad(8,a2),coad(8,a3))*f(coad(8,a),coad(8,a1),coad(8
       ↪  ,a4))
+I*gs^2*g(mink(4,mu3),mink(4,mu2))*g(mink(4,mu4),mink(4,mu1))
       *f(coad(8,a),coad(8,a3),coad(8,a4))*f(coad(8,a),coad(8,a1),coad(8
       ↪  ,a2))
+I*gs^2*g(mink(4,mu3),mink(4,mu2))*g(mink(4,mu4),mink(4,mu1))
       *f(coad(8,a),coad(8,a1),coad(8,a3))*f(coad(8,a),coad(8,a2),coad(8
       ↪  ,a4))
```

The summed adjoint index `a` is the contraction between the two structure constants. This vertex is a good illustration of why the canonicalisation of Section 6.2 needs a controlled use of the Jacobi identity: the same expression can be written with different pairs of f's, and only a fixed convention makes two derivations comparable.

Ghost–gluon

```
gs*f(coad(8, a1),coad(8, a3),coad(8, a2))*pcomp(q1,mu3)
```

$g_s f^{a_1 a_3 a_2} q_1^{\mu_3}$, carrying the momentum of the antighost leg, as expected from $\mathcal{L}_{\text{gh}}$ in Equation (2.43).

The gluon two-point function

```
 I*g(coad(8,a1),coad(8,a2))*pcomp(q1,mu1)*pcomp(q2,mu2)/xi
+I*g(mink(4,mu1),mink(4,mu2))*g(coad(8,a1),coad(8,a2))
    *pcomp(q1,mu1_int)*pcomp(q2,mu1_int)
-I*g(coad(8,a1),coad(8,a2))*pcomp(q1,mu2)*pcomp(q2,mu1)
```

The gauge parameter ξ appears explicitly, so setting $\xi = 1$ recovers the Feynman-gauge inverse propagator.

A.5 Flavour-expanded Yukawa

```
Gen = flavor_index("Generation", 3, prefix="f")
lep = Field("l", spin=Fraction(1,2), self_conjugate=False,
            indices=(Gen, SPINOR_INDEX), symbol=S("l"),
            conjugate_symbol=S("lbar"), flavor_index=Gen,
            class_members=("e", "mu", "ta"))
Y   = Parameter("Y", indices=(Gen, Gen))
e, mu_, ta = lep.class_members

L = Model(Y(f1, f2) * lep.bar(f1) * lep(f2) * Phi).lagrangian()
```

Without expansion the rule is generation-generic,

```
I*g(bis(4, i1),bis(4, i2))*Y(f1,f2)
```

and with `flavor_expand=True` the same declaration yields nine explicit vertices:

```
ebar  e   Phi -> I*g(bis(4,i1),bis(4,i2))*Y(1,1)
ebar  mu  Phi -> I*g(bis(4,i1),bis(4,i2))*Y(1,2)
ebar  ta  Phi -> I*g(bis(4,i1),bis(4,i2))*Y(1,3)
mubar e   Phi -> I*g(bis(4,i1),bis(4,i2))*Y(2,1)
mubar mu  Phi -> I*g(bis(4,i1),bis(4,i2))*Y(2,2)
mubar ta  Phi -> I*g(bis(4,i1),bis(4,i2))*Y(2,3)
tabar e   Phi -> I*g(bis(4,i1),bis(4,i2))*Y(3,1)
tabar mu  Phi -> I*g(bis(4,i1),bis(4,i2))*Y(3,2)
tabar ta  Phi -> I*g(bis(4,i1),bis(4,i2))*Y(3,3)
```

Had `Y` been declared with `components` setting its off-diagonal entries to zero, the six off-diagonal rules would simply not exist.

A.6 Electroweak vertices from the packaged Standard Model

The vertices below come from `build_standard_model()` (Section 3.20), so they are already in the physical basis produced by the field transformations of Section 3.13.

```
sm = build_standard_model()
L  = sm.lagrangian
F  = sm.fields
```

Triple gauge coupling AW^-W^+

```
 I*ee*g(mink(4,mu1),mink(4,mu2))*pcomp(q1,mu3)
-I*ee*g(mink(4,mu1),mink(4,mu2))*pcomp(q2,mu3)
-I*ee*g(mink(4,mu1),mink(4,mu3))*pcomp(q1,mu2)
+I*ee*g(mink(4,mu1),mink(4,mu3))*pcomp(q3,mu2)
+I*ee*g(mink(4,mu2),mink(4,mu3))*pcomp(q2,mu1)
-I*ee*g(mink(4,mu2),mink(4,mu3))*pcomp(q3,mu1)
```

Structurally identical to the three-gluon vertex above, with the structure constant replaced by the electric charge e — which is what the W^3–B mixing of Equation (2.49) does to the $\mathrm{SU}(2)_L$ self-coupling.

Higgs–W–W and the triple Higgs coupling

```
L.feynman_rule(F.H, F.W.bar, F.W)  ->
↪  I/2*vev*ee^2*g(mink(4,mu2),mink(4,mu3))/sw^2

L.feynman_rule(F.H, F.H, F.H)      ->  -6I*lam*vev
```

Both are proportional to the vacuum expectation value, as they must be: they exist only because Φ was shifted by v in Equation (2.48).

Charged current $W^+\bar{u}d$

```
 I/4*(1/2)^(1/2)*ee*g(cof(3,c2),cof(3,c3))
    *gamma(bis(4,i2),bis(4,i3),mink(4,mu1))*CKM(fl2,fl3)/sw
-I/4*(1/2)^(1/2)*ee*g(cof(3,c2),cof(3,c3))
    *gamma(bis(4,i2),bis(4,bis_canon_2),mink(4,mu1))
    *gamma5(bis(4,bis_canon_2),bis(4,i3))*CKM(fl2,fl3)/sw
+ ... (two further terms)
```

The CKM matrix appears exactly where Section 2.3.7 says it should, contracted on the two quark flavour indices. The four terms are the expansion of the chiral projector pair $P_L \otimes P_L$ into $\frac{1}{4}(1 \mp \gamma^5) \otimes (1 \mp \gamma^5)$, which is the printed form the toolkit uses; recombining them is the chirality question examined in Section 6.7. The neutral currents $A\bar{\ell}\ell$ and $Z\bar{\nu}\nu$ have the same shape, with the electric-charge and s_W/c_W combinations in place of the CKM matrix.

Appendix B

SMEFT Per-Row Comparison Results

This appendix lists the exact-symbolic verdict for every FeynRules reference vertex in the SMEFT comparison of Section 6.5. It is generated directly from `models/SMEFT/comparison/artifacts/vertex_comparison_report.json`, so it cannot drift out of step with the comparison itself.

The verdict column uses the abbreviations of Table 6.3: *direct* is canonical tensor-map equality with no packaging assumption; *global Ec* additionally uses the single global evanescent charge-conjugation convention of Section 6.6; and *pinned CC* additionally uses one explicitly tabulated row-specific transform.

The 184 rows break down as 161 direct, 15 global Ec, 8 pinned CC.

Arity	External fields	Family	Verdict
3	`B|Phi|Phibar`	bosonic	direct
3	`B|dR|dRbar`	2-fermion	direct
3	`B|eR|eRbar`	2-fermion	direct
3	`B|lL|lLbar`	2-fermion	direct
3	`B|qL|qLbar`	2-fermion	direct
3	`B|uR|uRbar`	2-fermion	direct
3	`G|G|G`	bosonic	direct
3	`G|dR|dRbar`	2-fermion	direct
3	`G|qL|qLbar`	2-fermion	direct
3	`G|uR|uRbar`	2-fermion	direct
3	`Phibar|dRbar|qL`	2-fermion	direct
3	`Phibar|eRbar|lL`	2-fermion	direct
3	`Phibar|qLbar|uR`	2-fermion	direct
3	`Phi|Phibar|Wi`	bosonic	direct
3	`Phi|dR|qLbar`	2-fermion	direct
3	`Phi|eR|lLbar`	2-fermion	direct
3	`Phi|qL|uRbar`	2-fermion	direct
3	`Wi|Wi|Wi`	bosonic	direct
3	`Wi|lL|lLbar`	2-fermion	direct

Arity	External fields	Family	Verdict
3	`Wi|qL|qLbar`	2-fermion	direct
4	`B|B|Phi|Phibar`	bosonic	direct
4	`B|B|dR|dRbar`	2-fermion	direct
4	`B|B|eR|eRbar`	2-fermion	direct
4	`B|B|lL|lLbar`	2-fermion	direct
4	`B|B|qL|qLbar`	2-fermion	direct
4	`B|B|uR|uRbar`	2-fermion	direct
4	`B|G|dR|dRbar`	2-fermion	direct
4	`B|G|qL|qLbar`	2-fermion	direct
4	`B|G|uR|uRbar`	2-fermion	direct
4	`B|Phibar|dRbar|qL`	2-fermion	direct
4	`B|Phibar|eRbar|lL`	2-fermion	direct
4	`B|Phibar|qLbar|uR`	2-fermion	direct
4	`B|Phi|Phibar|Wi`	bosonic	direct
4	`B|Phi|dR|qLbar`	2-fermion	direct
4	`B|Phi|eR|lLbar`	2-fermion	direct
4	`B|Phi|qL|uRbar`	2-fermion	direct
4	`B|Wi|lL|lLbar`	2-fermion	direct
4	`B|Wi|qL|qLbar`	2-fermion	direct
4	`G|G|G|G`	bosonic	direct
4	`G|G|Phi|Phibar`	bosonic	direct
4	`G|G|dR|dRbar`	2-fermion	direct
4	`G|G|qL|qLbar`	2-fermion	direct
4	`G|G|uR|uRbar`	2-fermion	direct
4	`G|Phibar|dRbar|qL`	2-fermion	direct
4	`G|Phibar|qLbar|uR`	2-fermion	direct
4	`G|Phi|dR|qLbar`	2-fermion	direct
4	`G|Phi|qL|uRbar`	2-fermion	direct
4	`G|Wi|qL|qLbar`	2-fermion	direct
4	`Phibar|Phibar|dRbar|uR`	2-fermion	direct
4	`Phibar|Phibar|lLbar|lLbar`	2-fermion	pinned CC
4	`Phibar|Wi|dRbar|qL`	2-fermion	direct
4	`Phibar|Wi|eRbar|lL`	2-fermion	direct
4	`Phibar|Wi|qLbar|uR`	2-fermion	direct
4	`Phi|Phibar|Wi|Wi`	bosonic	direct
4	`Phi|Phibar|dR|dRbar`	2-fermion	direct
4	`Phi|Phibar|eR|eRbar`	2-fermion	direct
4	`Phi|Phibar|lL|lLbar`	2-fermion	direct
4	`Phi|Phibar|qL|qLbar`	2-fermion	direct
4	`Phi|Phibar|uR|uRbar`	2-fermion	direct
4	`Phi|Phi|Phibar|Phibar`	bosonic	direct
4	`Phi|Phi|dR|uRbar`	2-fermion	direct
4	`Phi|Phi|lL|lL`	2-fermion	pinned CC
4	`Phi|Wi|dR|qLbar`	2-fermion	direct
4	`Phi|Wi|eR|lLbar`	2-fermion	direct
4	`Phi|Wi|qL|uRbar`	2-fermion	direct
4	`Wi|Wi|Wi|Wi`	bosonic	direct
4	`Wi|Wi|lL|lLbar`	2-fermion	direct
4	`Wi|Wi|qL|qLbar`	2-fermion	direct
4	`dRbar|eR|lLbar|qL`	4-fermion	pinned CC
4	`dRbar|qL|qL|uRbar`	4-fermion	pinned CC

Arity	External fields	Family	Verdict
4	dR|dRbar|eR|eRbar	4-fermion	global Ec
4	dR|dRbar|lL|lLbar	4-fermion	global Ec
4	dR|dRbar|qL|qLbar	4-fermion	global Ec
4	dR|dRbar|uR|uRbar	4-fermion	global Ec
4	dR|dR|dRbar|dRbar	4-fermion	global Ec
4	dR|eRbar|lL|qLbar	4-fermion	pinned CC
4	dR|qLbar|qLbar|uR	4-fermion	pinned CC
4	eRbar|lL|qL|uRbar	4-fermion	pinned CC
4	eR|eRbar|lL|lLbar	4-fermion	global Ec
4	eR|eRbar|qL|qLbar	4-fermion	global Ec
4	eR|eRbar|uR|uRbar	4-fermion	global Ec
4	eR|eR|eRbar|eRbar	4-fermion	global Ec
4	eR|lLbar|qLbar|uR	4-fermion	pinned CC
4	lL|lLbar|qL|qLbar	4-fermion	global Ec
4	lL|lLbar|uR|uRbar	4-fermion	global Ec
4	lL|lL|lLbar|lLbar	4-fermion	global Ec
4	qL|qLbar|uR|uRbar	4-fermion	global Ec
4	qL|qL|qLbar|qLbar	4-fermion	global Ec
4	uR|uR|uRbar|uRbar	4-fermion	global Ec
5	B|B|B|Phi|Phibar	bosonic	direct
5	B|B|B|dR|dRbar	2-fermion	direct
5	B|B|B|eR|eRbar	2-fermion	direct
5	B|B|B|lL|lLbar	2-fermion	direct
5	B|B|B|qL|qLbar	2-fermion	direct
5	B|B|B|uR|uRbar	2-fermion	direct
5	B|B|G|dR|dRbar	2-fermion	direct
5	B|B|G|qL|qLbar	2-fermion	direct
5	B|B|G|uR|uRbar	2-fermion	direct
5	B|B|Phibar|dRbar|qL	2-fermion	direct
5	B|B|Phibar|eRbar|lL	2-fermion	direct
5	B|B|Phibar|qLbar|uR	2-fermion	direct
5	B|B|Phi|Phibar|Wi	bosonic	direct
5	B|B|Phi|dR|qLbar	2-fermion	direct
5	B|B|Phi|eR|lLbar	2-fermion	direct
5	B|B|Phi|qL|uRbar	2-fermion	direct
5	B|B|Wi|lL|lLbar	2-fermion	direct
5	B|B|Wi|qL|qLbar	2-fermion	direct
5	B|G|G|dR|dRbar	2-fermion	direct
5	B|G|G|qL|qLbar	2-fermion	direct
5	B|G|G|uR|uRbar	2-fermion	direct
5	B|G|Phibar|dRbar|qL	2-fermion	direct
5	B|G|Phibar|qLbar|uR	2-fermion	direct
5	B|G|Phi|dR|qLbar	2-fermion	direct
5	B|G|Phi|qL|uRbar	2-fermion	direct
5	B|G|Wi|qL|qLbar	2-fermion	direct
5	B|Phibar|Wi|dRbar|qL	2-fermion	direct
5	B|Phibar|Wi|eRbar|lL	2-fermion	direct
5	B|Phibar|Wi|qLbar|uR	2-fermion	direct
5	B|Phi|Phibar|Wi|Wi	bosonic	direct
5	B|Phi|Phibar|dR|dRbar	2-fermion	direct
5	B|Phi|Phibar|eR|eRbar	2-fermion	direct

Arity	External fields	Family	Verdict
5	B|Phi|Phibar|lL|lLbar	2-fermion	direct
5	B|Phi|Phibar|qL|qLbar	2-fermion	direct
5	B|Phi|Phibar|uR|uRbar	2-fermion	direct
5	B|Phi|Phi|Phibar|Phibar	bosonic	direct
5	B|Phi|Wi|dR|qLbar	2-fermion	direct
5	B|Phi|Wi|eR|lLbar	2-fermion	direct
5	B|Phi|Wi|qL|uRbar	2-fermion	direct
5	B|Wi|Wi|lL|lLbar	2-fermion	direct
5	B|Wi|Wi|qL|qLbar	2-fermion	direct
5	G|G|G|G|G	bosonic	direct
5	G|G|G|Phi|Phibar	bosonic	direct
5	G|G|G|dR|dRbar	2-fermion	direct
5	G|G|G|qL|qLbar	2-fermion	direct
5	G|G|G|uR|uRbar	2-fermion	direct
5	G|G|Phibar|dRbar|qL	2-fermion	direct
5	G|G|Phibar|qLbar|uR	2-fermion	direct
5	G|G|Phi|dR|qLbar	2-fermion	direct
5	G|G|Phi|qL|uRbar	2-fermion	direct
5	G|G|Wi|qL|qLbar	2-fermion	direct
5	G|Phibar|Wi|dRbar|qL	2-fermion	direct
5	G|Phibar|Wi|qLbar|uR	2-fermion	direct
5	G|Phi|Phibar|dR|dRbar	2-fermion	direct
5	G|Phi|Phibar|qL|qLbar	2-fermion	direct
5	G|Phi|Phibar|uR|uRbar	2-fermion	direct
5	G|Phi|Wi|dR|qLbar	2-fermion	direct
5	G|Phi|Wi|qL|uRbar	2-fermion	direct
5	G|Wi|Wi|qL|qLbar	2-fermion	direct
5	Phibar|Phibar|Wi|dRbar|uR	2-fermion	direct
5	Phibar|Wi|Wi|dRbar|qL	2-fermion	direct
5	Phibar|Wi|Wi|eRbar|lL	2-fermion	direct
5	Phibar|Wi|Wi|qLbar|uR	2-fermion	direct
5	Phi|Phibar|Phibar|dRbar|qL	2-fermion	direct
5	Phi|Phibar|Phibar|eRbar|lL	2-fermion	direct
5	Phi|Phibar|Phibar|qLbar|uR	2-fermion	direct
5	Phi|Phibar|Wi|Wi|Wi	bosonic	direct
5	Phi|Phibar|Wi|dR|dRbar	2-fermion	direct
5	Phi|Phibar|Wi|eR|eRbar	2-fermion	direct
5	Phi|Phibar|Wi|lL|lLbar	2-fermion	direct
5	Phi|Phibar|Wi|qL|qLbar	2-fermion	direct
5	Phi|Phibar|Wi|uR|uRbar	2-fermion	direct
5	Phi|Phi|Phibar|Phibar|Wi	bosonic	direct
5	Phi|Phi|Phibar|dR|qLbar	2-fermion	direct
5	Phi|Phi|Phibar|eR|lLbar	2-fermion	direct
5	Phi|Phi|Phibar|qL|uRbar	2-fermion	direct
5	Phi|Phi|Wi|dR|uRbar	2-fermion	direct
5	Phi|Wi|Wi|dR|qLbar	2-fermion	direct
5	Phi|Wi|Wi|eR|lLbar	2-fermion	direct
5	Phi|Wi|Wi|qL|uRbar	2-fermion	direct
5	Wi|Wi|Wi|Wi|Wi	bosonic	direct
5	Wi|Wi|Wi|lL|lLbar	2-fermion	direct
5	Wi|Wi|Wi|qL|qLbar	2-fermion	direct

Arity	External fields	Family	Verdict
6	B|B|B|B|Phi|Phibar	bosonic	direct
6	B|B|B|Phi|Phibar|Wi	bosonic	direct
6	B|B|Phi|Phibar|Wi|Wi	bosonic	direct
6	B|B|Phi|Phi|Phibar|Phibar	bosonic	direct
6	B|Phi|Phibar|Wi|Wi|Wi	bosonic	direct
6	B|Phi|Phi|Phibar|Phibar|Wi	bosonic	direct
6	G|G|G|G|G|G	bosonic	direct
6	G|G|G|G|Phi|Phibar	bosonic	direct
6	Phi|Phibar|Wi|Wi|Wi|Wi	bosonic	direct
6	Phi|Phi|Phibar|Phibar|Wi|Wi	bosonic	direct
6	Phi|Phi|Phi|Phibar|Phibar|Phibar	bosonic	direct
6	Wi|Wi|Wi|Wi|Wi|Wi	bosonic	direct

Consistency check against Table 6.3: 161 direct, 15 global Ec, 8 pinned CC, 0 unresolved, out of 184 supported rows.